

**An
Assignment Report
For
Pearson BTEC International Level 4 HND**

**In
Database Design and Development
(Unit 04)**

Unit Tutor: *Ms. Jency Varghese*

**Submitted By
(*Youssef Rabie Elkhoully*)
Student ID: “2510293”**



British Applied College
الكلية البريطانية التطبيقية

**School of Computing
British Applied College, Umm Al Quwain**

Table of Contents

What do we mean by Database?	5
My Role and Current situation?.....	5
Designing an Efficient Hospital Database System	6
The Purpose?	6
Problem in the Current System?.....	6
Database Concept?	7
User Requirements?.....	8
System Requirements?	8
Relationships Between Entities?	9
ER diagram to represent the Case?	10
Modernising Hospital Data Management: Comprehensive Database System Design	12
Firstly: System Planning and Theoretical Design.....	12
Secondly: System Implementation and Practical Design	13
Thirdly: System Functionality, Output Design, and Data Validation.....	41
Fourthly: Data Normalisation (1NF, 2NF, 3NF).....	45
Hospital Management System Design Evaluation	46
Evaluation of RDBMS Usage	47
Evaluation of Table Design.....	48
Evaluation of User Interface Design	49
Evaluation of Data Handling (CRUD Operations)	50
Evaluation of Output Design	50
Evaluation of Data Validation.....	51
Limitations and Suggested Improvements	51
Implementing the Hospital Database System From Design to Reality	53
Introduction to MySQL and Development Tools	53
MySQL Workbench Implementation.....	54
Data Validation	69
SQL Scripts Section.....	71
Final Evidence Mapping.....	72
Final System Diagram	72
Moreover, Query Implementation in a Relational Database System for Efficient Data Management	73
Overview of SQL Queries Used.....	73
Data Retrieval Queries (SELECT & WHERE)	74
Sorting and Filtering Logic.....	74

Multi-Table Queries (JOIN Operations)	74
Data Manipulation Queries (INSERT, UPDATE, DELETE)	75
Error Handling and Data Validation.....	75
Implementation of Advanced SQL JOIN Queries	76
Implementation of Database Security and Maintenance Strategies.....	77
Database Security Implementation	77
Data Validation as a Security Measure	78
Database Maintenance Practices	78
Space for Improvement and Security Assessment.....	79
Meaningful Analysis of Queries for Management Decision-Making	80
Practical Testing of the Database System Against User and System Requirements	82
Testing Approach	82
Test Plan Followed	82
Testing Results.....	84
Measured Evaluation of Testing Effectiveness and Test Data Selection ..	89
Evaluation of Testing Quality	89
Justification of Test Data Used.....	89
Effectiveness of Testing.....	90
Limitations of Testing	90
Suggested Improvements “Testing”	91
Critical Evaluation of the Hospital Database System “Design Depth and Decision Impact”	92
Evaluation Against User Requirements	92
Evaluation Against System Requirements	93
Strengths, Weaknesses, and Suggested Improvements	94
Overall	95
Professional Technical and User Documentation for the Hospital Database System	96
Firstly: Technical Documentation	96
Secondly: User Documentation	100
Modelled Architecture and Data Flow of the Hospital Database System	102
System Architecture Overview.....	102
Data Flow Diagram	103
System Flowchart.....	104
Detailed Explanation of System Operation	105
Overall System Operation Summary.....	105

Deep Evaluation and Justified Improvements of the Hospital Database System.....	105
Detailed Evaluation Based on Implemented System.....	106
Then Justified Improvements based on my system	107
Final Word	108
Sources and References used.....	108
<i>The End</i>	109

Database

Design and Implementation

What do we mean by Database?

Database is an organized collection of data that is stored in a structured way so it can be easily accessed, managed, and updated. Instead of keeping information in scattered files or spreadsheets, a database keeps everything in one system where data is connected and controlled. In most modern systems, especially relational databases, data is stored in tables made up of rows and columns, where each table represents a specific type of information, such as patients or doctors. These tables are linked using keys, which helps avoid data duplication and keeps the information consistent. Database Management Systems (DBMS), such as MySQL, allow users to create, edit, and retrieve data using simple commands, making the system faster, more accurate, and more secure. Overall, databases are important because they help organizations handle large amounts of data efficiently and support better decision-making. (*Connolly and Begg, 2014; Kroenke and Auer, 2012*)

My Role and Current situation?

Al Noor Care Hospital (ANCH) in Abu Dhabi is a medium-sized hospital that manages patients, doctors, and staff across different departments. Currently, the hospital uses an Excel-based system to store data, but this has become inefficient as the amount of data has increased. It is difficult to manage records, avoid errors, and quickly retrieve information, which affects the overall performance of the hospital.

The aim is to design and develop a relational database system that improves data management in the hospital. The new system will organize data into structured tables, make it easier to access and update information, and reduce errors. It will include key entities such as patients, doctors, departments, and appointments, and support basic operations like data entry, searching, and reporting.

As a Junior Database Developer, my role is to design and develop a relational database system that meets the needs of Al Noor Care Hospital. This includes understanding the current problems in the Excel-based system and identifying what users need from the new

system. I am responsible for creating a structured database with multiple related tables, such as patients, doctors, departments, and appointments, while ensuring data is accurate, consistent, and easy to access.

In addition, I will implement the database using SQL, apply data validation rules to prevent errors, and create queries to retrieve useful information for the hospital staff. I will also test the system to make sure it meets both user and system requirements, and finally provide proper documentation to explain how the system works. Overall, my role is to build a reliable, efficient, and user-friendly database solution that improves how the hospital manages its data.

Designing an Efficient Hospital Database System

ANCH

The Purpose?

Providing an efficient and reliable way to manage data is the main purpose of this database system for Al Noor Care Hospital. Currently, the hospital uses Excel sheets, which makes it difficult to store large amounts of data, avoid errors, and quickly access information. This system will replace the manual method with a structured relational database that organizes data into connected tables.

The new database will help hospital staff to store, update, and retrieve information easily. It will also reduce data duplication, improve accuracy, and support faster decision-making. Overall, the system aims to make hospital operations more efficient and reliable.

Problem in the Current System?

Al Noor Care Hospital is facing several problems due to using an Excel-based system. As the number of patients and staff increases, it becomes harder to manage the data properly. There is no strong connection between the data, which leads to duplication and inconsistency.

Another issue is that searching and updating records takes time and can cause errors. For example, patient information may not be updated correctly, or data may be stored in multiple places. These problems show the need for a better system, such as a relational database, which can handle large data more efficiently and securely.

Database Concept?

A database is a structured system used to store and manage data efficiently. There are different types of databases, such as hierarchical, network, and relational databases, but the most commonly used type is the relational database. In a relational database, data is stored in tables that are connected using relationships. Each table has rows and columns, and data is linked using primary keys and foreign keys. Primary keys uniquely identify each record, while foreign keys create connections between tables. Data types are used to define the kind of data stored, such as numbers, text, or dates. Relationships between tables can be one-to-one, one-to-many, or many-to-many. To improve data quality, normalization is used, which organizes data into different normal forms (1NF, 2NF, 3NF) to reduce redundancy and ensure consistency. (Churcher, 2012; Flejoles, 2018)

Concept	Explanation
Database	A system used to store and manage data in an organized way
Database Types	Hierarchical, Network, Relational (most common)
Relational Database	Stores data in tables that are connected using relationships
How it Works	Data is stored in rows and columns and linked using keys
Primary Key	Unique identifier for each record in a table
Foreign Key	A field that links one table to another
Data Types	Define type of data (INT, VARCHAR, DATE, etc.)
Relationships	One-to-One, One-to-Many, Many-to-Many
Normalisation	Process of organizing data to reduce duplication
1NF	Remove repeating groups, atomic values
2NF	Remove partial dependency
3NF	Remove transitive dependency

User Requirements?

The main users of this system are hospital staff such as receptionists, doctors, nurses, and administrators. Each user interacts with the system in different ways. For example, receptionists need to add and update patient records, schedule appointments, and search for patient information. Doctors need to view patient details and appointment records, while administrators manage staff, departments, and overall system data.

The system must store important data such as patient information, doctor details, department data, room allocation, and appointment records. Users should be able to easily enter new data, update existing records, search for information, and generate basic reports. The system should also ensure that only correct and valid data is entered.

System Requirements?

The system requirements define what the database should do and how it should perform. These are divided into functional and non-functional requirements. Functional requirements focus on the actions the system must perform, such as adding, updating, deleting, and searching data. Non-functional requirements focus on system quality, such as performance, security, and usability.

Type	Requirement
Functional	Add new patient records
	Update doctor and staff details
	Delete incorrect records
	Search for patients and appointments
	Generate reports
Non-Functional	System should be fast and responsive
	Data must be secure
	System should be easy to use
	Prevent invalid data entry

(Microsoft, 2022)

Relationships Between Entities?

In a relational database, tables are connected using relationships. The most common type is one-to-many, where one record in a table is related to multiple records in another table. These relationships are created using foreign keys, which help maintain data consistency and integrity.

	Entity 1	Relationship Type	Entity 2	Description
1	Department	One-to-Many (1:M)	Doctor	One department can have many doctors, but each doctor belongs to one department
2	Doctor	One-to-Many (1:M)	Patient	One doctor can treat many patients, but each patient is assigned to one doctor
3	Doctor	One-to-Many (1:M)	Appointment	One doctor can have many appointments, but each appointment is linked to one doctor
4	Room	One-to-Many (1:M)	Patient	One room can have multiple patients over time, but each patient is assigned to one room
5	Nurse	One-to-Many (1:M)	Patient	One nurse can take care of many patients, but each patient is assigned to one nurse
6	Doctor	One-to-Many (1:M)	Lab	One doctor can manage multiple lab records, but each lab is linked to one doctor

ER diagram to represent the Case?

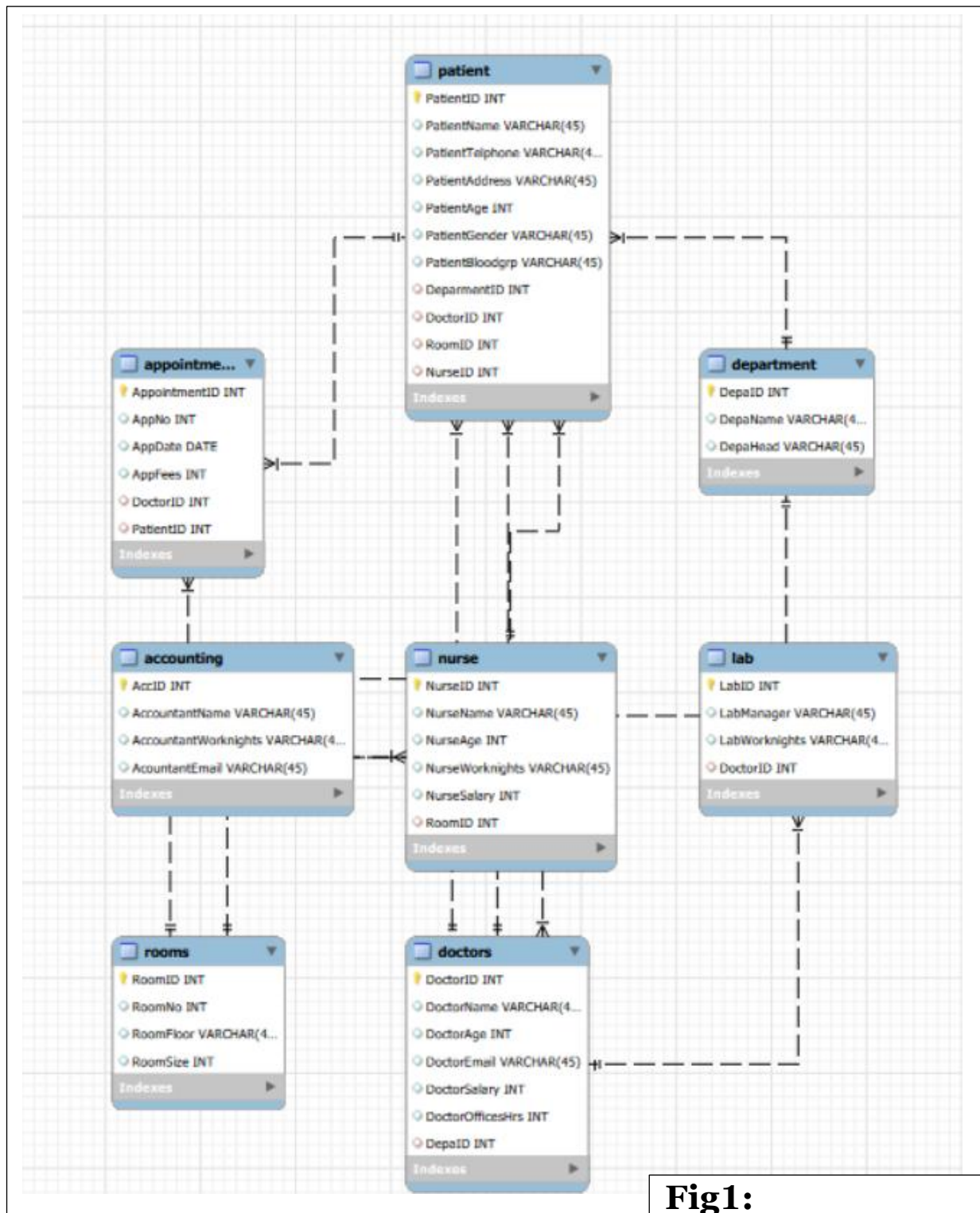


Fig1:

ER diagram for the current Hospital management system

What is ER Diagram?

It represents the structure of the database for Al Noor Care Hospital. It shows how different entities such as patient, doctor, department, nurse, room, appointment, lab, and accounting are connected with each other. Each entity is shown as a table with its attributes, where the primary key (PK) uniquely identifies each record, and foreign keys (FK) are used to create relationships between tables. This structure helps organize the data in a clear and logical way.

The reason for using an ER diagram is to visually design the database before implementation. It helps to understand how data flows between different parts of the system and ensures that all entities are properly connected. For example, the patient table is linked to doctor, department, room, and nurse using foreign keys, which allows the system to store complete patient information without duplication. Similarly, the doctor table is connected to department, appointment, and lab, which reflects real hospital operations where doctors belong to departments and handle appointments and lab activities. (*Visual Paradigm, n.d.; Microsoft, 2022*)

Why this ER Diagram is Effective?

ER diagram is effective because it follows a relational structure where each entity has a clear role and is connected logically to others. It uses one-to-many relationships in most cases, which is suitable for a hospital system. For example, one doctor can treat many patients, and one department can have many doctors. This reduces redundancy and improves data consistency.

Also, the design supports normalization by separating data into different tables instead of storing everything in one place. This makes the database more efficient, easier to manage, and less prone to errors. Overall, this ER diagram provides a strong foundation for building a functional database system that meets the hospital's needs.

Modernising Hospital Data Management: Comprehensive Database System Design

Managing large amounts of data in healthcare systems is a critical task that requires accuracy, speed, and reliability. In modern hospitals, traditional tools such as Excel are no longer sufficient to handle complex and growing datasets. As the number of patients, staff, and services increases, there is a strong need for a structured and efficient system that can organize data, reduce errors, and support daily operations.

In this section, a comprehensive design of a fully functional relational database system is presented for Al Noor Care Hospital. The design focuses not only on the database structure but also on how users interact with the system, how data is validated, and how output is generated. This ensures that the system is practical, user-friendly, and ready for real-world implementation.

Firstly: System Planning and Theoretical Design

This section represents the planning stage of the system, where the overall structure and user interaction are designed before actual implementation. Based on the scenario of Al Noor Care Hospital, the system is designed to replace the current Excel-based method with a more organized and efficient database solution.

The system follows a simple workflow. First, users must **log** into the system using their hospital ID and password, which ensures security and controlled access. After logging in, users are directed to the **homepage** where they can select different services such as managing **doctors**, **patients**, **nurses**, **appointments**, and **other** hospital data. Each section provides forms that allow users to insert, update, delete, and view records. The system is designed to be user-friendly, with clear navigation and structured forms that make data entry easy and efficient. This design ensures that hospital staff can perform their tasks quickly while maintaining data accuracy and consistency. *(Connolly and Begg, 2014)*

Secondly: System Implementation and Practical Design

(Kroenke and Auer, 2012)

A. Interface Design – Login System

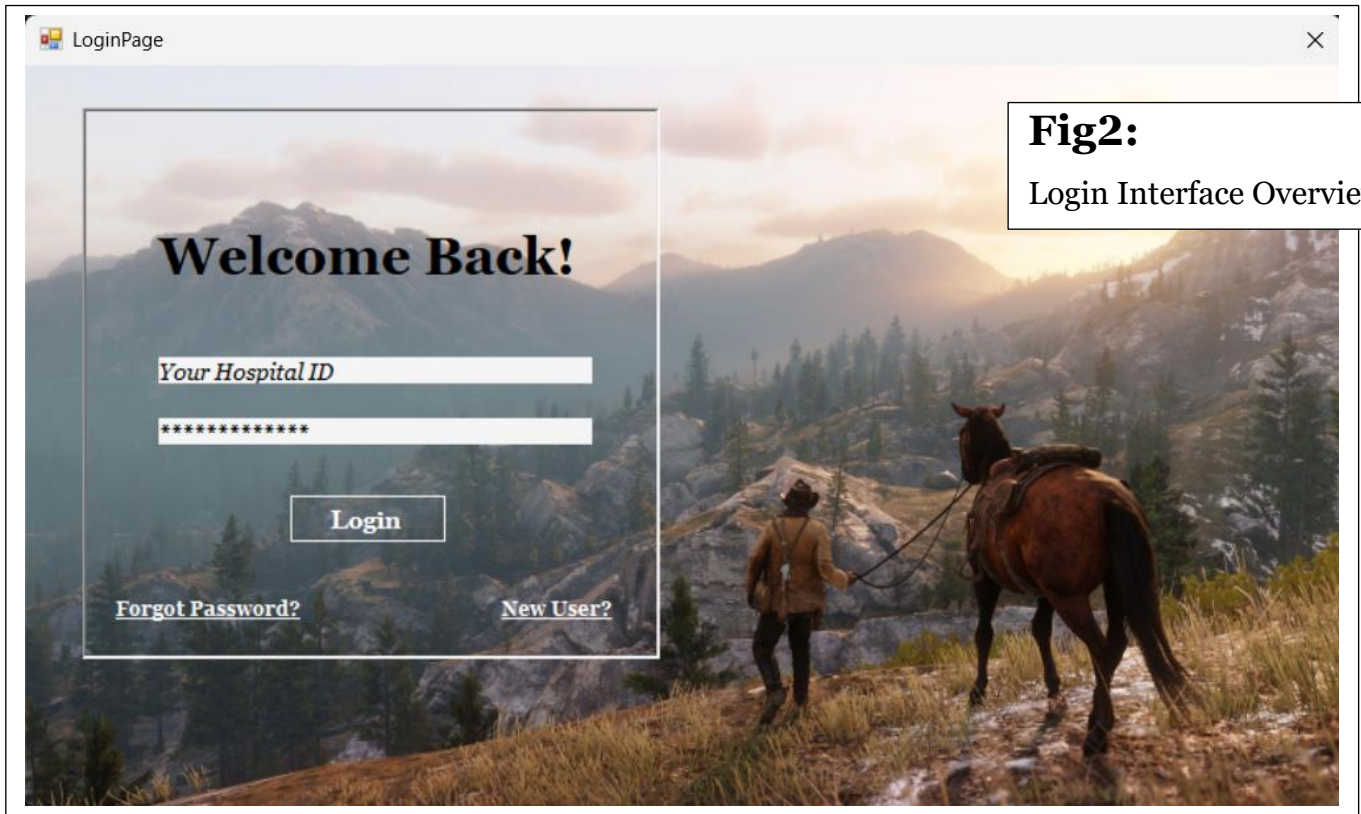


Fig2:

Login Interface Overview

The system begins with a login interface designed to provide secure and controlled access to the hospital database. As shown in Figure 2, the login screen presents a clean and welcoming design with the message “Welcome Back!”, creating a friendly user experience for hospital staff. The background image was carefully chosen to create a calm and professional atmosphere, which helps reduce stress for users who interact with the system daily.

The interface includes two main input fields: **Hospital ID** and **Password**, along with a login button. Additional options such as “*Forgot Password?*” and “*New User?*” are also provided to support usability. This design ensures that only authorized users can access the system, which is essential in a healthcare environment where sensitive data must be protected.

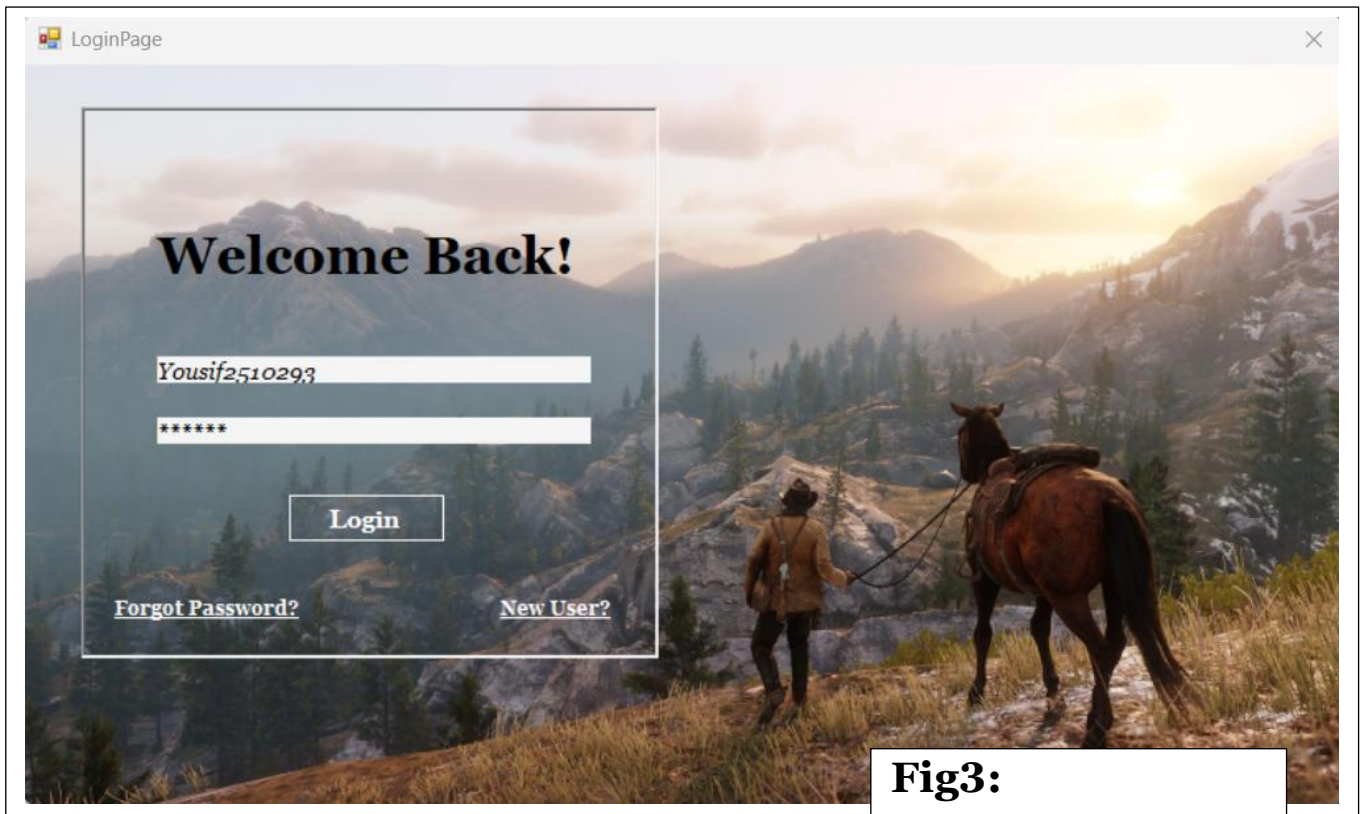


Fig3:

User Input Example

In Figure 3, the login fields are filled with user credentials, showing how the system is used in practice. The user enters their Hospital ID and password before clicking the login button. This step represents the first interaction between the user and the system.

From a design perspective, the input fields are clearly labeled and easy to use, which reduces user errors. The system expects correct input format, and this stage can include validation checks such as ensuring fields are not empty and that the password meets required conditions. This helps prevent incorrect or incomplete data from being processed.

Fig4:**Login Validation Logic (Code Implementation)**

Figure 4 shows the backend logic used to validate the login process. The system checks whether the entered Hospital ID and password match the predefined credentials. If the input is correct, the system opens the homepage and hides the login screen.

This implementation ensures that only valid users can proceed. Although the current version uses a simple condition check, it represents the basic concept of authentication, which can be expanded in the future using a database-based login system. This step is important because it connects the user interface with the system logic.

```

1  using System;
2  using System.Collections.Generic;
3  using System.ComponentModel;
4  using System.Data;
5  using System.Drawing;
6  using System.Linq;
7  using System.Text;
8  using System.Threading.Tasks;
9  using System.Windows.Forms;
10
11 namespace AssignHospital
12 {
13     4 references
14     public partial class LoginPage : Form
15     {
16         1 reference
17         public LoginPage()
18         {
19             InitializeComponent();
20         }
21
22         1 reference
23         private void label1_Click(object sender, EventArgs e)
24         {
25         }
26
27         0 references
28         private void label2_Click(object sender, EventArgs e)
29         {
30         }
31
32         1 reference
33         private void Form2_Load(object sender, EventArgs e)
34         {
35         }
36     }
37 }

```

```

30
31 private void Form2_Load(object sender, EventArgs e)
32 {
33 }
34
35 1 reference
36 private void textBox2_TextChanged(object sender, EventArgs e)
37 {
38 }
39
40 1 reference
41 private void textBox1_TextChanged(object sender, EventArgs e)
42 {
43 }
44
45 1 reference
46 private void button1_Click(object sender, EventArgs e)
47 {
48     if (textBox1.Text == "Yousif2510293" && textBox2.Text == "712006")
49     {
50         Homepage home = new Homepage();
51         home.Show();
52         this.Hide();
53     }
54     else
55     {
56         MessageBox.Show("Login Failed - Wrong username or password");
57     }
58 }
59
60

```

Fig5:**Error Handling and Security Feedback**

If incorrect login details are entered, the system displays an error message, as shown in Figure 5. The message “Login Failed – Wrong username or password” informs the user that access is denied.

This feature improves both usability and security. From a usability perspective, it clearly informs the user about the issue without crashing the system. From a security perspective, it prevents unauthorized access to hospital data. Error handling like this is important in real-world systems because it ensures the system remains stable even when users make mistakes.

Interface Design – Homepage Navigation System

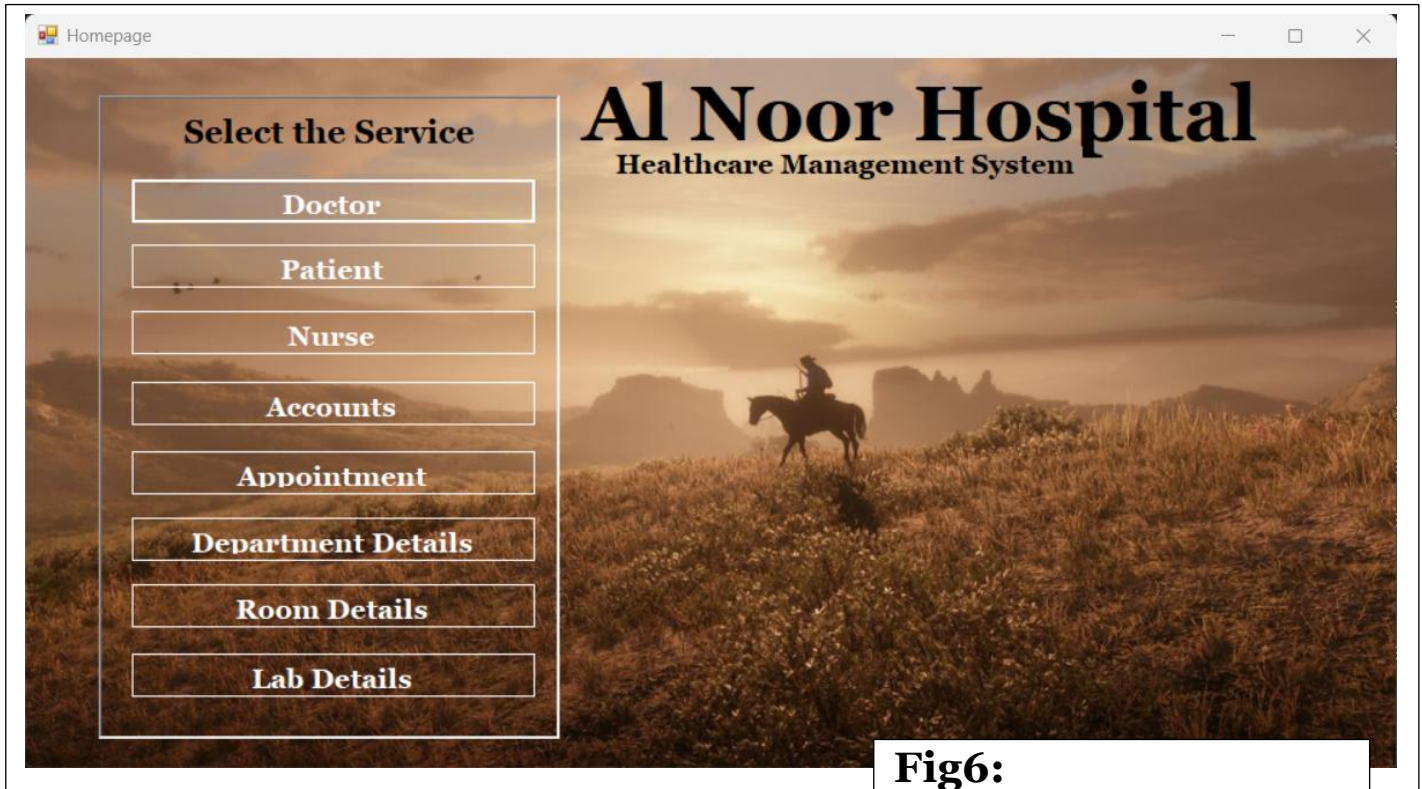


Fig6:

Homepage Interface Overview

After successful login, the user is redirected to the homepage, as shown in Figure 6. This page acts as the central control panel of the system, where users can select different services based on their tasks. The interface clearly displays the title “*Al Noor Hospital – Healthcare Management System*”, reinforcing the identity of the system and its purpose.

On the left side, a structured list of options is provided, including Doctor, Patient, Nurse, Accounts, Appointment, Department Details, Room Details, and Lab Details. This design allows users to easily navigate between different modules without confusion. Similar to the login screen, the background image creates a calm and professional environment, making the system visually appealing while maintaining usability. This is important in a hospital setting, where users need a system that is both functional and easy to use.

Fig7:**Navigation Logic – Opening Modules**

Figure 7 shows part of the backend implementation of the homepage. Each button is connected to a specific module in the system. For example, when the user clicks on the “Doctor” button, a new Doctors form is opened using:

```
Doctors home = new Doctors();
home.Show();
```

This logic allows the system to switch between different sections smoothly. Instead of loading everything in one screen, the system separates functionalities into different forms, which improves organization and performance. This design ensures that users only interact with the part of the system they need.

```

Homepage.cs
Homepage.cs [Design]
AssignHospital
1 using AssignHospital;
2 using System;
3 using System.Collections.Generic;
4 using System.ComponentModel;
5 using System.Data;
6 using System.Drawing;
7 using System.Linq;
8 using System.Text;
9 using System.Threading.Tasks;
10 using System.Windows.Forms;
11
12 namespace AssignHospital
13 {
14     21 references
15     public partial class Homepage : Form
16     {
17         9 references
18         public Homepage()
19         {
20             InitializeComponent();
21         }
22
23         1 reference
24         private void label1_Click(object sender, EventArgs e)
25         {
26         }
27
28         0 references
29         private void button2_Click(object sender, EventArgs e)
30         {
31         }
32
33         1 reference
34         private void button5_Click(object sender, EventArgs e)
35         {
36         }
37     }
38 }
75 %
No issues found

```

```

31     1 reference
32     private void button5_Click(object sender, EventArgs e)
33     {
34         Appointment home = new Appointment();
35         home.Show();
36     }
37
38     1 reference
39     private void button1_Click(object sender, EventArgs e)
40     {
41         Doctors home = new Doctors();
42         home.Show();
43     }
44
45     1 reference
46     private void button2_Click_1(object sender, EventArgs e)
47     {
48         Patients home = new Patients();
49         home.Show();
50     }
51
52     1 reference
53     private void button3_Click(object sender, EventArgs e)
54     {
55         Nurses home = new Nurses();
56         home.Show();
57     }
58
59

```

Fig8:**Multi-Module Navigation Structure**

As shown in Figure 8, the system supports multiple modules such as Patients, Nurses, and Accounting. Each module is opened using a similar structure, which keeps the code consistent and easy to manage.

This approach reflects real-world hospital operations, where different departments work independently but are connected through a central system. For example, a receptionist may access the Patient module, while an administrator may use the Accounting module. The system design ensures flexibility while maintaining a unified structure.

```
60      1 reference
61      private void button4_Click(object sender, EventArgs e)
62      {
63          Accounting home = new Accounting();
64          home.Show();
65      }
66
67      1 reference
68      private void button6_Click(object sender, EventArgs e)
69      {
70          Departments home = new Departments();
71          home.Show();
72      }
73
74      1 reference
75      private void button7_Click(object sender, EventArgs e)
76      {
77          Rooms home = new Rooms();
78          home.Show();
79      }
80
81      1 reference
82      private void button8_Click(object sender, EventArgs e)
83      {
84          Labs home = new Labs();
85          home.Show();
86      }
87
88
89      1 reference
90      private void Homepage_Load(object sender, EventArgs e)
91      {
```

Fig9:

Extended Navigation –
Departments, Rooms, and
Labs

Figure 9 completes the navigation structure by showing additional modules such as Departments, Rooms, and Labs. These modules are important for managing hospital resources and operations.

By including these options, the system provides full coverage of hospital data management. The consistent button-click logic ensures that each module is accessed in the same way, which improves user experience and reduces learning time. This design also makes the system scalable, meaning more modules can be added in the future without changing the overall structure.

Interface Design – Doctor Management Module

The screenshot displays the 'Doctors' management interface. At the top, the title 'Doctors' is centered. Below it, there are seven input fields for doctor information: DoctorID, DoctorName, DoctorAge, DoctorEmail, DoctorSalary, DoctorOfficesHrs, and DepartmentID. To the right of these fields are three buttons: 'Insert', 'Update', and 'Delete'. Below the input fields is a data grid showing a list of doctors. The grid has columns for DoctorID, DoctorName, DoctorAge, DoctorEmail, DoctorSalary, DoctorOfficesHrs, and DepaID. The first six rows contain data for doctors with IDs 1 through 6. The seventh row is empty and marked with an asterisk (*).

	DoctorID	DoctorName	DoctorAge	DoctorEmail	DoctorSalary	DoctorOfficesHrs	DepaID
▶	1	Yousif	20	ysf@gmail.com	99000	5	1
	2	Mohamed	41	moh@gmail.com	80000	7	1
	3	Fatma	34	fat@gmail.com	7000	7	2
	4	Karema	19	k@gmail.com	1000	2	2
	5	Ahmed	90	Ahmed@gmail.com	17000	9	3
	6	Ahmedoooo	90	Ahmed@gmail.com	17000	9	3
*							

Fig10:

Doctor Management
Interface Overview

Figure 10 presents the Doctor Management interface, which is one of the core modules in the Al Noor Care Hospital system. This screen allows users to manage doctor information in a structured and efficient way. The interface includes input fields such as DoctorID, DoctorName, DoctorAge, DoctorEmail, DoctorSalary, DoctorOfficeHrs, and DepartmentID.

The layout is designed to be simple and clear, so users can easily enter and modify data without confusion. The background design remains consistent with previous screens, creating a unified visual experience across the system. Below the input fields, a data grid is displayed, which shows all existing doctor records in the database. This allows users to view data in real time and verify any changes immediately.

Doctors

Doctors

DoctorID: 7
 DoctorName: SpiderMan
 DoctorAge: 20
 DocorEmail: Spider@gmail.com
 DocorSalary: 15
 DoctorOfficesHrs: 20000
 DepartmentID: 1

Doctor record inserted successfully

OK

Insert Update Delete

DoctorID	DoctorName	DoctorAge	DoctorEmail	DoctorSalary	DoctorOfficesHrs	DepaID
1	Yousif	20	ysf@gmail.com	99000	5	1
2	Mohamed	41	moh@gmail.com	80000	7	1
3	Fatma	34	fat@gmail.com	7000	7	2
4	Karema	19	k@gmail.com	1000	2	2
5	Ahmed	90	Ahmed@gmail.com	17000	9	3
6	Ahmedoooo	90	Ahmed@gmail.com	17000	9	3

Fig11 & 12:

Insert Operation (Adding New Doctor)

Doctors

Doctors

DoctorID: 7
 DoctorName: SpiderMan
 DoctorAge: 20
 DocorEmail: Spider@gmail.com
 DocorSalary: 15
 DoctorOfficesHrs: 20000
 DepartmentID: 1

Insert Update Delete

DoctorID	DoctorName	DoctorAge	DoctorEmail	DoctorSalary	DoctorOfficesHrs	DepaID
1	Yousif	20	ysf@gmail.com	99000	5	1
2	Mohamed	41	moh@gmail.com	80000	7	1
3	Fatma	34	fat@gmail.com	7000	7	2
4	Karema	19	k@gmail.com	1000	2	2
5	Ahmed	90	Ahmed@gmail.com	17000	9	3
6	Ahmedoooo	90	Ahmed@gmail.com	17000	9	3
7	SpiderMan	20	Spider@gmail.com	15	20000	1

Figures 11 and 12 demonstrate the process of inserting a new doctor record into the system. The user begins by entering the required information into the input fields. For example, a new doctor named *SpiderMan* is added with corresponding details such as age, email, salary, and department.

After clicking the **Insert** button, the system processes the data and stores it in the database. A confirmation message “*Doctor record inserted successfully*” is displayed, which provides feedback to the user that the operation was completed successfully. The newly added record then appears in the data grid below, showing that the database has been updated in real time.

This operation is important because it allows hospital staff to continuously add new doctors as the hospital expands, ensuring that all records remain up to date.

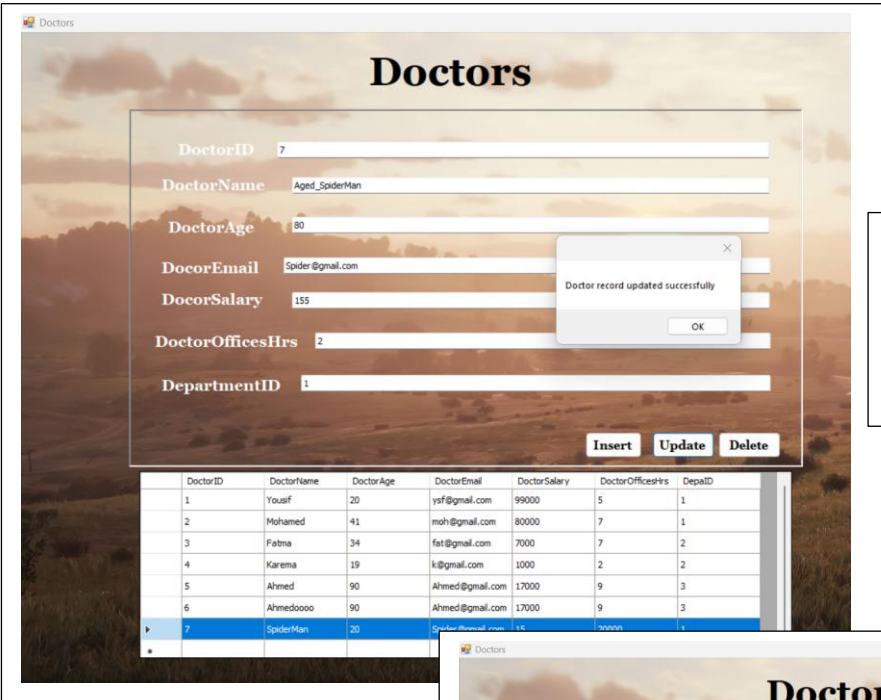


Fig13 & 14:
Update Operation (Modifying Existing Data)



Figures 13 and 14 show how an existing doctor record can be updated. The user selects a record from the data grid, and the data is loaded into the input fields. The user can then modify any required information, such as updating the doctor's name, age, salary, or working hours.

After clicking the **Update** button, the system saves the changes to the database and displays a confirmation message *"Doctor record updated successfully"*. The updated information is immediately reflected in the data grid, allowing the user to verify that the changes were applied correctly.

This feature is essential in a hospital system, where information such as schedules, salaries, or department assignments may change frequently.

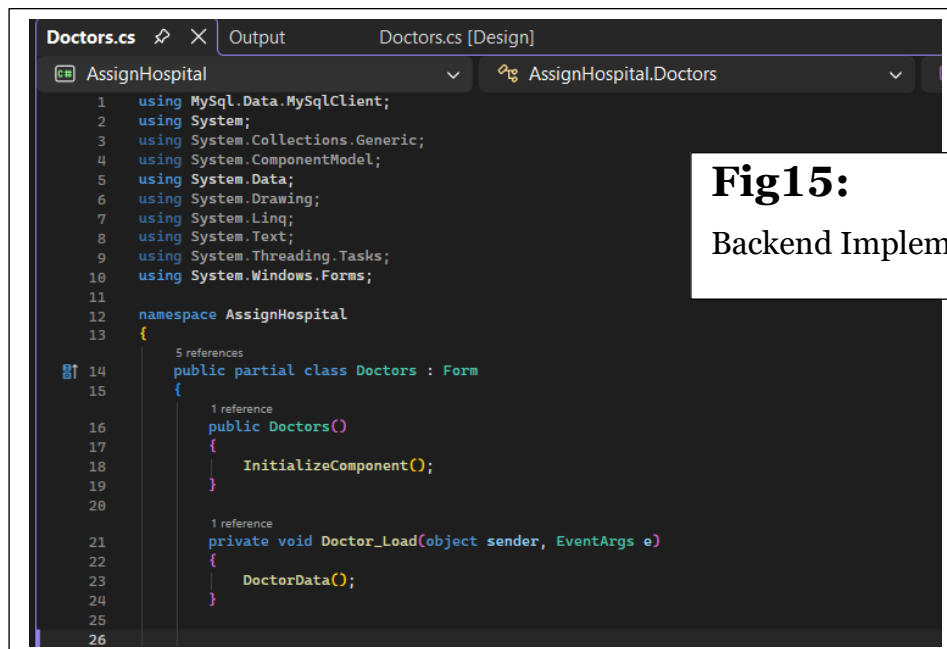


Fig15:
Backend Implementation Logic

Figure 15 shows part of the backend code used to control the Doctor Management module. The system uses MySQL connection libraries and functions to retrieve and manipulate data from the database. For example, the method `DoctorData()` is used to load doctor records into the interface when the form is opened.

This connection between the user interface and the database ensures that all operations such as insert, update, and delete are performed accurately. It also allows the system to maintain consistency between what the user sees on the screen and what is stored in the database.


```

27      4 references
28      private void DoctorData()
29      {
30          string query = "select * from doctors";
31          using (MySQLConnection conn = new DatabaseConnention().GetConnection())
32          {
33              conn.Open();
34              using (MySQLCommand cmd = new MySQLCommand(query, conn))
35              {
36                  using (MySQLDataAdapter adapter = new MySQLDataAdapter(cmd))
37                  {
38                      DataTable dataTable = new DataTable();
39                      adapter.Fill(dataTable);
40                      dataGridView1.DataSource = dataTable;
41                  }
42              }
43          }
44      }

```

Fig16:

Data Loading Function (DoctorData Method)

Figure 16 presents the implementation of the DoctorData() function, which is responsible for retrieving doctor records from the database and displaying them in the system interface. This function plays a key role in connecting the database with the user interface, ensuring that users can view up-to-date information at all times.

The function works by executing an SQL query `SELECT * FROM doctors`, which retrieves all records from the doctors table. A connection to the MySQL database is established using the system's database connection class. Then, a data adapter is used to transfer the retrieved data into a `DataTable`, which is finally displayed in the `DataGridView` component on the form.

From a system perspective, this function ensures that whenever the Doctor module is opened, all existing records are automatically loaded and displayed to the user. This improves usability, as staff members do not need to manually refresh or search for data. In the context of Al Noor Care Hospital, this feature is essential because it allows administrators and staff to quickly access doctor information, supporting efficient decision-making and daily operations.

Additionally, this method helps maintain consistency between the database and the interface, ensuring that any inserted or updated data is reflected immediately in the system view.

```

56 1 reference
57 private void button1_Click(object sender, EventArgs
58 {
59     int DoctorID = int.Parse(textBox1.Text);
60     string DoctorName = textBox2.Text;
61     int DoctorAge = int.Parse(textBox3.Text);
62     string DoctorEmail = textBox7.Text;
63     int DoctorSalary = int.Parse(textBox4.Text);
64     int DoctorOfficesHrs = int.Parse(textBox5.Text);
65     int DepaID = int.Parse(textBox6.Text);
66
67     string query = "INSERT INTO doctors (DoctorID, DoctorName, DoctorAge, DoctorEmail, DoctorSalary, DoctorOfficesHrs, DepaID) " +
68         "VALUES (@DoctorID, @DoctorName, @DoctorAge, @DoctorEmail, @DoctorSalary, @DoctorOfficesHrs, @DepaID)";
69     using (MySqlConnection conn = new DatabaseConnention().GetConnection())
70     {
71         conn.Open();
72         using (MySqlCommand cmd = new MySqlCommand(query, conn))
73         {
74             cmd.Parameters.AddWithValue("@DoctorID", DoctorID);
75             cmd.Parameters.AddWithValue("@DoctorName", DoctorName);
76             cmd.Parameters.AddWithValue("@DoctorAge", DoctorAge);
77             cmd.Parameters.AddWithValue("@DoctorEmail", DoctorEmail);
78             cmd.Parameters.AddWithValue("@DoctorSalary", DoctorSalary);
79             cmd.Parameters.AddWithValue("@DoctorOfficesHrs", DoctorOfficesHrs);
80             cmd.Parameters.AddWithValue("@DepaID", DepaID);
81
82             cmd.ExecuteNonQuery();
83         }
84     }
85
86     MessageBox.Show("Doctor record inserted successfully");
87     DoctorData();
88 }

```

Fig17:

Insert Operation Implementation

Figure 17 presents the backend implementation of the insert operation for adding a new doctor record into the system. When the user clicks the **Insert** button, the system collects the input values from the text fields such as DoctorID, DoctorName, DoctorAge, DoctorEmail, DoctorSalary, DoctorOfficeHrs, and DepartmentID.

The system then constructs an SQL INSERT query to store this data into the doctors table. Parameters are used instead of direct values, which improves security and prevents SQL injection. After executing the query, the system displays a confirmation message *"Doctor record inserted successfully"* to inform the user that the operation was completed.

Finally, the DoctorData() function is called to refresh the data grid and display the updated list of doctors. In the context of Al Noor Care Hospital, this function allows administrators to easily add new doctors as the hospital grows, ensuring that the database remains accurate and up to date.

```

private void button2_Click(object sender, EventArgs e)
{
    int DoctorID = int.Parse(textBox1.Text);
    string DoctorName = textBox2.Text;
    int DoctorAge = int.Parse(textBox3.Text);
    string DoctorEmail = textBox7.Text;
    int DoctorSalary = int.Parse(textBox4.Text);
    int DoctorOfficesHrs = int.Parse(textBox5.Text);
    int DepaID = int.Parse(textBox6.Text);

    string query = "update doctors set DoctorName=@DoctorName, DoctorAge=@DoctorAge, DoctorEmail=@DoctorEmail, " +
        "DoctorSalary=@DoctorSalary, DoctorOfficesHrs=@DoctorOfficesHrs, DepaID=@DepaID where DoctorID=@DoctorID";

    using (MySQLConnection conn = new DatabaseConnention().GetConnection())
    {
        conn.Open();
        using (MySQLCommand cmd = new MySQLCommand(query, conn))
        {
            cmd.Parameters.AddWithValue("@DoctorID", DoctorID);
            cmd.Parameters.AddWithValue("@DoctorName", DoctorName);
            cmd.Parameters.AddWithValue("@DoctorAge", DoctorAge);
            cmd.Parameters.AddWithValue("@DoctorEmail", DoctorEmail);
            cmd.Parameters.AddWithValue("@DoctorSalary", DoctorSalary);
            cmd.Parameters.AddWithValue("@DoctorOfficesHrs", DoctorOfficesHrs);
            cmd.Parameters.AddWithValue("@DepaID", DepaID);

            cmd.ExecuteNonQuery();
        }
    }

    MessageBox.Show("Doctor record updated successfully");
    DoctorData();
}

```

Fig18:

Update Operation Implementation

Figure 18 shows the implementation of the update operation, which allows users to modify existing doctor records. When the **Update** button is clicked, the system retrieves the updated values from the input fields and applies them to the selected doctor record using an SQL UPDATE query.

The query updates multiple fields such as name, age, email, salary, working hours, and department, based on the DoctorID. Similar to the insert operation, parameters are used to ensure secure data handling. Once the update is completed, a confirmation message *"Doctor record updated successfully"* is displayed.

This operation is particularly important in a hospital environment, where doctor information may frequently change, such as working hours or department assignments. The ability to update records quickly ensures that the system always reflects the most current information.

```

1 reference
private void button3_Click(object sender, EventArgs e)
{
    int DoctorID = int.Parse(textBox1.Text);

    string query = "delete from doctors where DoctorID=@DoctorID";

    using (MySQLConnection conn = new DatabaseConnention().GetConnection())
    {
        conn.Open();
        using (MySQLCommand cmd = new MySQLCommand(query, conn))
        {
            cmd.Parameters.AddWithValue("@DoctorID", DoctorID);

            cmd.ExecuteNonQuery();
        }
    }

    MessageBox.Show("Doctor record deleted successfully");
    DoctorData();
}

```

Fig19:

Delete Operation Implementation

Figure 19 demonstrates the delete operation, which allows users to remove a doctor record from the database. When the **Delete** button is clicked, the system identifies the doctor using the DoctorID and executes an SQL DELETE query.

After the record is removed, a confirmation message *"Doctor record deleted successfully"* is displayed to inform the user. The DoctorData() function is then called again to refresh the data grid and remove the deleted record from the interface.

This feature is essential for maintaining a clean and accurate database. In the hospital scenario, it allows administrators to remove outdated or incorrect records, ensuring that only valid and relevant data is stored in the system.

Interface Design – Patient Management Module

The screenshot displays the 'Patients' management interface. At the top, the title 'Patients' is centered. Below it is a form with two columns of input fields. The left column contains PatientID, PatientName, PatientTelephone, PatientAddress, RoomID, and NurseID. The right column contains PatientAge, PatientGender, PatientBloodgrp, DepartmentID, and DoctorID. Below the form are three buttons: Insert, Update, and Delete. At the bottom is a data grid with 11 columns: PatientID, PatientName, PatientTelephone, PatientAddress, PatientAge, PatientGender, PatientBloodgrp, DepartmentID, DoctorID, RoomID, and NurseID. The grid contains 6 rows of patient data.

	PatientID	PatientName	PatientTelephone	PatientAddress	PatientAge	PatientGender	PatientBloodgrp	DepartmentID	DoctorID	RoomID	NurseID
1		Omar Ali	0501112233	Dubai	25	Male	A+	1	1	1	1
2		Sara Khan	0554445566	Sharjah	30	Female	O-	1	2	2	2
3		Zayed Ahmed	0527778899	Abu Dhabi	45	Male	B+	2	3	3	3
4		Maryam J.	0564445556	Ajman	19	Female	AB+	3	5	4	4
5		John Smith	0583332211	Dubai	72	Male	O+	1	4	1	1
6		Yosuf	05012345689	galaa	19	Male	B+	1	1	2	3

Fig20:

Patient Management Interface Overview

Figure 20 presents the Patient Management interface, which is one of the most important parts of the system because it handles all patient-related information. The interface includes multiple input fields such as PatientID, PatientName, PatientTelephone, PatientAddress, PatientAge, PatientGender, PatientBloodgrp, DepartmentID, DoctorID, RoomID, and NurseID.

Compared to the Doctor module, this interface is more complex because a patient is connected to multiple entities such as doctors, rooms, departments, and nurses. This clearly reflects the relational database design created earlier. The data grid below the form displays all patient records, allowing users to view and verify data instantly.

The operations available in this module (Insert, Update, Delete) follow the same logic used in the Doctor module, ensuring consistency across the system and making it easier for users to interact without needing extra training.

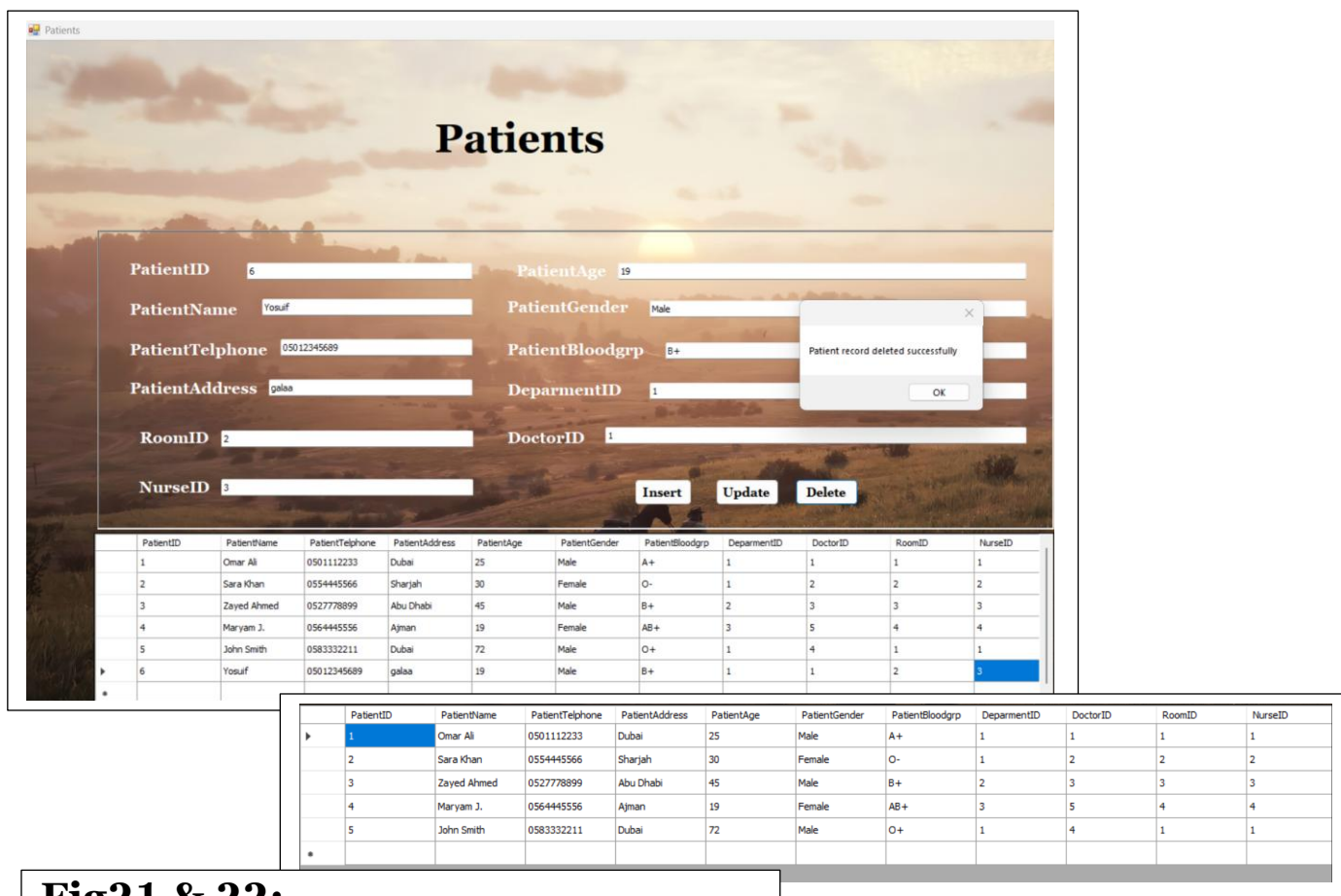


Fig21 & 22:

Delete Operation Scenario (Correcting Data Entry)

Figures 21 and 22 demonstrate a real-life scenario where a mistake occurs during data entry. A patient name was entered incorrectly as “Yosuf” instead of “Yousif”. To correct this, the delete operation is used to remove the incorrect record before re-entering the correct information.

When the Delete button is clicked, the system removes the selected patient record based on the PatientID and displays a confirmation message “Patient record deleted successfully”. The data grid is then refreshed automatically, showing that the incorrect record has been removed.

This scenario shows how the system handles real-world human errors smoothly, which is very important in a hospital environment where accuracy of patient data is critical.


```

4 references
private void Patientdata()
{
    string query = "select * from patient";
    using (MySQLConnection conn = new DatabaseConnention().GetConnection())
    {
        conn.Open();
        using (MySQLCommand cmd = new MySQLCommand(query, conn))
        {
            using (MySQLDataAdapter adapter = new MySQLDataAdapter(cmd))
            {
                DataTable dataTable = new DataTable();
                adapter.Fill(dataTable);
                dataGridView1.DataSource = dataTable;
            }
        }
    }
}

```

Fig23:

Data Loading Function (Patientdata Method)

Figure 23 shows the implementation of the Patientdata() function, which is responsible for retrieving patient records from the database and displaying them in the DataGridView.

This function executes the SQL query SELECT * FROM patient and uses a data adapter to load the results into a DataTable. The data is then displayed in the interface, allowing users to view all patient records when the form loads or after any operation.

This ensures that the system always shows updated and accurate data, which is essential for hospital staff who rely on quick access to patient information.

```

Output Patients.cs Patients.cs [Design]
AssignHospital AssignHospital.Patients button1_Click(object sender, EventArgs e)
184 string PatientName = textBox2.Text;
185 string PatientTelephone = textBox3.Text;
186 string PatientAddress = textBox4.Text;
187 int PatientAge = int.Parse(textBox5.Text);
188 string PatientGender = textBox6.Text;
189 string PatientBloodgrp = textBox7.Text;
190 int DeperamentID = int.Parse(textBox8.Text);
191 int DoctorID = int.Parse(textBox9.Text);
192 int RoomID = int.Parse(textBox10.Text);
193 int NurseID = int.Parse(textBox11.Text);
194
195 string query = "insert into patient (PatientID, PatientName, PatientTelephone, PatientAddress, PatientAge, PatientGender, Patient
196          values (@PatientID, @PatientName, @PatientTelephone, @PatientAddress, @PatientAge, @PatientGender, @PatientBloodg
197
198 using (MySQLConnection conn = new DatabaseConnention().GetConnection())
199 {
200     conn.Open();
201     using (MySQLCommand cmd = new MySQLCommand(query, conn))
202     {
203         cmd.Parameters.AddWithValue("@PatientID", PatientID);
204         cmd.Parameters.AddWithValue("@PatientName", PatientName);
205         cmd.Parameters.AddWithValue("@PatientTelephone", PatientTelephone);
206         cmd.Parameters.AddWithValue("@PatientAddress", PatientAddress);
207         cmd.Parameters.AddWithValue("@PatientAge", PatientAge);
208         cmd.Parameters.AddWithValue("@PatientGender", PatientGender);
209         cmd.Parameters.AddWithValue("@PatientBloodgrp", PatientBloodgrp);
210         cmd.Parameters.AddWithValue("@DeperamentID", DeperamentID);
211         cmd.Parameters.AddWithValue("@DoctorID", DoctorID);
212         cmd.Parameters.AddWithValue("@RoomID", RoomID);
213         cmd.Parameters.AddWithValue("@NurseID", NurseID);
214
215         cmd.ExecuteNonQuery();
216     }
217 }
218
219 MessageBox.Show("Patient record inserted successfully");
220
75 % No issues found Ln: 141, Ch: 10 SPC CRLF UTF-8 with BOM

```

Fig24:

Insert Operation Implementation (Patient Module)

Figure 24 presents the backend logic for inserting a new patient record. The system collects data from multiple input fields, including relationships such as DoctorID, RoomID, and NurseID, which connect the patient to other entities in the system.

An SQL INSERT query is executed using parameters to securely store the data in the database. After successful insertion, a confirmation message is displayed, and the data grid is refreshed using the Patientdata() function.

This operation is critical because it allows hospital staff to register new patients efficiently, ensuring that all relevant details are captured and linked correctly within the system.

```

148 string PatientAddress = textBox4.Text;
149 int PatientAge = int.Parse(textBox5.Text);
150 string PatientGender = textBox6.Text;
151 string PatientBloodgrp = textBox7.Text;
152 int DepartmentID = int.Parse(textBox8.Text);
153 int DoctorID = int.Parse(textBox9.Text);
154 int RoomID = int.Parse(textBox10.Text);
155 int NurseID = int.Parse(textBox11.Text);
156
157 string query = "update patient set PatientName=@PatientName, PatientTelephone=@PatientTelephone, PatientAddress=@PatientAddress, "
158 "PatientAge=@PatientAge, PatientGender=@PatientGender, PatientBloodgrp=@PatientBloodgrp, DepartmentID=@DepartmentID "
159 "DoctorID=@DoctorID, RoomID=@RoomID, NurseID=@NurseID where PatientID=@PatientID";
160
161 using (MySqlConnection conn = new DatabaseConnention().GetConnection())
162 {
163     conn.Open();
164     using (MySqlCommand cmd = new MySqlCommand(query, conn))
165     {
166         cmd.Parameters.AddWithValue("@PatientID", PatientID);
167         cmd.Parameters.AddWithValue("@PatientName", PatientName);
168         cmd.Parameters.AddWithValue("@PatientTelephone", PatientTelephone);
169         cmd.Parameters.AddWithValue("@PatientAddress", PatientAddress);
170         cmd.Parameters.AddWithValue("@PatientAge", PatientAge);
171         cmd.Parameters.AddWithValue("@PatientGender", PatientGender);
172         cmd.Parameters.AddWithValue("@PatientBloodgrp", PatientBloodgrp);
173         cmd.Parameters.AddWithValue("@DepartmentID", DepartmentID);
174         cmd.Parameters.AddWithValue("@DoctorID", DoctorID);
175         cmd.Parameters.AddWithValue("@RoomID", RoomID);
176         cmd.Parameters.AddWithValue("@NurseID", NurseID);
177
178         cmd.ExecuteNonQuery();
179     }
180
181     MessageBox.Show("Patient record updated successfully");
182     Patientdata();
183 }

```

Fig25:
Update Operation Implementation (Patient Module)

```

1 reference
private void button3_Click(object sender, EventArgs e)
{
    int PatientID = int.Parse(textBox1.Text);

    string query = "delete from patient where PatientID=@PatientID";

    using (MySqlConnection conn = new DatabaseConnention().GetConnection())
    {
        conn.Open();
        using (MySqlCommand cmd = new MySqlCommand(query, conn))
        {
            cmd.Parameters.AddWithValue("@PatientID", PatientID);

            cmd.ExecuteNonQuery();
        }
    }

    MessageBox.Show("Patient record deleted successfully");
    Patientdata();
}

```

Fig25:
Delete Operation Implementation (Patient Module)

Figure 25 demonstrates how patient records can be updated. The system retrieves existing data, allows the user to modify it, and then applies the changes using an SQL UPDATE query.

All fields, including relational attributes such as DepartmentID, DoctorID, RoomID, and NurseID, can be updated. This ensures flexibility in managing patient data, especially when changes occur such as room transfers or doctor reassignment.

The system confirms the update with a success message and refreshes the displayed data to reflect the changes immediately.

Figure 26 shows the implementation of the delete operation for patient records. The system uses the PatientID to identify the record and executes an SQL DELETE query to remove it from the database.

After deletion, a confirmation message is shown, and the data grid is refreshed. This ensures that the interface always reflects the current state of the database.

This functionality is important for maintaining data accuracy and removing outdated or incorrect patient records within the hospital system.

Interface Design – Appointment Management Module

Appointment

Appointment

AppointmentID

AppNo

AppDate

AppFees

DoctorID

PatientID

Insert

Update

Delete

	AppointmentID	AppNo	AppDate	AppFees	DoctorID	PatientID
1	1	1	3/10/2026	250	1	4
2	2	2	3/10/2026	150	2	3
3	3	3	3/11/2026	300	3	1
4	4	4	3/11/2026	200	4	2

Fig26:
Appointment Management
Interface Overview

```

4 references
private void AppointmentData()
{
    string query = "select * from appointment";
    using (MySQLConnection conn = new DatabaseConnention().GetConnection())
    {
        conn.Open();
        using (MySQLCommand cmd = new MySQLCommand(query, conn))
        {
            using (MySQLDataAdapter adapter = new MySQLDataAdapter(cmd))
            {
                DataTable dataTable = new DataTable();
                adapter.Fill(dataTable);
                dataGridView1.DataSource = dataTable;
            }
        }
    }
}

```

Fig27:

Data Loading Function
(AppointmentData Method)

Fig28:

Insert Operation
Implementation (Appointment
Module)

```

1 reference
private void button1_Click(object sender, EventArgs e)
{
    int AppointmentID = int.Parse(textBox1.Text);
    int AppNo = int.Parse(textBox2.Text);
    DateTime AppDate = DateTime.Parse(textBox3.Text);
    int AppFees = int.Parse(textBox4.Text);
    int DoctorID = int.Parse(textBox5.Text);
    int PatientID = int.Parse(textBox6.Text);

    string query = "insert into appointment (AppointmentID, AppNo, AppDate, AppFees, DoctorID, PatientID) " +
        "values (@AppointmentID, @AppNo, @AppDate, @AppFees, @DoctorID, @PatientID)";

    using (MySQLConnection conn = new DatabaseConnention().GetConnection())
    {
        conn.Open();
        using (MySQLCommand cmd = new MySQLCommand(query, conn))
        {
            cmd.Parameters.AddWithValue("@AppointmentID", AppointmentID);
            cmd.Parameters.AddWithValue("@AppNo", AppNo);
            cmd.Parameters.AddWithValue("@AppDate", AppDate);
            cmd.Parameters.AddWithValue("@AppFees", AppFees);
            cmd.Parameters.AddWithValue("@DoctorID", DoctorID);
            cmd.Parameters.AddWithValue("@PatientID", PatientID);

            cmd.ExecuteNonQuery();
        }
    }

    MessageBox.Show("Appointment record inserted successfully");
    AppointmentData();
}

```

```

1 reference
private void button2_Click(object sender, EventArgs e)
{
    int AppointmentID = int.Parse(textBox1.Text);
    int AppNo = int.Parse(textBox2.Text);
    DateTime AppDate = DateTime.Parse(textBox3.Text);
    int AppFees = int.Parse(textBox4.Text);
    int DoctorID = int.Parse(textBox5.Text);
    int PatientID = int.Parse(textBox6.Text);

    string query = "update appointment set AppNo=@AppNo, AppDate=@AppDate, AppFees=@AppFees, " +
        "DoctorID=@DoctorID, PatientID=@PatientID where AppointmentID=@AppointmentID";

    using (MySQLConnection conn = new DatabaseConnention().GetConnection())
    {
        conn.Open();
        using (MySQLCommand cmd = new MySQLCommand(query, conn))
        {
            cmd.Parameters.AddWithValue("@AppointmentID", AppointmentID);
            cmd.Parameters.AddWithValue("@AppNo", AppNo);
            cmd.Parameters.AddWithValue("@AppDate", AppDate);
            cmd.Parameters.AddWithValue("@AppFees", AppFees);
            cmd.Parameters.AddWithValue("@DoctorID", DoctorID);
            cmd.Parameters.AddWithValue("@PatientID", PatientID);

            cmd.ExecuteNonQuery();
        }
    }

    MessageBox.Show("Appointment record updated successfully");
    AppointmentData();
}

```

Fig29:

Update Operation
Implementation (Appointment
Module)

Fig30:

Delete Operation
Implementation (Appointment
Module)

```

1 reference
private void button3_Click(object sender, EventArgs e)
{
    int AppointmentID = int.Parse(textBox1.Text);

    string query = "delete from appointment where AppointmentID=@AppointmentID";

    using (MySQLConnection conn = new DatabaseConnention().GetConnection())
    {
        conn.Open();
        using (MySQLCommand cmd = new MySQLCommand(query, conn))
        {
            cmd.Parameters.AddWithValue("@AppointmentID", AppointmentID);

            cmd.ExecuteNonQuery();
        }
    }

    MessageBox.Show("Appointment record deleted successfully");
    AppointmentData();
}

```

Figure 26 presents the Appointment Management interface, which is responsible for managing appointments between patients and doctors. This module plays a key role in hospital operations because it connects two important entities: patients and doctors.

The interface includes input fields such as AppointmentID, AppNo, AppDate, AppFees, DoctorID, and PatientID. These fields are designed to capture all necessary details for scheduling an appointment. The data grid below displays all existing appointment records, allowing users to quickly review schedules and confirm bookings.

Similar to previous modules, the interface design is consistent in layout and style, which improves usability and makes it easier for hospital staff to navigate the system without confusion.

Figure 27 shows the implementation of the AppointmentData() function, which retrieves appointment records from the database.

The function executes the SQL query `SELECT * FROM appointment` and uses a data adapter to load the results into a DataTable. This data is then displayed in the DataGridView component.

This ensures that whenever the Appointment module is opened or updated, the system displays the most recent appointment data. In a hospital environment, this is very important because staff need quick and accurate access to appointment schedules.

Figure 28 demonstrates how a new appointment is added to the system. The user enters the appointment details such as date, fees, doctor ID, and patient ID.

The system converts the input values into appropriate data types, including converting the date field into a DateTime format. Then, an SQL INSERT query is executed using parameters to securely store the data in the database.

After the operation is completed, a confirmation message is displayed, and the AppointmentData() function is called to refresh the data grid. This allows users to immediately see the newly created appointment.

Figure 29 presents the update functionality for appointment records. The user selects an existing appointment, modifies the required fields, and clicks the Update button.

The system executes an SQL UPDATE query to apply the changes based on the AppointmentID. All fields such as appointment number, date, fees, doctor ID, and patient ID can be updated.

This feature is essential because appointment details may change frequently, such as rescheduling or assigning a different doctor

Figure 30 shows how appointment records can be deleted from the system. When the Delete button is clicked, the system removes the record using the AppointmentID.

After deletion, a confirmation message is displayed, and the data grid is refreshed automatically. This ensures that outdated or cancelled appointments are removed from the system.

This function helps maintain accurate scheduling and prevents confusion in hospital operations.

Interface Design – Supporting Modules (Honourable Mention)

Nurses

NurseID

NurseName

NurseAge

NurseWorknights

NurseSalary

RoomID

Insert

Update

Delete

NurseID	NurseName	NurseAge	NurseWorknights	NurseSalary	RoomID
1	Fatma	29	3	12000	1
2	Elena	34	5	14500	2
3	Grace	27	2	11000	3
4	Jia	40	5	16000	4

Fig31:
Nurse Module

Fig32:
Accounting Module

Accounting

AccountingID

AccountantName

AccountantWorknights

AccountantEmail

Insert

Update

Delete

AccID	AccountantName	AccountantWorknights	AccountantEmail
1	Sami Zeid	2	sami.z@hospital...
2	Huda Omar	1	huda.o@hospi...
3	Kareem Ali	3	kareem.a@hospi...
*			

DepaID	DepaName	DepaHead
1	cardiology	YoussefElkhouly
2	Emergency	MohamedAlassad
3	Surgery	HayaAhmed
4	Pharmacy	Hala Mohamed
5	admin	Youssef Ahmed

Fig33:
Department Module

Fig34:
Room Module

RoomID	RoomNo	RoomFloor	RoomSize
1	101	first floor	15
2	102	First floor	20
3	201	Second Floor	25
4	301	Third Floor	35

LabID	LabManager	LabWorknights	DoctorID
1	YousifRA	9 to 11 AM	1
2	Ahmedo	11 to 1 PM	2
3	Medo	12 to 3 PM	3
4	Hasan	1 to 4 PM	1
5	Fathy	5 to 12 Am	4

Fig34:
Lab Module

Figures 31 to 35 present additional modules implemented in the system, including **Nurses, Accounting, Departments, Rooms, and Labs**. These modules follow the same design structure and logic used in previous sections such as Doctors, Patients, and Appointments.

Each module provides a user-friendly interface with input fields, action buttons (Insert, Update, Delete), and a data grid that displays stored records. This consistent design approach ensures that users can interact with different parts of the system without needing to learn new layouts or workflows.

System Consistency and Design Approach

All these modules use the same backend logic and database interaction methods:

- ❖ SQL queries (INSERT, UPDATE, DELETE)
- ❖ Parameterized commands for security
- ❖ DataGridView for displaying records
- ❖ Automatic refresh after each operation

This consistency is very important because it improves system usability and reduces user errors. For example, once a staff member understands how to use the Doctor module, they can easily use the Nurse or Department modules without additional training.

Each module supports a specific part of hospital operations:

- ❖ **Nurse Module:** manages nurse details and links them to rooms
- ❖ **Accounting Module:** handles financial staff and basic administrative data
- ❖ **Department Module:** organizes hospital departments and their heads
- ❖ **Room Module:** manages hospital rooms and their details
- ❖ **Lab Module:** connect laboratory operations with doctors

Why These Modules Are Important

Although these modules follow the same logic as previous ones, they are important because they complete the full hospital system. They show that the system is not limited to one area but supports multiple operations, making it a fully functional relational database application.

Interface Design – Database Connection Core

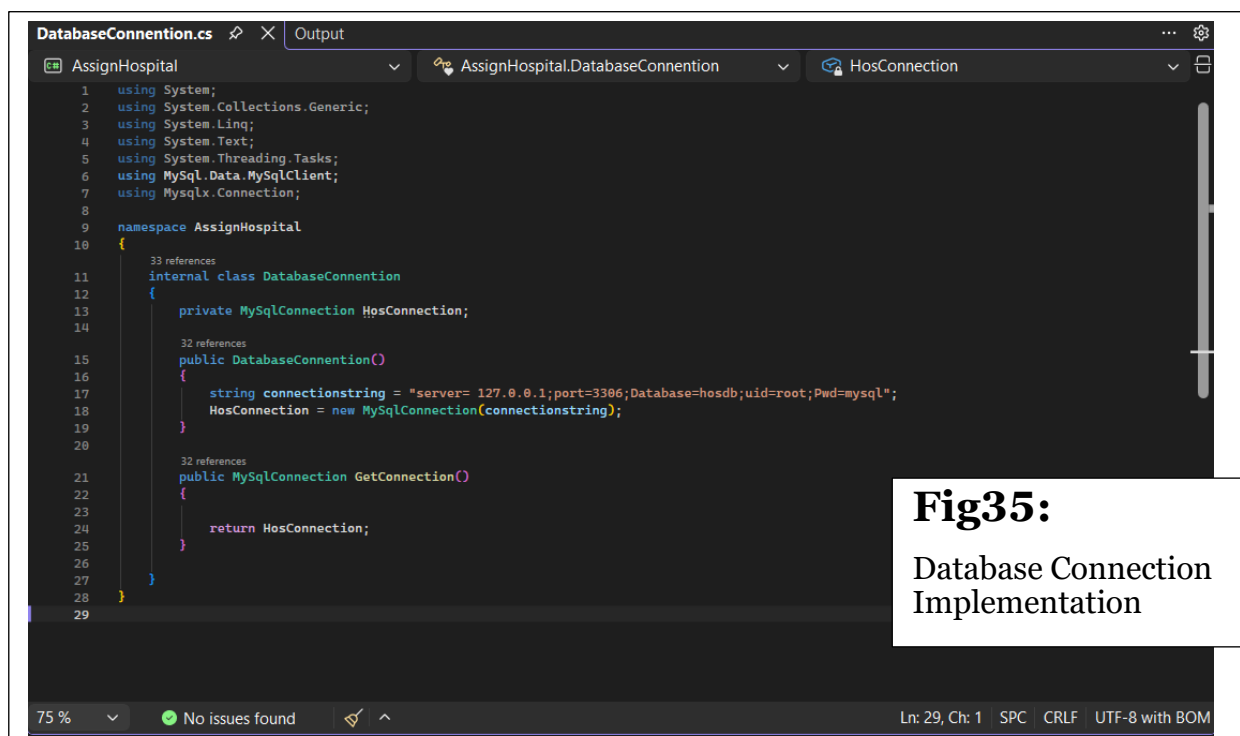


Fig35:

Database Connection Implementation

Figure 35 presents the implementation of the DatabaseConnection class, which acts as the central component responsible for connecting the system to the MySQL database. This class is used across all modules such as Doctors, Patients, Appointments, and others, making it one of the most important parts of the system.

The connection is established using a connection string that includes details such as the server address (127.0.0.1), port number (3306), database name, username, and password. This allows the system to communicate directly with the database and perform operations such as retrieving, inserting, updating, and deleting data.

How the Connection Works

The system creates an instance of the MySQL connection using the provided connection string. The method `GetConnection()` is then used to return this connection whenever it is needed by other parts of the system.

This design ensures that all modules use a single, consistent connection method instead of repeating connection code multiple times. As a result, the system becomes more organized, easier to maintain, and less prone to errors.

Importance in the System

This component acts as a bridge between the user interface and the database. Without it, none of the CRUD operations (Insert, Update, Delete, Select) would function.

In the context of Al Noor Care Hospital, this connection ensures that all data entered by users is stored safely and can be retrieved instantly when needed. It supports real-time data access across different modules, which is essential for efficient hospital management.

Design Advantage

By separating the database connection into its own class, the system follows a clean and modular design approach. This makes it easier to update or modify the connection settings in the future without affecting the rest of the system.

Development Environment – Visual Studio Implementation

Use of Visual Studio 2026 Community Edition

The system was developed using Visual Studio 2026 Community Edition, which is a powerful integrated development environment (IDE) used for building desktop applications. This software provided all the necessary tools required to design, develop, and test the hospital database system in one place.

Visual Studio was used to create a Windows Forms application, which allowed the design of graphical user interfaces such as login screens, data entry forms, and management modules. The drag-and-drop design feature made it easy to build user-friendly

interfaces, while the code editor allowed efficient implementation of backend logic using C#.

Fig36:
Visual Studio 2026 Purple Icon



Development Benefits

Benefit	Explanation	Impact on System
Faster Development	Visual Studio provides ready tools like drag-and-drop UI and templates	Reduced time to build forms such as Login, Doctors, Patients
Error Detection (Debugging)	Built-in debugger helps find and fix errors during runtime	Ensures the system runs correctly without crashes
Code Organization	Files, classes, and forms are structured inside one project	Makes the system easier to maintain and update
IntelliSense Support	Auto-suggestions and syntax help while writing code	Reduces coding mistakes and improves speed
Easy Database Integration	Supports MySQL libraries for direct database connection	Enables smooth CRUD operations in all modules
Reusability of Code	Functions like DatabaseConnection() can be reused	Avoids repeating code and improves efficiency
User-Friendly Design Tools	Windows Forms Designer simplifies UI creation	Helps create clean and consistent interfaces
Scalability	System can be expanded with new forms or features	Suitable for future hospital system upgrades

Key Features Used in the Project

Feature	Description	How It Was Used in Project
Windows Forms Designer	Visual tool for designing interfaces	Used to create Login, Homepage, Doctors, Patients, etc.
C# Programming Language	Main language for logic implementation	Used for CRUD operations and event handling
Event-Driven Programming	Executes code when user interacts (clicks buttons)	Used in Insert, Update, Delete button functions
IntelliSense	Provides code suggestions and auto-completion	Helped write correct syntax faster
Debugging Tools	Tools for testing and fixing errors	Used to detect issues in login and database operations
MySQL Integration	Connection with external database	Used through MySqlConnection and SQL queries
DataGridView Component	Displays database data in table format	Used in all modules to show records
Solution Explorer	Organizes project files and forms	Helped manage multiple modules
External Libraries (MySQL.Data)	Enables database communication	Used to connect C# application with MySQL

Why Visual Studio Was Suitable

Visual Studio was highly suitable for this project because it provides a complete environment for both interface design and backend development in one place. It allowed the system to be built using Windows Forms for the user interface and C# for implementing the logic, which made development smooth and efficient.

Additionally, Visual Studio supports direct integration with MySQL through external libraries, which enabled the system to perform all required database operations such as insert, update, delete, and retrieve data. This made it possible to develop a fully functional hospital management system that handles real-world data effectively.

Another important reason is its debugging and error-handling capabilities, which helped in identifying and fixing issues during

development. This ensured that the final system is stable, reliable, and ready for practical use in a hospital environment like Al Noor Care Hospital. (*Gunjal, 2003*)

Thirdly: System Functionality, Output Design, and Data Validation

System Function Overview

The developed system for Al Noor Care Hospital follows a structured workflow that allows users to manage hospital data efficiently. The process begins with the login interface (Figure 2), where users must enter valid credentials to access the system. After successful login, the user is redirected to the homepage (Figure 6), which acts as the main control panel.

From the homepage, users can navigate to different modules such as Doctors (Figure 10), Patients (Figure 20), and Appointments (Figure 26). Within each module, users can perform operations such as inserting new records, updating existing data, or deleting incorrect entries. These operations are connected directly to the database through backend functions (Figures 15, 17, 18, 19, 23, 24, 25, 26).

All data is stored in a MySQL database and displayed in real time using DataGridView components. This ensures that any changes made by the user are immediately visible, supporting fast and efficient decision-making in the hospital environment.

User Interaction with the System

Users interact with the system through a simple and consistent interface design. First, they log in using the login screen (Figure 2). After accessing the homepage (Figure 6), they select the required module depending on their role.

For example, a receptionist may use the Patient module (Figure 20) to register new patients or update their details, while an administrator may use the Doctor module (Figure 10) to manage staff information. Each module provides input fields and action buttons (Insert, Update, Delete), making it easy for users to perform tasks without technical knowledge.

The use of consistent layouts and navigation across all modules improves usability and reduces the chance of user errors, which is important in a hospital setting.

Output Design

The system provides different types of outputs to support user interaction and feedback. The main output is the **DataGridView**, which displays stored records in a table format, as shown in Figures 10, 20, and 26. This allows users to view, verify, and manage data in real time.

Another important output is the use of **MessageBox notifications**, which provide feedback after each operation. For example:

- ❖ “Doctor record inserted successfully” (Figure 12)
- ❖ “Patient record deleted successfully” (Figure 22)
- ❖ “Login Failed Case” (Figure 5)

These outputs help users understand whether their actions were successful or if an error occurred. This improves the overall user experience and ensures smooth interaction with the system.

Data Validation Rules “With evidence”

Field	Validation Rule	Example / Reason
DoctorID / PatientID “FigM.1”	Must be unique (Primary Key)	Prevent duplicate records
Age “Figs M.2/M.3”	Must be a number > 0 Or a number	Prevent invalid age values
Salary / Fees “Figs M.2/M.3”	Must be numeric and ≥ 0 or a number	Avoid incorrect financial data
Phone Number “Figs M.2/M.3”	Must be valid format or a number	Prevent wrong contact info
Date (Appointment) “Figs M.4/M.5”	Must be valid date format	Ensure correct scheduling
Foreign Keys (DoctorID, DeptID) “Figs M.6”	Must exist in related table	Maintain relationships

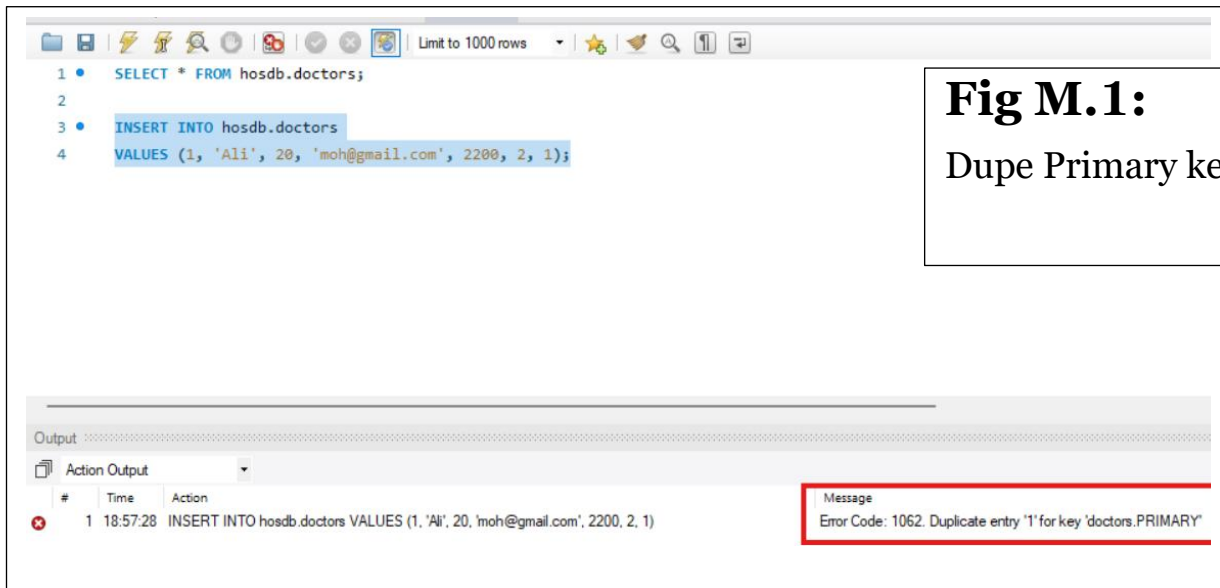


Fig M.1:
Dupe Primary key entry

Doctors

DoctorID

DoctorName

DoctorAge

DocorEmail

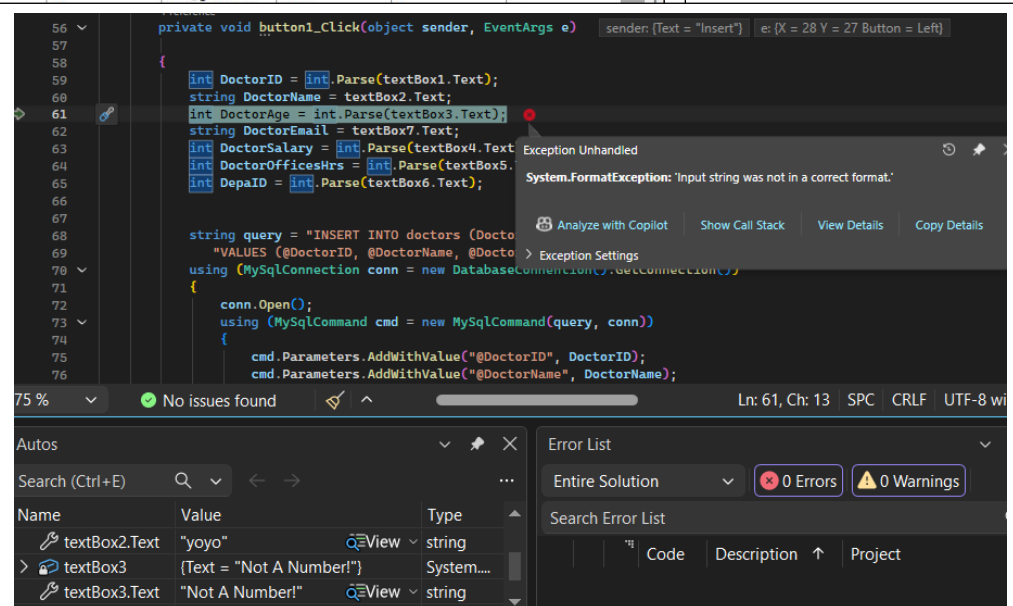
DocorSalary

DoctorOfficesHrs

DepartmentID

DoctorID	DoctorName	DoctorAge	DoctorEmail	DoctorSalary	DoctorOfficesHrs	DepaID
2	Mohamed	41	moh@gmail.com	80000	7	1
3	Fatma					
4	Karema					
5	Ahmed					
6	Ahmedoo					

Figs M.2/M.3:
String in A integer
textbox
“Same for
salary/Phone case”



Appointment

AppointmentID
AppNo
AppDate
AppFees
DoctorID
PatientID

Figs M.4/M.5:
Valid Date and time

```

76 private void button1_Click(object sender, EventArgs e)
77 {
78     int AppointmentID = int.Parse(textBox1.Text);
79     int AppNo = int.Parse(textBox2.Text);
80     DateTime AppDate = DateTime.Parse(textBox3.Text);
81     int AppFees = int.Parse(textBox4.Text);
82
83     conn.Open();
84     using (MySqlCommand cmd = new MySqlCommand(query, conn))
85     {
86         cmd.Parameters.AddWithValue("@AppointmentID", AppointmentID);
87         cmd.Parameters.AddWithValue("@AppNo", AppNo);
88         cmd.Parameters.AddWithValue("@AppDate", AppDate);
89         cmd.Parameters.AddWithValue("@AppFees", AppFees);
90         cmd.Parameters.AddWithValue("@DoctorID", DoctorID);
91         cmd.Parameters.AddWithValue("@PatientID", PatientID);
92         cmd.ExecuteNonQuery();
93     }
94 }

```

Exception Unhandled
System.FormatException: 'String was not recognized as a valid DateTime.'

Autos

Name	Value	Type
textBox2.Text	"7"	string
textBox3.Text	{Text = "222000"}	System....
textBox4.Text	222000	string

Error List

Entire Solution 0 Errors 0 Warnings

Search Error List

Code	Description	Project

```

1 SELECT * FROM hosdb.doctors;
2
3 INSERT INTO hosdb.doctors
4 VALUES (1, 'Ali', 20, 'moh@gmail.com', 2200, 2, 9999999999999999);

```

Figs M.6:
F.K Validation

Output

Action Output

#	Time	Action
1	19:24:29	INSERT INTO hosdb.doctors VALUES (1, 'Ali', 20, 'moh@gmail.com', 2200, 2, 9999999999999999)

Message
Error Code: 1264. Out of range value for column 'DepalID' at row 1

Validation plays a critical role in maintaining data integrity within the hospital system. Without validation, users may enter incorrect or incomplete data, which can lead to serious problems such as wrong patient information or scheduling errors.

By applying validation rules, the system ensures that only accurate and meaningful data is stored in the database. This is especially important in Al Noor Care Hospital, where data accuracy directly affects patient care and hospital operations.

Fourthly: Data Normalisation (1NF, 2NF, 3NF)

Data normalisation is the process of organizing data in a database to reduce duplication and improve data consistency. In the Al Noor Care Hospital system, normalisation is applied to ensure that data is stored efficiently and that each piece of information is placed in the correct table.

First Normal Form (1NF)

The database satisfies the First Normal Form because all tables contain atomic values, meaning each field stores only one value and there are no repeating groups.

For example, in the Patient table (Figure 20), each field such as PatientName, PatientAge, and PatientBloodgrp contains a single value only. There are no multiple values stored in one field, and each record is uniquely identified using a primary key (PatientID).

Second Normal Form (2NF)

The database satisfies the Second Normal Form because all non-key attributes depend fully on the primary key. This means that each table is designed so that its attributes are directly related to its own primary key.

For example, in the Doctor table (Figure 10), all attributes such as DoctorName, DoctorEmail, and DoctorSalary depend only on DoctorID. Similarly, in the Appointment table (Figure 26), all fields depend on AppointmentID. This ensures that there is no partial dependency in the system.

Third Normal Form (3NF)

The database satisfies the Third Normal Form because there are no transitive dependencies, meaning non-key attributes do not depend on other non-key attributes.

For example, department details are stored in the Department table instead of being repeated in the Doctor or Patient tables. The Doctor table only stores DepaID as a foreign key, which links to the Department table. This avoids duplication and ensures that data is stored in the correct location. *(Connolly and Begg, 2014; Airtable, 2023)*

Summary of Normalisation

Normal Form	How It Is Applied in the System
1NF	All fields contain single values and no repeating groups exist
2NF	All attributes depend fully on the primary key
3NF	No transitive dependencies; data is stored in separate related tables

Hospital Management System Design Evaluation

The **D**esigned system for Al Noor Care Hospital demonstrates a structured and functional approach to managing hospital data. The system integrates multiple modules such as Doctors, Patients, Appointments, and Departments into a single interface, allowing users to perform operations efficiently. As shown from Figure 2 to Figure 35, the system provides a clear workflow starting from login, moving to the homepage, and then accessing different functional forms.

Overall, the system design is effective in meeting basic hospital management requirements, particularly in terms of data organization, usability, and real-time interaction with the database.

Evaluation of RDBMS Usage

The use of a Relational Database Management System (MySQL) plays a critical role in improving the effectiveness of the Al Noor Care Hospital system. By organizing data into structured and related tables such as doctors, patient, appointment, and department, the system ensures high levels of data integrity and consistency. Relationships between entities, such as linking DoctorID and PatientID in the Appointment table (Figure 26), allow the system to manage data efficiently without duplication.

This relational structure reduces redundancy and improves performance when retrieving or updating records. However, despite these strengths, the system currently depends on manual entry of foreign keys, which increases the risk of user errors. Therefore, while the RDBMS design is strong, usability can be further improved through better input methods such as dropdown selections.

Aspect	How It Is Applied in the System	Effectiveness	Limitation	Suggested Improvement
Data Organization	Data stored in separate tables (Doctors, Patients, etc.)	High – keeps data structured and clear	None major	Maintain this structure
Relationships	Tables linked using foreign keys (e.g., DoctorID, PatientID)	High – avoids duplication and supports data linking	Users must enter IDs manually	Use dropdown lists instead of manual input
Data Integrity	Primary keys ensure unique records	High – prevents duplicate entries	No validation to stop wrong IDs	Add validation for foreign keys
Data Redundancy	Normalised tables reduce repeated data	High – efficient storage	Requires understanding of relationships	Add user-friendly UI guidance

Query Performance	SQL queries retrieve data efficiently	High – fast operations	May slow with large data in future	Add indexing if system grows
Scalability	Database can be expanded with new tables	Medium–High	No advanced optimisation yet	Improve with indexing and optimization

Evaluation of Table Design

The database design follows a well-structured relational model, as shown in the ER diagram and implemented tables. Each entity is separated into its own table (e.g., Doctors – Figure 10, Patients – Figure 20), which supports data normalization and reduces duplication.

Primary keys such as DoctorID and PatientID uniquely identify each record, while foreign keys maintain relationships between tables. This design ensures that data remains consistent and organized.

Despite these strengths, the reliance on numeric IDs for relationships may reduce usability for non-technical users. For example, users must remember or manually input IDs instead of selecting names, which increases the chance of incorrect data entry.

Aspect	How It Is Applied in the System	Effectiveness	Limitation	Suggested Improvement
Table Separation	Each entity has its own table (Doctors, Patients, etc.)	High – improves organization and clarity	None major	Maintain this design
Primary Keys	Unique IDs (DoctorID, PatientID) used	High – prevents duplicate records	Users must manually enter IDs	Auto-generate IDs
Foreign Keys	Used to link tables (DoctorID, DeptID)	High – ensures relationships between data	Hard for users to remember IDs	Use dropdown lists with names
Data Redundancy	Data split across tables	High – reduces duplication	Requires joins for queries	Acceptable (normal in RDBMS)
Data Consistency	Relationships enforce correct linking	High – maintains accurate data	No validation for wrong IDs	Add constraint validation

Usability	Users input numeric IDs manually	Medium – works but not user-friendly	High risk of input mistakes	Replace with selection controls
-----------	----------------------------------	--------------------------------------	-----------------------------	---------------------------------

Evaluation of User Interface Design

The user interface is simple, consistent, and easy to navigate. Starting from the login screen (Figure 2), users are guided to the homepage (Figure 6), where they can access different modules through clearly labeled buttons.

Each form (Doctors – Figure 10, Patients – Figure 20) follows the same layout structure, which improves usability and reduces the learning curve for users. The consistent placement of input fields and action buttons (Insert, Update, Delete) allows users to perform tasks efficiently.

Additionally, the use of a visual background enhances the user experience by making the interface more engaging. However, this design choice may also slightly reduce readability in some areas.

A limitation of the interface is the absence of advanced input controls such as dropdown menus or validation prompts, which could help prevent incorrect data entry.

Aspect	How It Is Applied in the System	Effectiveness	Limitation	Suggested Improvement
Navigation	Login Then Homepage then Modules (Figures 2 & 6)	High – clear user flow	No shortcut navigation	Add menu bar or quick access
Layout Consistency	Same design across all forms (Doctors, Patients)	High – easy to learn and use	Can feel repetitive	Add slight visual variation per module
Input Fields	Textboxes used for all data entry	Medium – simple to use	High risk of wrong input	Use dropdowns, date pickers
Action Buttons	Insert, Update, Delete clearly placed	High – easy to understand	No confirmation for delete	Add confirmation dialog
Data Display	DataGridView shows real-time data	High – clear and organized	No sorting/filtering	Add search and filter options
Visual Design	Background images used	Medium-High – attractive UI	May reduce readability	Use overlay or blur effect

User Feedback	MessageBox for actions (success/fail)	High – informs user	Limited error detail	Add detailed error messages
----------------------	---------------------------------------	---------------------	----------------------	-----------------------------

Evaluation of Data Handling (CRUD Operations)

The system effectively implements CRUD operations (Create, Read, Update, Delete), as demonstrated in Figures 17–19 and Figures 24–26. Users can insert new records, update existing data, and delete incorrect entries.

The integration of DataGridView ensures that all changes are reflected immediately, allowing users to verify operations in real time. This improves system responsiveness and usability.

However, one limitation is the lack of confirmation dialogs before deletion. This may lead to accidental data loss if a user deletes a record unintentionally. Adding a confirmation step would improve system reliability.

Evaluation of Output Design

The output design is clear and functional. The DataGridView component provides a structured view of stored data, making it easy to read and manage information. This is evident in Figures 10, 20, and 26.

MessageBox notifications also provide immediate feedback to users, such as:

- ❖ successful insertion (Figure 12)
- ❖ successful update (Figure 14)
- ❖ successful deletion (Figure 22)

These outputs enhance user understanding of system actions and improve overall interaction.

Evaluation of Data Validation

Basic data validation is partially implemented through the use of data types (e.g., integers for IDs and salaries). However, the system lacks advanced validation mechanisms at the user interface level.

For example:

- ❖ Users can enter invalid formats (incorrect email format)
- ❖ No checks for empty fields
- ❖ No range validation for values like age or salary

This limitation may affect data accuracy and reliability. Implementing stronger validation rules would significantly improve system quality.

Limitations and Suggested Improvements

Although the system developed for Al Noor Care Hospital is functional and meets basic requirements, it still has several limitations that affect its performance and usability in a real-world environment. Some of these limitations are related to user interaction, data validation, and system security. For example, the system allows all users to access and modify data without restriction, which may lead to unauthorized changes. In addition, the manual input of IDs increases the risk of human errors, especially when linking related data such as doctors and patients.

To improve the system, several enhancements can be implemented. These include adding dropdown lists for selecting IDs, improving data validation to prevent incorrect input, and introducing role-based access control. By addressing these limitations, the system can become more reliable, user-friendly, and suitable for real hospital operations.

Limitation	Impact on System	Suggested Improvement	Expected Benefit
No role-based access control	All users can modify data, risk of unauthorized changes	Implement user roles (Admin, Staff, Receptionist)	Improves security and control over data

Manual input of IDs	High chance of incorrect data linking	Use dropdown lists for DoctorID, DepartmentID, etc.	Reduces user errors and improves usability
Lack of input validation	Users can enter invalid or empty data	Add validation rules (email format, required fields, numeric checks)	Ensures data accuracy and integrity
No search/filter functionality	Hard to find specific records quickly	Add search bar and filtering options	Improves efficiency in data retrieval
No delete confirmation	Risk of accidental data loss	Add confirmation dialog before delete	Prevents unintended deletions
Basic security implementation	Weak protection of sensitive hospital data	Improve security (encryption, stronger login system)	Enhances system safety and reliability
UI readability issues (background)	May affect user comfort and visibility	Improve UI design (simpler background, better contrast)	Improves user experience

Finally, the system design for Al Noor Care Hospital is effective in providing a structured and functional database solution. The use of an RDBMS, normalized tables, and a consistent interface supports efficient data management.

However, the system can be further improved by enhancing validation, usability, and security features. With these improvements, the system has the potential to become a fully reliable solution for real-world hospital operations.

Implementing the Hospital Database System From Design to Reality

Putting the designed system into practice is an essential step in transforming theoretical database design into a fully functional application. In this stage, the database for Al Noor Care Hospital is implemented using MySQL Workbench and integrated with the developed application in Visual Studio. This allows the system to perform real operations such as storing, retrieving, updating, and deleting data efficiently.

The implementation focuses on translating the previously designed tables, relationships, and constraints into actual database structures and SQL scripts. It also demonstrates how the system handles real data through user interaction, ensuring that all modules such as Doctors, Patients, and Appointments work correctly within a connected environment.

Introduction to MySQL and Development Tools

Before presenting the implementation, it is important to understand the tools used in building the database system.

MySQL is a **Relational Database Management System (RDBMS)** used to store and manage structured data. It allows users to create databases, define tables, and perform operations using Structured Query Language (SQL). In this project, MySQL is used to store all hospital data such as patient records, doctor information, and appointments.

MySQL Workbench is the graphical tool used to interact with the database. It provides an interface for writing SQL queries, creating tables, viewing data, and managing the database structure. This tool makes it easier to visualize the database and execute commands without needing complex configurations.

(Churcher, 2012)



Fig37

Additionally, the system is connected to a Visual Studio application, where C# is used to build the user interface and implement the system logic. This integration allows real-time communication between the application and the database, ensuring that any operation performed by the user is directly reflected in the database.

MySQL Workbench Implementation

Once the design of the database was completed, the next step was to bring it into reality. This is where MySQL Workbench was used as the main tool to build and manage the database for Al Noor Care Hospital. Instead of working only with diagrams and theory, the system was now created as an actual working database that could store and manage real data.

Using MySQL Workbench, the process began by setting up connections to the database environment. As shown in **Figure 38**, multiple database connections were configured, including the main hospital database (*HospitalDB*). This step allowed the system to establish a link between the application and the database server, making it possible to execute queries and manage data efficiently.

Through this interface, the database environment became accessible, allowing the creation of tables such as doctors, patient, appointment, and department. MySQL Workbench provided a clear and organized workspace where all database activities could be performed, including writing SQL scripts, executing commands, and monitoring connections.

This stage represents the starting point of the implementation, where the system transitions from a conceptual design into a real, connected database system capable of supporting hospital operations.

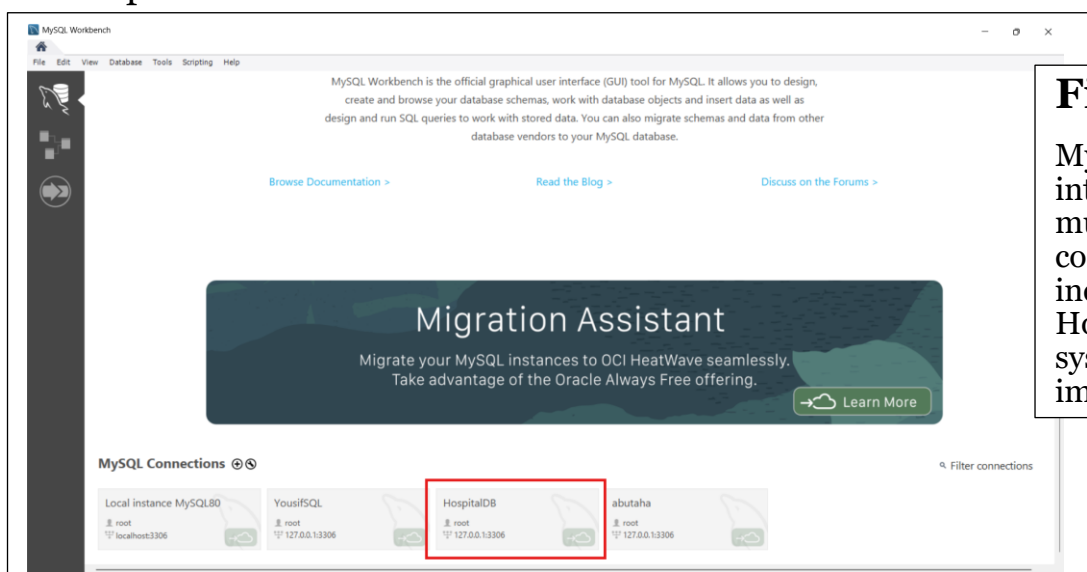


Fig38:

MySQL Workbench interface showing multiple database connections including the HospitalDB used for system implementation.

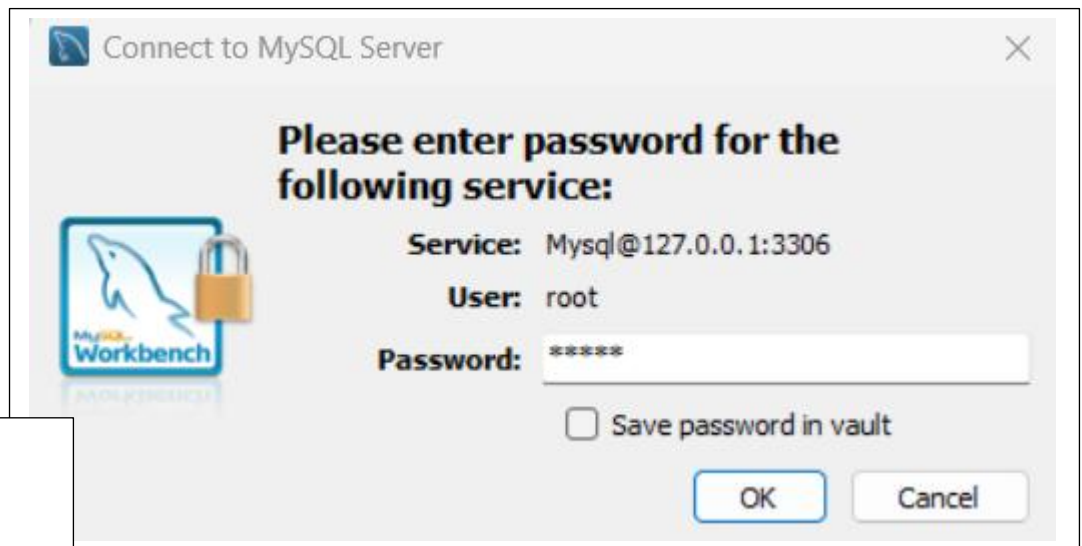


Fig39:

MySQL Workbench authentication window requiring user credentials to connect to the MySQL server.

After selecting the database connection, authentication is required to access the MySQL server. As shown in **Figure 39**, the system prompts the user to enter the password associated with the database connection. In this project, the root user credentials were used to successfully establish access to the database environment.

This step ensures that only authorized access is granted before performing any operations such as creating tables or executing SQL queries. Once authenticated, the user gains full control over the database, allowing the implementation process to continue.

From an evaluation perspective, this represents a basic level of database security. While it is sufficient for development purposes, a real hospital system would require more advanced authentication mechanisms such as user roles and restricted access levels.

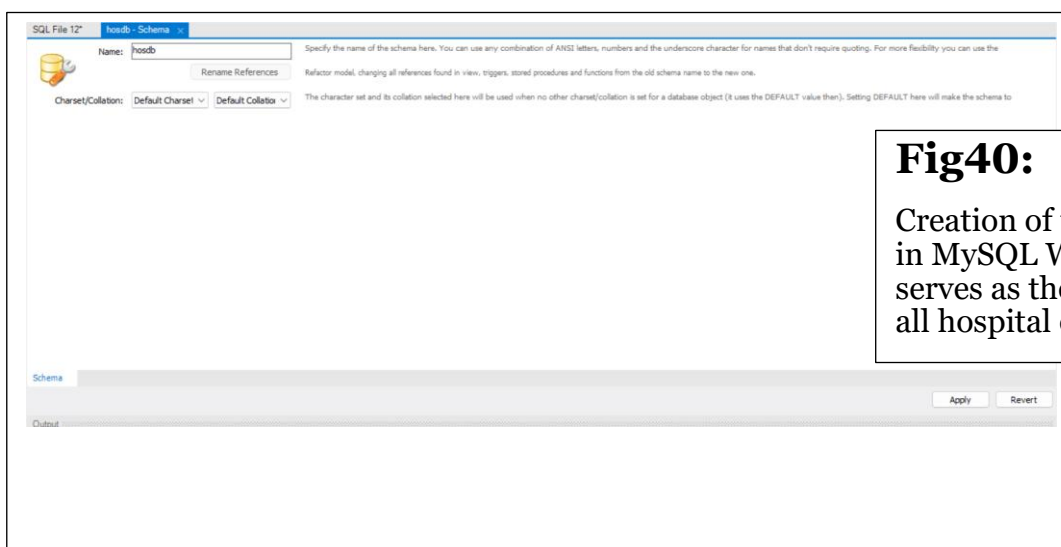


Fig40:

Creation of the “hosdb” schema in MySQL Workbench, which serves as the main container for all hospital database tables.

Following successful authentication, the next step in the implementation process is the creation of the database schema, which acts as the foundation of the entire system. As illustrated in **Figure 40**, a new schema named “**hosdb**” was defined within MySQL Workbench.

This schema represents the logical container that stores all database objects, including tables such as *patients*, *doctors*, *appointments*, *nurses*, and *departments*. By organizing all entities under a single schema, the system ensures better structure, easier management, and improved scalability.

From a design perspective, defining the schema at this stage is essential because it establishes a centralized environment where all relational tables will be created and linked together. It also ensures consistency in character encoding and collation settings, which is important for handling textual data such as patient names and addresses.

Additionally, this step reflects good database design practice, as it separates this hospital system from other databases that may exist on the same MySQL server. This isolation improves maintainability and reduces the risk of data conflicts.

After successfully creating the schema, the system transitions into defining and organizing the core database tables that represent the hospital environment. As shown in **Figure 41**, the *hosdb* schema now contains a structured collection of tables including **patient**, **doctors**, **appointment**, **nurse**, **department**, **rooms**, **lab**, and **accounting**.

This stage marks a critical milestone in the development process, where the abstract design (ER diagram) is transformed into an actual relational database structure. Each table represents a real-world entity within Al Noor Hospital, and together they form an interconnected system that supports efficient data storage and retrieval.

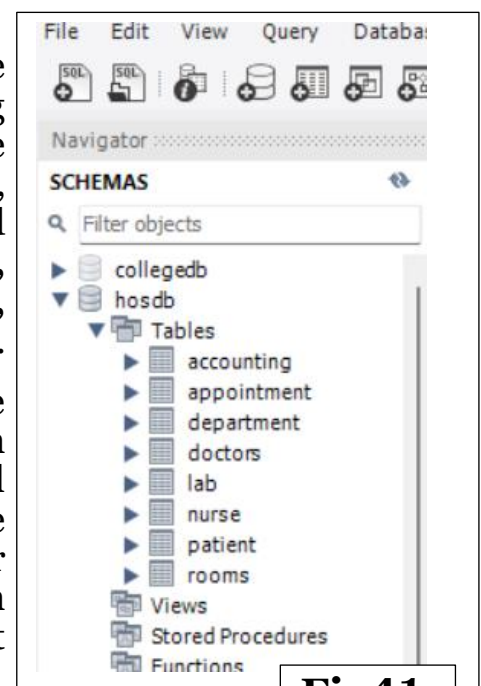


Fig41

From a functional perspective, this structure enables the system to handle multiple operations simultaneously. For example, the patient table stores personal and medical-related data, while the doctors table manages staff information. The appointment table acts as a bridge between patients and doctors, ensuring proper scheduling and coordination. Similarly, tables like rooms, labs, and

accounting extend the system's capability to manage hospital resources and financial records.

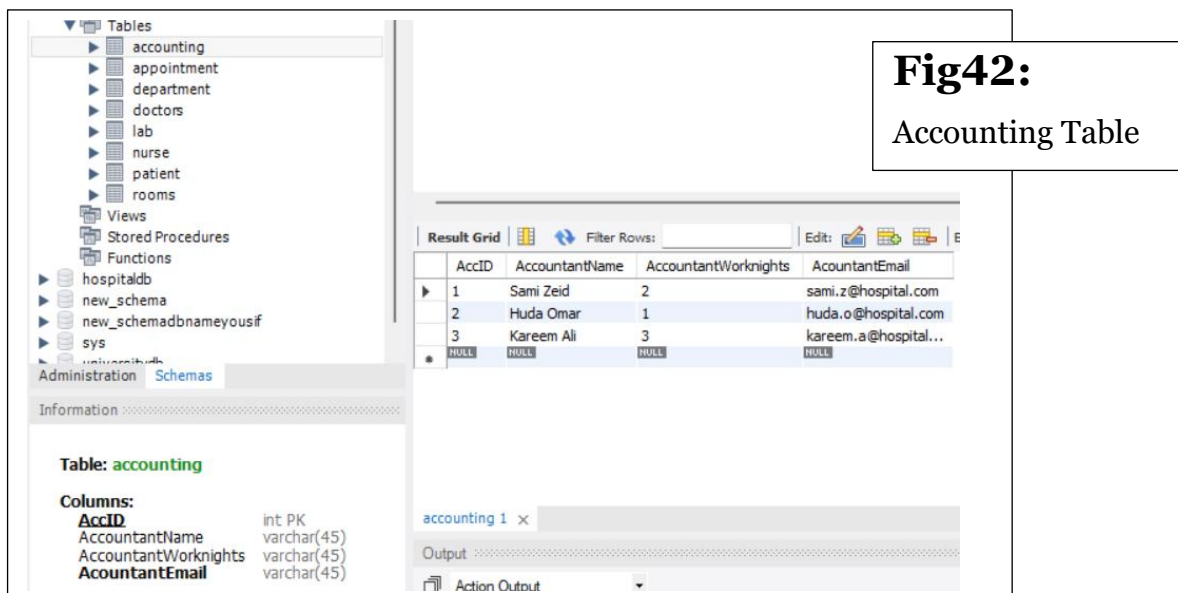
Moreover, the separation of data into multiple tables reflects the application of normalization principles introduced earlier in the design phase. By avoiding redundant data storage and ensuring that each table serves a specific purpose, the system improves data integrity, reduces anomalies, and enhances overall performance.

Another important observation from **Figure 41** is the clarity of organization within MySQL Workbench. The hierarchical view allows developers to easily navigate between tables, which simplifies database management and debugging processes during development.

Table Implementation and Structural Design

After defining the database schema, the next step involved creating the actual tables within the *hosdb* database. This phase translates the conceptual ER design into a fully functional relational structure supported by MySQL Workbench.

Each table was carefully designed with appropriate **data types**, **primary keys**, and **foreign key constraints** to ensure data integrity and enforce relationships between entities.



As illustrated in **Figure 42**, the *accounting* table was created to store financial staff data.

This table includes attributes such as:

- ❖ **AccID (Primary Key)** to uniquely identify each accountant
- ❖ **AccountantName, AccountantWorkingHrs, AccountantEmail**

The use of a primary key ensures that each record is uniquely identifiable, while VARCHAR data types allow flexible storage of textual data such as names and emails.

Fig43:
Appointment Table

AppointmentID	AppNo	AppDate	AppFees	DoctorID	PatientID
1	1	2026-03-10	250	1	4
2	2	2026-03-10	150	2	3
3	3	2026-03-11	300	3	1
4	4	2026-03-11	200	4	2
5	5	2026-03-12	250	5	5
6	6	2026-10-10	270	2	2
NULL	NULL	NULL	NULL	NULL	NULL

Table: appointment

Columns:

- AppointmentID int PK
- AppNo int
- AppDate date
- AppFees int
- DoctorID int
- PatientID int

appointment 1 x

Output

Action Output

#	Time	Action
1	01:26:22	SELECT * FROM hosdb.accounting LIMIT 0, 1000
2	01:27:48	SELECT * FROM hosdb.appointment LIMIT 0, 1000

Fig44

Foreign Key Name	Referenced Table
AppDoCRK	'hosdb', 'doctors'
AppPatient	'hosdb', 'patient'

Column	Referenced Column
☐ AppointmentID	
☐ AppNo	
☐ AppDate	
☐ AppFees	
☒ DoctorID	DoctorID
☐ PatientID	

Foreign Key Options

On Update: CASCADE

On Delete: CASCADE

☐ Skip in SQL generation

Foreign Key Comment

Fig45

Foreign Key Name	Referenced Table
AppDoCRK	'hosdb', 'doctors'
AppPatient	'hosdb', 'patient'

Column	Referenced Column
☐ AppointmentID	
☐ AppNo	
☐ AppDate	
☐ AppFees	
☐ DoctorID	
☒ PatientID	PatientID

Foreign Key Options

On Update: CASCADE

On Delete: CASCADE

☐ Skip in SQL generation

Foreign Key Comment

The *appointment* table plays a central role in connecting patients with doctors. As shown in **Figures 43, 44, and 45**, this table includes:

- ❖ **AppointmentID (Primary Key)**
- ❖ **AppNo, AppDate, AppFees**
- ❖ **DoctorID and PatientID (Foreign Keys)**

The foreign key constraints establish relationships:

- ❖ DoctorID references *doctors table*

❖ PatientID references *patient* table

Additionally, the **CASCADE option** ensures that updates or deletions in parent tables automatically reflect in the appointment table. This is critical in maintaining referential integrity and avoiding orphan records.

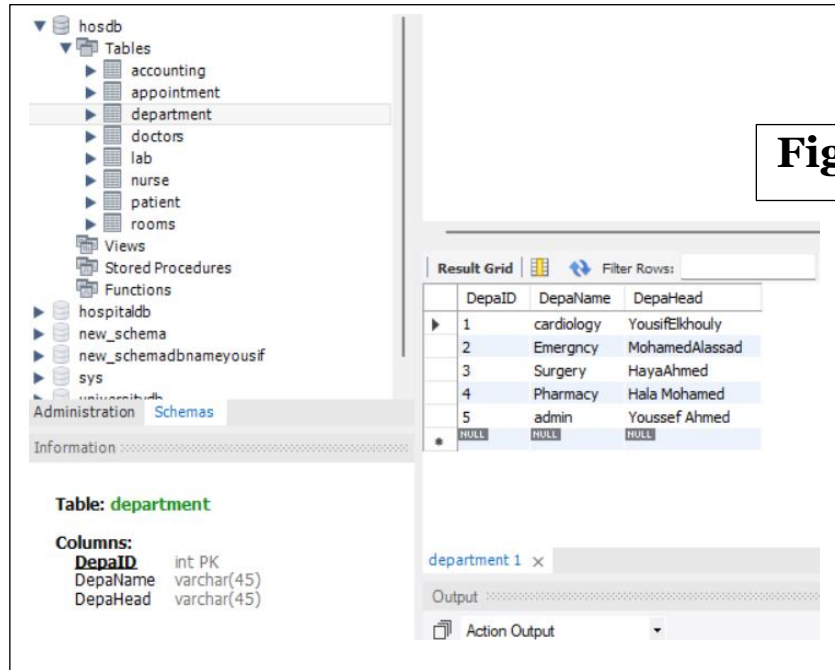


Fig46

DepaID	DepaName	DepaHead
1	cardiology	YousifElkhouly
2	Emergency	MohamedAlassad
3	Surgery	HayaAhmed
4	Pharmacy	Hala Mohamed
5	admin	Youssef Ahmed
NULL	NULL	NULL

Table: **department**

Columns:

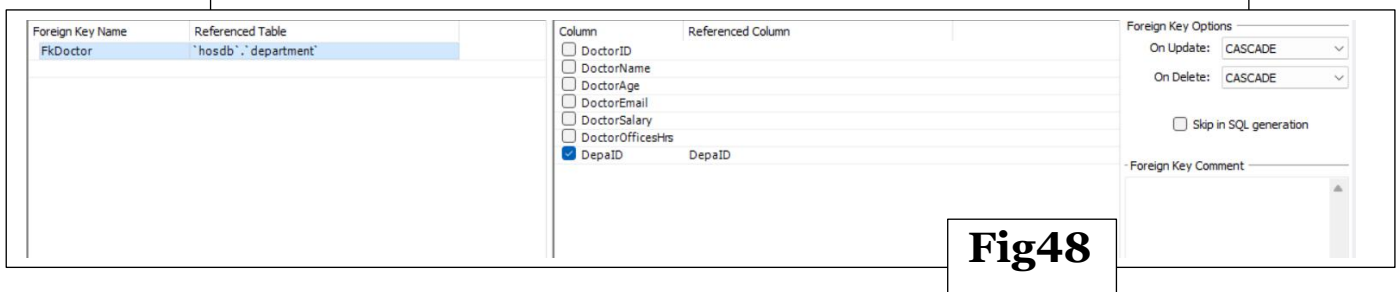
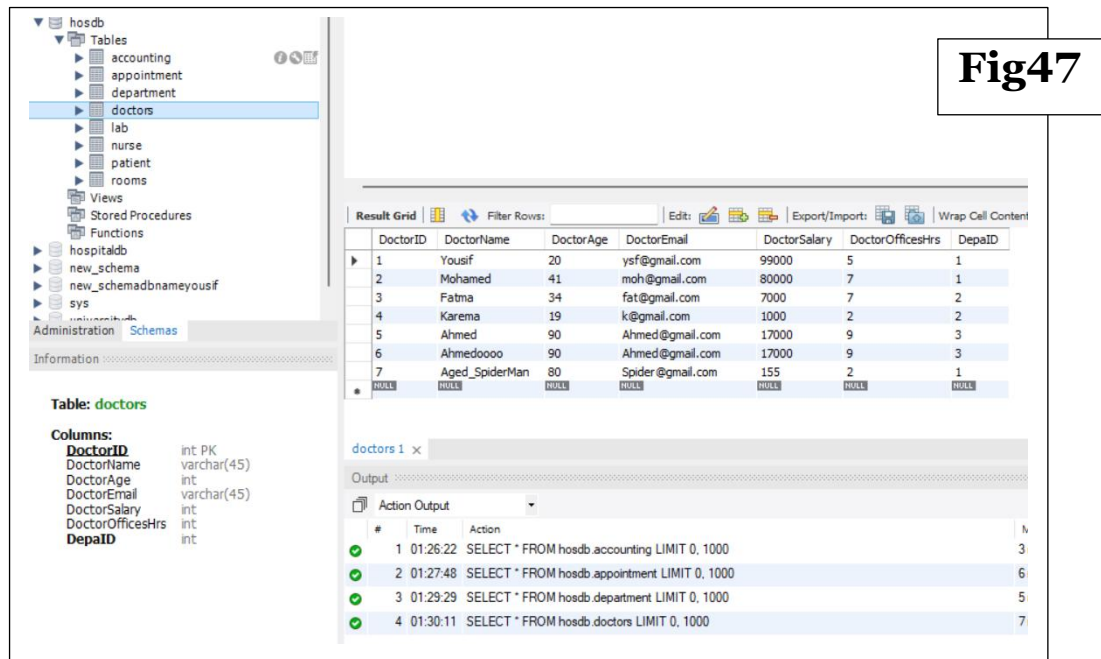
- DepaID** int PK
- DepaName varchar(45)
- DepaHead varchar(45)

As shown in **Figure 46**, the *department* table stores information about hospital departments.

It includes:

- ❖ **DepaID (Primary Key)**
- ❖ **DepaName**
- ❖ **DepaHead**

This table acts as a **central reference entity** that connects to other tables such as *doctors* and *patients*. By doing so, it ensures that all medical staff and patients are logically grouped under specific departments, which improves organization and reporting within the hospital system.



As illustrated in **Figures 47 and 48**, the *doctors* table is implemented to store detailed information about medical staff within Al Noor Hospital.

The table includes the following attributes:

- ❖ **DoctorID (Primary Key)** – uniquely identifies each doctor
- ❖ **DoctorName** – stores the doctor's full name
- ❖ **DoctorAge** – represents the doctor's age
- ❖ **DoctorEmail** – used for communication purposes
- ❖ **DoctorSalary** – stores salary details
- ❖ **DoctorOfficesHrs** – represents working hours
- ❖ **DepaID (Foreign Key)** – links each doctor to a specific department

From a relational design perspective, the most important feature shown in these figures is the **foreign key constraint between the doctors table and the department table**. This

relationship ensures that every doctor must belong to a valid department, which enforces data consistency across the system.

Additionally, the use of **CASCADE update and delete rules** guarantees that any modification in the department table will automatically reflect in the doctors table. This prevents issues such as invalid department references and maintains referential integrity.

Fig49

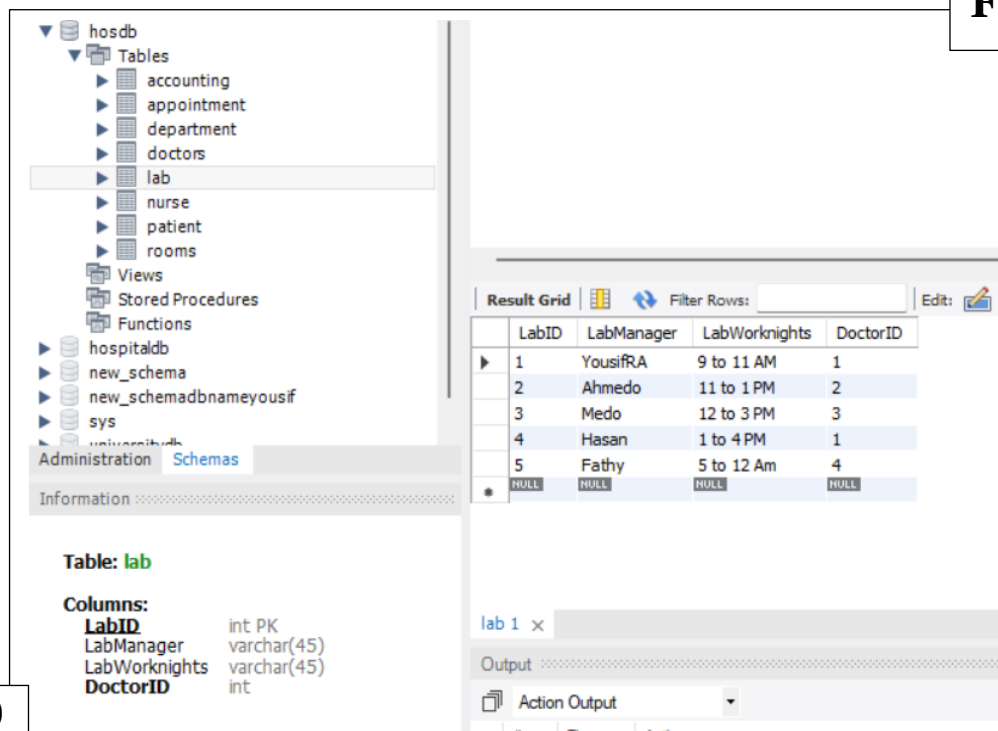


Fig50



As presented in **Figures 49 and 50**, the *lab* table is designed to manage laboratory-related information within the hospital system. This table plays a supportive but important role by linking lab operations directly to doctors.

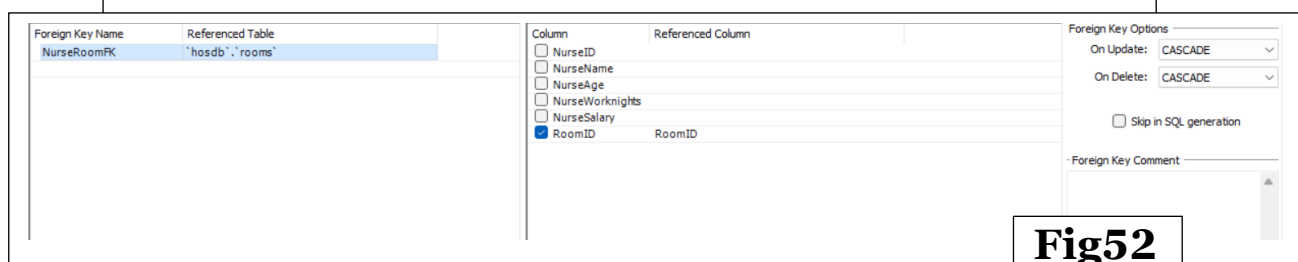
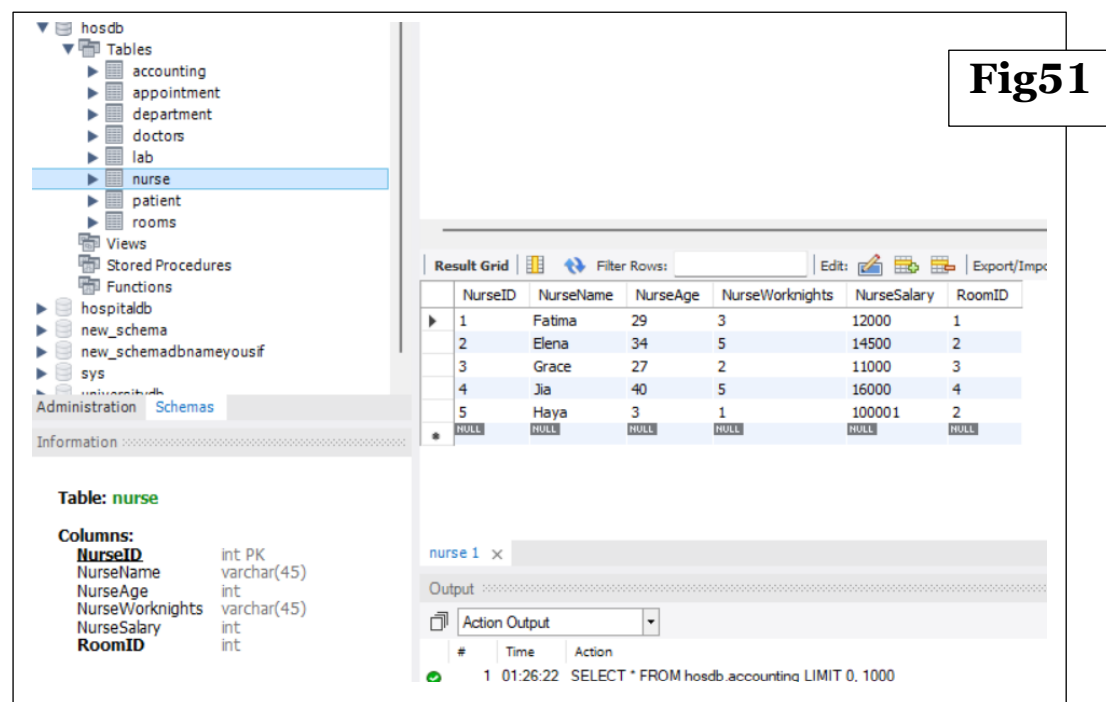
The table structure includes:

- ❖ **LabID (Primary Key)** – uniquely identifies each lab record
- ❖ **LabManager** – stores the name of the lab supervisor
- ❖ **LabWorkingHrs** – defines the working schedule of the lab

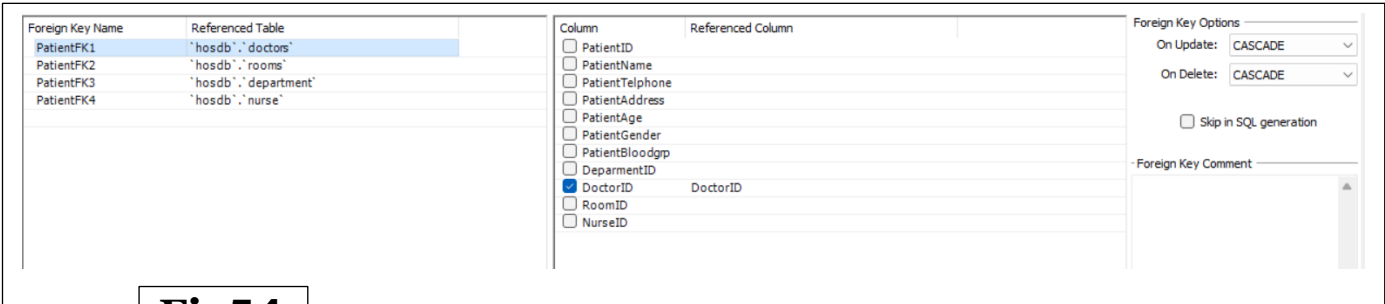
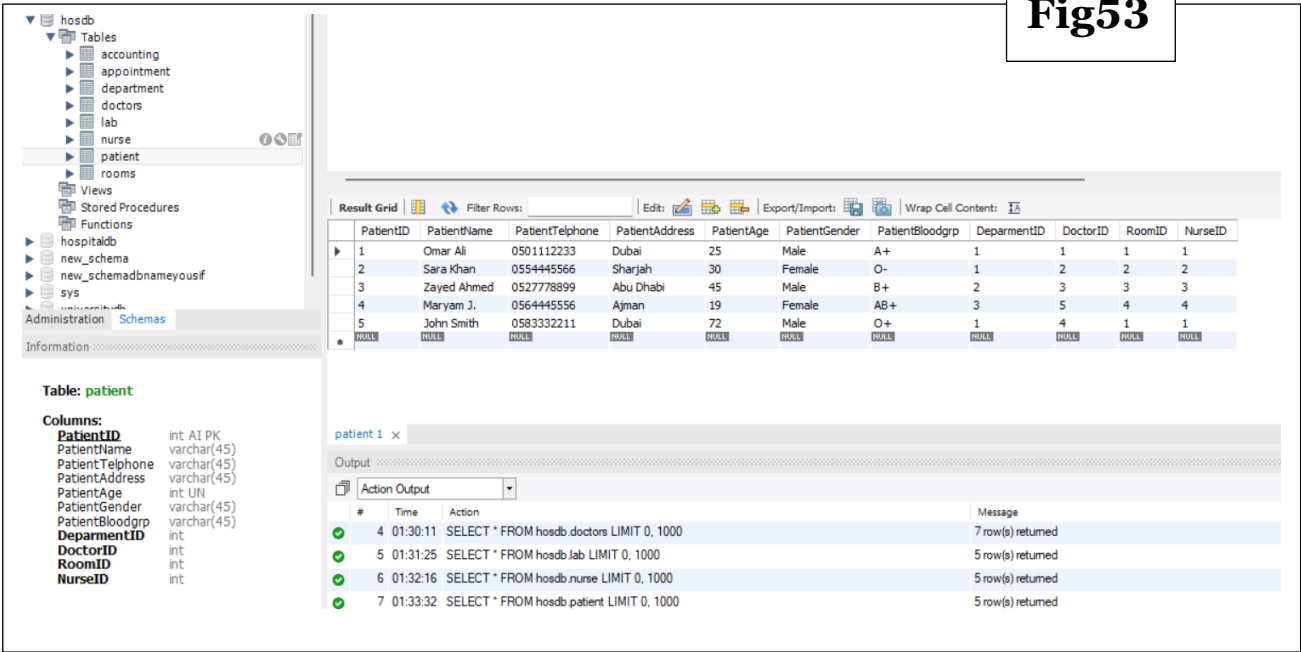
- ❖ **DoctorID (Foreign Key)** – connects the lab to a specific doctor

From a relational database perspective, the most critical aspect shown in these figures is the **foreign key relationship between the lab table and the doctors table**.

This relationship ensures that each lab is always assigned to a valid doctor, meaning no lab record can exist without referencing an existing doctor, which directly maintains data consistency across the system. In addition, the use of **CASCADE update and delete options** guarantees that any changes made to the doctor's ID are automatically reflected in the lab table, and if a doctor is deleted, all related lab records are handled accordingly. This mechanism prevents orphan records and preserves referential integrity within the database. From a system functionality perspective, this design supports efficient tracking of lab responsibilities, establishes a clear association between doctors and lab operations, and enables simplified querying, such as retrieving all labs managed by a specific doctor.



As illustrated in **Figures 51 and 52**, the *nurse* table is designed to manage nursing staff information while maintaining a direct relationship with the rooms table through the **RoomID foreign key**. This ensures that each nurse is assigned to a specific room, supporting better organization of patient care and hospital resources. The structure includes key attributes such as NurseID, NurseName, NurseAge, NurseWorkingHrs, and NurseSalary, allowing the system to store both personal and operational details. The foreign key constraint enforces that every nurse must be linked to a valid room, while the use of **CASCADE update and delete rules** ensures that any changes in the rooms table are automatically reflected in the nurse table. This design prevents inconsistencies and orphan records, and from a functional perspective, it improves resource allocation, simplifies tracking of staff responsibilities, and supports efficient management of hospital operations.



(Karwin, 2017)

As illustrated in **Figures 53 and 54**, the *patient* table represents one of the most important and complex entities in the hospital database, as it connects to multiple other tables within the system. The table includes key attributes such as PatientID (Primary Key with auto-increment), PatientName, PatientTelephone, PatientAddress, PatientAge, PatientGender, and PatientBloodgrp, which store essential personal and medical information. In addition, the table contains several **foreign keys**, including DepartmentID, DoctorID, RoomID, and NurseID, which establish relationships with other entities in the database. These relationships ensure that each patient is properly linked to a department, assigned doctor, room, and nurse, reflecting real-world hospital operations. The use of **CASCADE update and delete rules** maintains referential integrity by automatically updating or handling related records when changes occur in parent tables. This design prevents data inconsistency and duplication while enabling efficient data retrieval. From a functional perspective, this structure allows the system to fully track a patient's journey within the hospital, making it easier to manage care, resources, and administrative processes. (Karwin, 2017)

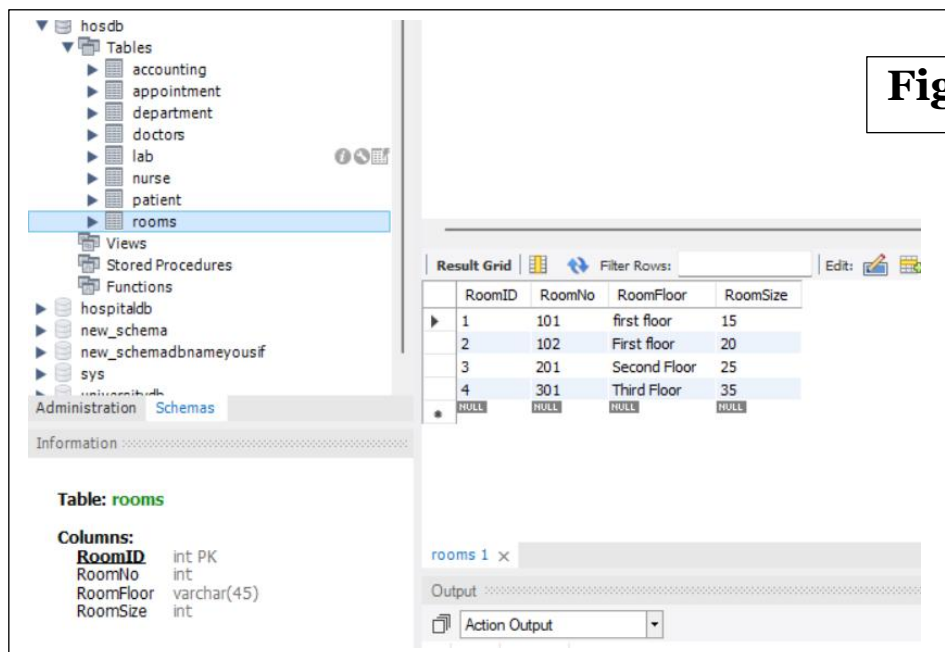


Fig55

RoomID	RoomNo	RoomFloor	RoomSize
1	101	first floor	15
2	102	First floor	20
3	201	Second Floor	25
4	301	Third Floor	35
NULL	NULL	NULL	NULL

Table: rooms

Columns:

- RoomID** int PK
- RoomNo int
- RoomFloor varchar(45)
- RoomSize int

As shown in **Figure 55**, the *rooms* table is implemented to manage the physical resources of the hospital, specifically patient room allocation. The table includes attributes such as **RoomID (Primary Key)**, **RoomNo**, **RoomFloor**, and **RoomSize**, which together provide detailed information about each room within the hospital.

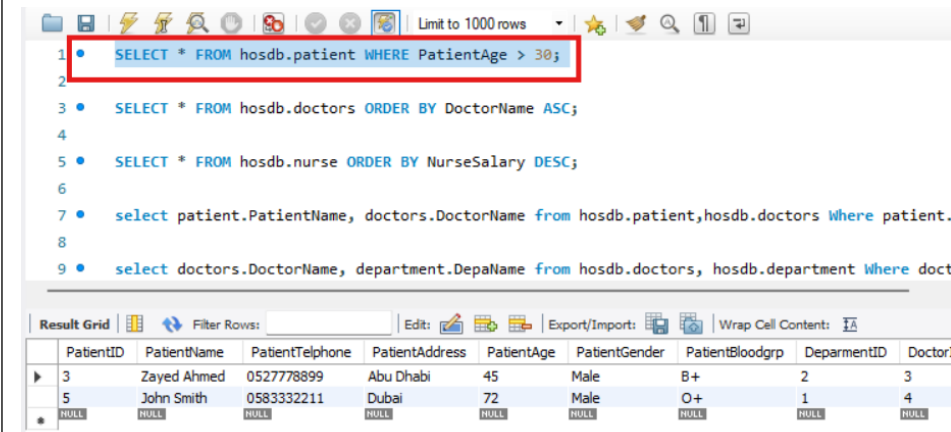
From a relational perspective, this table plays an important supporting role by linking with other entities such as the *patient* and

nurse tables through the RoomID foreign key. This ensures that each patient and nurse can be assigned to a valid room, improving organization and resource management within the system.

Additionally, the structured design of this table allows efficient tracking of room availability and allocation. It also supports better operational planning, as hospital staff can easily identify room locations and capacities. Overall, this table contributes to maintaining a well-organized and scalable hospital management system.

Powerful Query Processing and Data Retrieval

In this section, multiple SQL queries were implemented to demonstrate the system's ability to retrieve, filter, sort, and combine data across different tables. These queries reflect real-world hospital operations by allowing efficient access to specific information based on user needs.



```

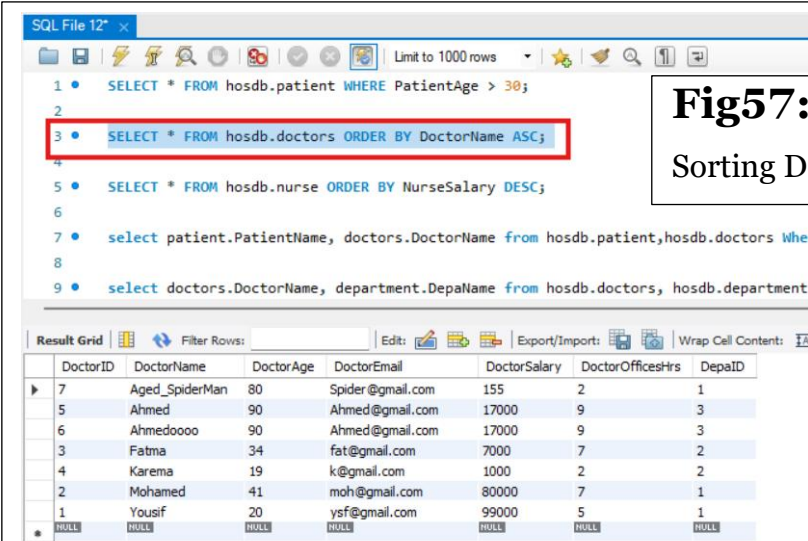
1 • SELECT * FROM hosdb.patient WHERE PatientAge > 30;
2
3 • SELECT * FROM hosdb.doctors ORDER BY DoctorName ASC;
4
5 • SELECT * FROM hosdb.nurse ORDER BY NurseSalary DESC;
6
7 • select patient.PatientName, doctors.DoctorName from hosdb.patient, hosdb.doctors where patient.DoctorID = doctors.DoctorID;
8
9 • select doctors.DoctorName, department.DepaName from hosdb.doctors, hosdb.department where doctors.DepaID = department.DepaID;

```

PatientID	PatientName	PatientTelephone	PatientAddress	PatientAge	PatientGender	PatientBloodgrp	DepartmentID	DoctorID	RoomID	NurseID
3	Zayed Ahmed	0527778899	Abu Dhabi	45	Male	B+	2	3	3	3
5	John Smith	0583332211	Dubai	72	Male	O+	1	4	1	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig56:

Filtering Patients by Age



```

1 • SELECT * FROM hosdb.patient WHERE PatientAge > 30;
2
3 • SELECT * FROM hosdb.doctors ORDER BY DoctorName ASC;
4
5 • SELECT * FROM hosdb.nurse ORDER BY NurseSalary DESC;
6
7 • select patient.PatientName, doctors.DoctorName from hosdb.patient, hosdb.doctors where patient.DoctorID = doctors.DoctorID;
8
9 • select doctors.DoctorName, department.DepaName from hosdb.doctors, hosdb.department where doctors.DepaID = department.DepaID;

```

DoctorID	DoctorName	DoctorAge	DoctorEmail	DoctorSalary	DoctorOfficeshrs	DepaID
7	Aged_SpiderMan	80	Spider@gmail.com	155	2	1
5	Ahmed	90	Ahmed@gmail.com	17000	9	3
6	Ahmedoooo	90	Ahmed@gmail.com	17000	9	3
3	Fatma	34	fat@gmail.com	7000	7	2
4	Karema	19	k@gmail.com	1000	2	2
2	Mohamed	41	moh@gmail.com	80000	7	1
1	Yousif	20	ysf@gmail.com	99000	5	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig57:

Sorting Doctors Alphabetically

```

5 • SELECT * FROM hosdb.nurse ORDER BY NurseSalary DESC;
6
7 • select patient.PatientName, doctors.DoctorName from hosdb
8
9 • select doctors.DoctorName, department.DepaName from hosdb

```

	NurseID	NurseName	NurseAge	NurseWorknights	NurseSalary	RoomID
▶	5	Haya	3	1	100001	2
	4	Jia	40	5	16000	4
	2	Elena	34	5	14500	2
	1	Fatima	29	3	12000	1
	3	Grace	27	2	11000	3
•	NULL	NULL	NULL	NULL	NULL	NULL

Fig58:

Sorting Nurses by Salary

```

7 • select patient.PatientName, doctors.DoctorName from hosdb.patient, hosdb.doctors Where patient.DoctorID = doctors.DoctorID;
8
9

```

	PatientName	DoctorName
▶	Omar Ali	Yousif
	Sara Khan	Mohamed
	Zayed Ahmed	Fatma
	Maryam J.	Ahmed
	John Smith	Karema

Fig59

```

12 • select doctors.DoctorName, department.DepaName from hosdb.doctors, hosdb.department Where doctors.DepaID = department.DepaID;
13
14 • select nurse.NurseName, rooms.RoomNo From hosdb.nurse, hosdb.rooms Where nurse.RoomID = rooms.RoomID;

```

	DoctorName	DepaName
▶	Yousif	cardiology
	Mohamed	cardiology
	Fatma	Emergency
	Karema	Emergency
	Ahmed	Surgery
	Ahmedoooo	Surgery
	Aged_SpiderMan	cardiology

Fig60

```

16 • select nurse.NurseName, patient.PatientName from hosdb.nurse, hosdb.patient where nurse.NurseID = patient.NurseID;
17
18 • select patient.PatientName, department.DepaName from hosdb.patient, hosdb.department where patient.DeparmentID = de

```

	NurseName	PatientName
▶	Fatima	Omar Ali
	Fatima	John Smith
	Elena	Sara Khan
	Grace	Zayed Ahmed
	Jia	Maryam J.

Fig62:

Linking Nurses with Patients

```

20 • select patient.PatientName, appointment.AppNo from hosdb.appointment, hosdb.patient where appointment.PatientID = patient.PatientID;
21
22 • select doctors.DoctorName, lab.LabWorknights from hosdb.doctors, hosdb.lab where doctors.DoctorID = lab.DoctorID;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
PatientName	AppNo		
▶ Omar Ali	3		
Sara Khan	4		
Sara Khan	6		
Zayed Ahmed	2		
Maryam J.	1		
John Smith	5		

Fig64

```

22 • select doctors.DoctorName, lab.LabWorknights from hosdb.doctors, hosdb.lab where doctors.DoctorID = lab.DoctorID;
23
24

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
DoctorName	LabWorknights		
▶ Yousif	9 to 11 AM		
Mohamed	11 to 1 PM		
Fatma	12 to 3 PM		
Yousif	1 to 4 PM		
Karema	5 to 12 Am		

Fig65

```

14 • select nurse.NurseName, rooms.RoomNo From hosdb.nurse, hosdb.rooms where nurse.RoomID = rooms.RoomID;
15
16 • select nurse.NurseName, patient.PatientName from hosdb.nurse, hosdb.patient where nurse.NurseID = pat

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
NurseName	RoomNo		
▶ Fatima	101		
Elena	102		
Grace	201		
Jia	301		
Haya	102		

Fig61

```

18 • select patient.PatientName, department.DepaName from hosdb.patient, hosdb.department where patient.DepartmentID = department.DepaID;
19
20 • select patient.PatientName, appointment.AppNo from hosdb.appointment, hosdb.patient where appointment.PatientID = patient.PatientID;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
PatientName	DepaName		
▶ Omar Ali	cardiology		
Sara Khan	cardiology		
Zayed Ahmed	Emergency		
Maryam J.	Surgery		
John Smith	cardiology		

Fig63

The query in Figure 56 retrieves all patients whose age is greater than 30 using a **WHERE condition**. This type of query is important for filtering specific groups of patients, such as identifying older patients who may require special medical attention.

As shown in Figure 57, the query uses **ORDER BY DoctorName ASC** to sort doctors in ascending alphabetical order. This improves readability and helps users quickly locate specific doctors within the system.

In Figure 58, the query sorts nurses based on salary in descending order using **ORDER BY NurseSalary DESC**. This allows administrators to easily analyze salary distribution and identify the highest-paid staff members.

Figure 59 demonstrates a query that combines data from both the *patient* and *doctors* tables using **FROM and WHERE conditions**. By matching DoctorID in both tables, the system retrieves patient names alongside their assigned doctors. This is essential for understanding doctor–patient relationships.

As illustrated in Figure 60, this query joins the *doctors* and *department* tables to display each doctor along with their department name. This improves organizational clarity and helps in managing hospital departments efficiently.

Figure 61 shows a query that connects the *nurse* and *rooms* tables using RoomID. The result displays nurse names alongside their assigned room numbers, which is crucial for tracking staff allocation and room management.

As shown in **Figure 62**, this query joins the *nurse* and *patient* tables by matching **NurseID**. The result displays nurse names alongside the patients they are responsible for. This is important for tracking caregiving relationships and ensures that each patient is clearly assigned to a nurse, improving coordination and accountability in hospital operations.

In **Figure 63**, the query connects the *patient* table with the *department* table using **DepartmentID**. This allows the system to display each patient along with their assigned department. Such functionality is essential for organizing patients based on medical specialization, such as cardiology or surgery, and supports better administrative management.

As illustrated in **Figure 64**, this query joins the *patient* and *appointment* tables using **PatientID**. The result shows patient names alongside their appointment numbers (AppNo). This enables efficient tracking of patient appointments and helps manage scheduling within the hospital system. (*Churcher, 2012*)

In the final query shown as **Figure 65**, the *doctors* and *lab* tables are joined using **DoctorID**. The result displays doctor names along with their associated lab working hours. This provides insight into doctor involvement in lab operations and supports better planning of medical resources.

Data Validation

Data validation is implemented in the system to ensure that only correct and meaningful data is entered into the database. This is essential in a hospital system where incorrect data could lead to serious operational issues. Validation is applied both at the application level (C#) and database level (MySQL constraints).

At the application level, numeric fields such as IDs, age, and salary use **int.Parse()**, which prevents users from entering non-numeric values. If invalid input is entered, the system will throw an error, ensuring that only valid numbers are stored. Additionally, text fields such as names and emails are captured using textboxes, ensuring structured input.

At the database level, **Primary Keys** ensure that each record is unique, preventing duplicate entries. Furthermore, **Foreign Keys** enforce relationships between tables, ensuring that referenced values (such as DoctorID or DepartmentID) must already exist in their respective tables. This prevents invalid or orphan records.

Validation Type	Where Applied	Example	Purpose
Data Type Validation	C# Code	int.Parse(textBox.Text)	Prevents entering text in numeric fields
Required Fields	UI (Textbox)	Empty DoctorName not allowed	Ensures all necessary data is entered
Primary Key Constraint	MySQL	DoctorID	Prevents duplicate records
Foreign Key Constraint	MySQL	DoctorID in Patient table	Ensures valid relationships
Logical Validation	UI / System	Age must be realistic	Prevents invalid real-world data

(Churcher, 2012; Flejoles, 2018)

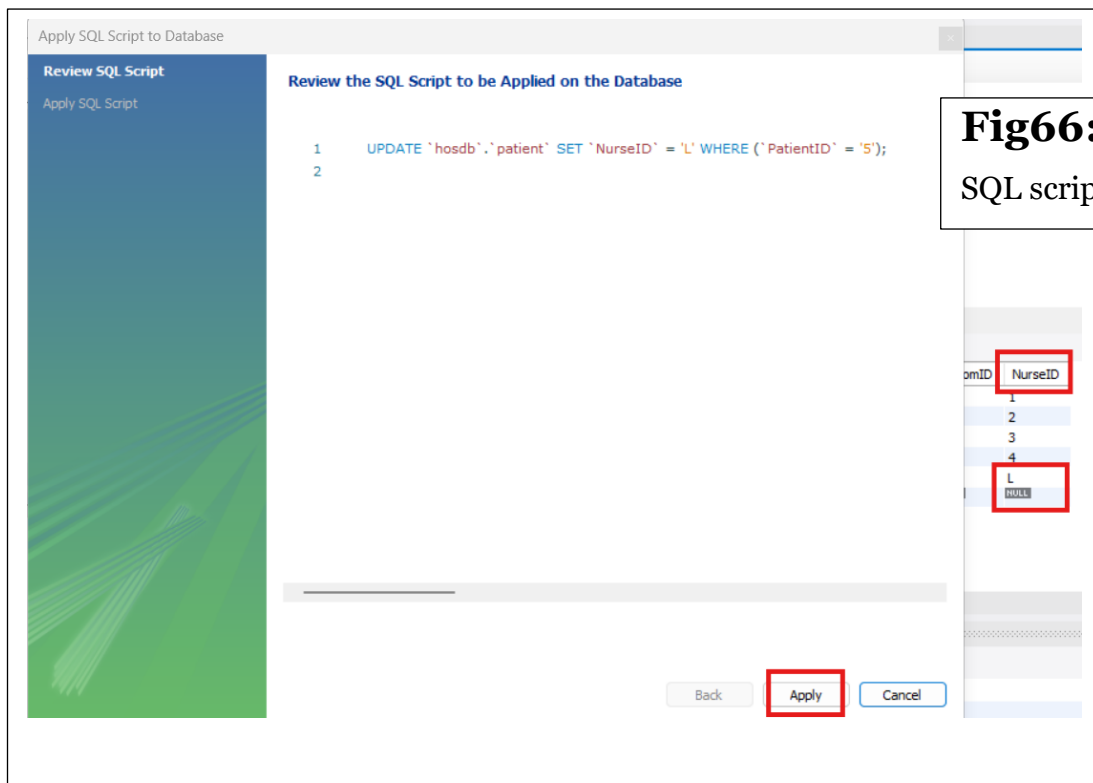


Fig66:
SQL script before applying

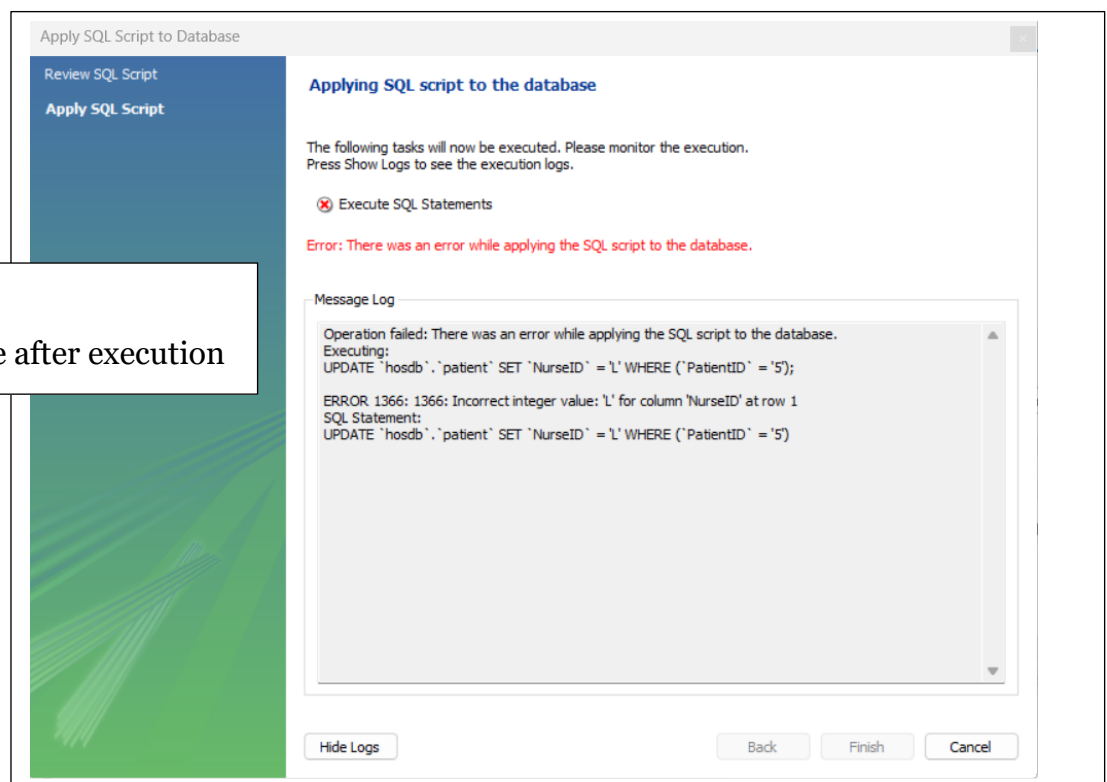


Fig67:
Error message after execution

As shown in **Figure 62**, an SQL update script was prepared to modify the NurseID value in the patient table. The script attempts to assign a value 'L' to the NurseID field for a specific patient.

In **Figure 63**, the system returns an error message during execution, indicating:

Incorrect integer value: 'L' for column 'NurseID'

This error occurs because the **NurseID** field is defined as an **integer**, while the value 'L' is a string. This demonstrates how MySQL enforces **data type validation at the database level**.

SQL Scripts Section

The database system is implemented using structured SQL scripts in MySQL Workbench. These scripts are responsible for creating tables, inserting records, and retrieving data.

The **SELECT queries** demonstrated earlier (Figures 56–65) show how data can be filtered, sorted, and joined across multiple tables. These scripts simulate real-world hospital operations, such as retrieving patient information, assigning doctors, and tracking appointments.

Additionally, **INSERT statements** are used to populate tables with initial data, while **UPDATE and DELETE operations** allow modification and removal of records when needed.

Operation	Example Used	Purpose
SELECT	SELECT * FROM patient WHERE PatientAge > 30	Retrieve filtered data
ORDER BY	ORDER BY DoctorName ASC	Sort data
INSERT	Insert patient/doctor records	Add new data
UPDATE	Update doctor salary	Modify data
DELETE	Delete incorrect patient	Remove data
JOIN	patient + doctor	Combine multiple tables

The outputs generated from the SQL queries confirm that the database system functions correctly. The results displayed in MySQL Workbench show accurate filtering, sorting, and relational data retrieval. These outputs demonstrate that the system can efficiently handle real-world hospital data scenarios.

Final Evidence Mapping

Requirement	Evidence
Database Implementation	Figures 38–41
Tables Created	Figures 42–55
Data Inserted	Doctors/Patients UI screenshots
User Interface	Forms (Doctors, Patients, etc.)
Query Execution	Figures 56–65
Data Validation	Figure 66-67
SQL Scripts	Figure 56-65

Final System Diagram

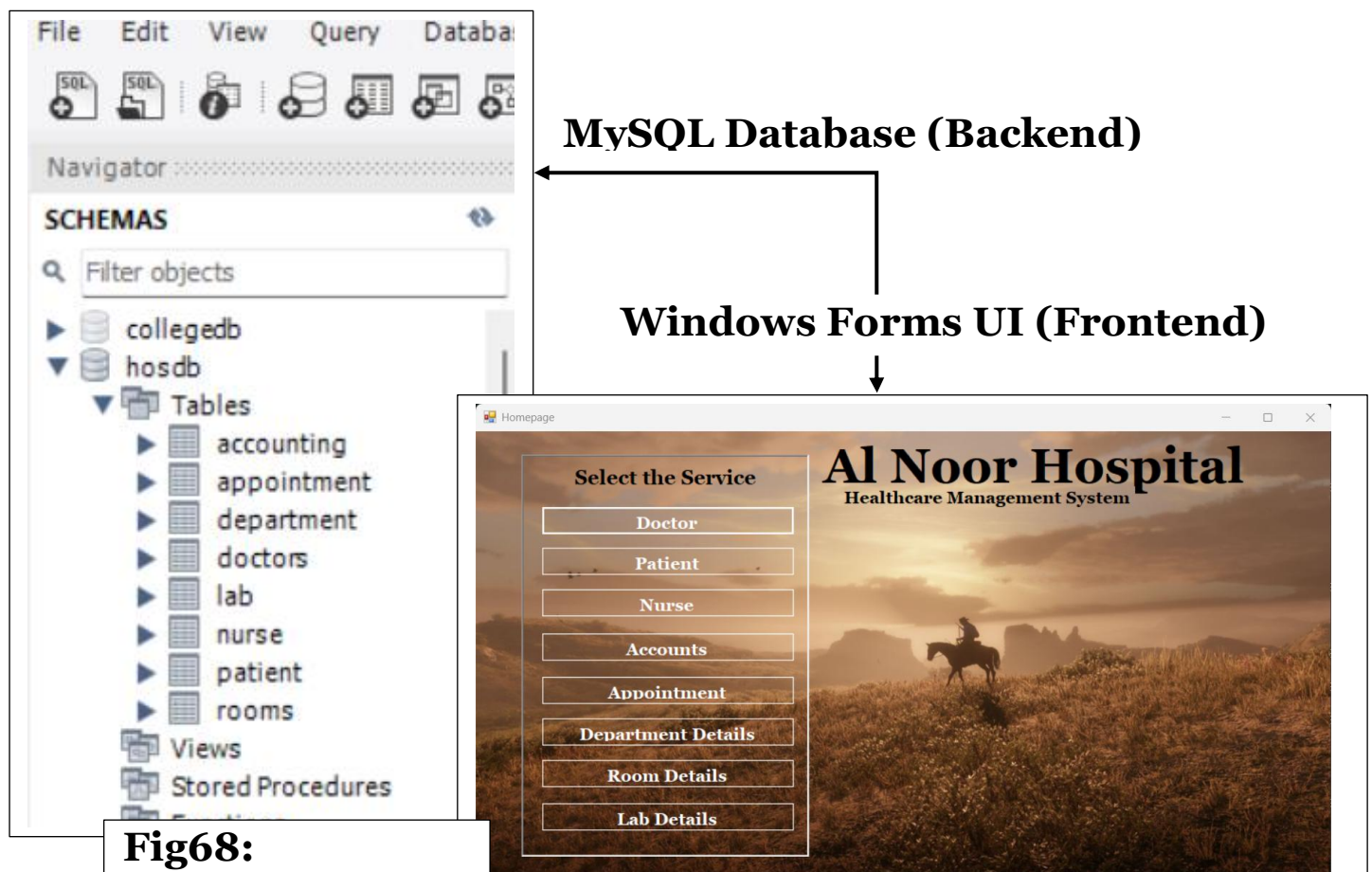


Fig68:

System Representation

Moreover, Query Implementation in a Relational Database System for Efficient Data Management

Practical implementation of SQL queries is a fundamental component in managing relational database systems effectively. In this project, Structured Query Language (SQL) was used to interact with the MySQL database, enabling operations such as retrieving, filtering, sorting, and combining data across multiple related tables. These queries allow the system to transform raw stored data into meaningful information that supports hospital operations such as patient management, doctor assignments, and appointment tracking. The following sections demonstrate how different SQL queries were implemented, supported with screenshots and outputs from the developed system.

Overview of SQL Queries Used

Query Type	Purpose	Example Use in System
SELECT	Retrieve data	Display patients/doctors
WHERE	Filter data	Patients older than 30
ORDER BY	Sort data	Doctors by name
INSERT	Add new records	Add patient
UPDATE	Modify records	Update doctor info
DELETE	Remove records	Delete wrong entry
JOIN (FROM + WHERE)	Combine tables	Patient + Doctor

Data Retrieval Queries (SELECT & WHERE)

Data retrieval queries are essential for extracting and organizing information from the database. In this system, the SELECT statement was used with conditions and sorting to display relevant data efficiently. As shown in **Figure 56**, the WHERE clause was applied to filter patients older than 30, allowing focused data retrieval. Additionally, sorting was implemented using ORDER BY, where **Figure 57** shows doctors arranged alphabetically in ascending order, and **Figure 58** presents nurses sorted by salary in descending order for comparison purposes. These queries demonstrate how SELECT, WHERE, and ORDER BY work together to provide accurate, structured, and meaningful data output within the system. (*Chandra, 1988*)

Sorting and Filtering Logic

Sorting and filtering logic enhance the usability and clarity of data within the system by allowing users to retrieve only relevant and well-organized information. As demonstrated in **Figures 56–58**, the WHERE clause is used to apply conditions that filter specific records, while the ORDER BY clause arranges the output either in ascending or descending order. This combination improves data readability, supports better decision-making, and ensures that users can efficiently interact with large datasets by focusing only on the information that meets their requirements. (*Chandra, 1988*)

Multi-Table Queries (JOIN Operations)

Multi-table queries are used to combine data from different related tables, allowing the system to display meaningful information that cannot be obtained from a single table alone. As shown in **Figures 59–65**, relationships between tables are established using common fields such as DoctorID, PatientID, RoomID, and DepartmentID. These queries enable the system to link patients with doctors, doctors with departments, nurses with rooms, and other related entities. This improves data integration, reflects real-world hospital relationships, and supports efficient data retrieval across the system. (*Connolly and Begg, 2014*)

Figure	Tables Linked	Purpose
Fig 59	Patient + Doctor	Show patient with assigned doctor
Fig 60	Doctor + Department	Show doctor's department
Fig 61	Nurse + Rooms	Assign nurses to rooms
Fig 62	Nurse + Patient	Show nurse responsible for patient
Fig 63	Patient + Department	Identify patient's department
Fig 64	Patient + Appointment	Track patient appointments
Fig 65	Doctor + Lab	Show doctor lab involvement

Data Manipulation Queries (INSERT, UPDATE, DELETE)

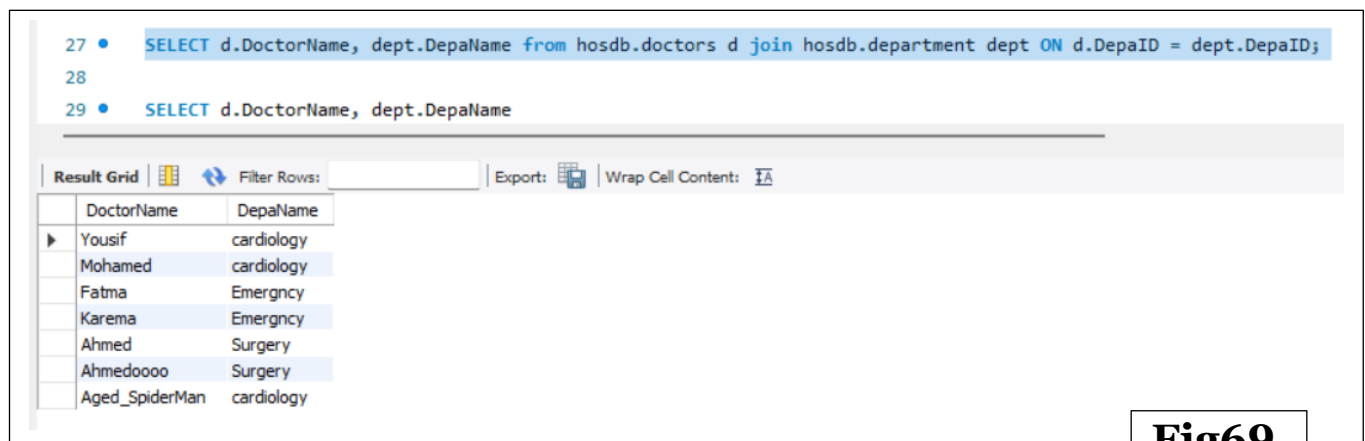
Data manipulation queries are used to modify the data stored in the database, allowing the system to remain dynamic and up to date. In this system, INSERT, UPDATE, and DELETE operations were implemented to manage records efficiently. These operations are integrated within the user interface, where users can add new data, update existing information, or remove incorrect records. This functionality ensures that the database reflects real-time changes and supports accurate hospital data management.

Error Handling and Data Validation

Error handling and data validation are essential to ensure that only correct and meaningful data is stored in the database. As demonstrated in **Figures 66 and 67**, the system enforces strict data type rules, where an attempt to insert an invalid value (such as a character into an integer field) results in an SQL error. This prevents incorrect data from being stored and maintains the integrity of the database. Such validation ensures that all inputs follow the required format and that relationships between tables remain consistent. (*Karwin, 2017*)

Validation Type	Example	System Response	Benefit
Data Type Validation	NurseID = 'L'	SQL Error (Incorrect integer value)	Prevents invalid data entry
Primary Key Constraint	Duplicate ID	Rejected	Ensures uniqueness
Foreign Key Constraint	Invalid reference ID	Rejected	Maintains relationships
Input Validation	Wrong format data	Error / not accepted	Improves data accuracy

Implementation of Advanced SQL JOIN Queries



The screenshot shows a SQL query editor with two queries. Query 27 is an INNER JOIN query: `SELECT d.DoctorName, dept.DepaName from hosdb.doctors d join hosdb.department dept ON d.DepaID = dept.DepaID;`. Query 29 is a simple SELECT: `SELECT d.DoctorName, dept.DepaName`. Below the queries is a result grid with columns 'DoctorName' and 'DepaName'. The grid contains seven rows of data, including 'Yousif' in 'cardiology', 'Mohamed' in 'cardiology', 'Fatma' in 'Emergency', 'Karema' in 'Emergency', 'Ahmed' in 'Surgery', 'Ahmedoooo' in 'Surgery', and 'Aged_SpiderMan' in 'cardiology'.

DoctorName	DepaName
Yousif	cardiology
Mohamed	cardiology
Fatma	Emergency
Karema	Emergency
Ahmed	Surgery
Ahmedoooo	Surgery
Aged_SpiderMan	cardiology

Fig69

Figure 69 demonstrates the use of an INNER JOIN to combine data from the doctors and department tables. The query links both tables using the DepaID field, which acts as a common key between them. As a result, the output displays each doctor's name alongside their corresponding department name. This type of query is essential for retrieving related data stored across multiple tables in a structured and meaningful way.

The outputs generated from the SQL queries are displayed in the result grid, allowing users to clearly view and interpret the retrieved data. These results confirm that the queries are functioning correctly and demonstrate how data can be filtered, sorted, and combined across multiple tables. This ensures that the system provides accurate and meaningful information to support hospital operations.

Implementation of Database Security and Maintenance Strategies

Managing Database security and maintenance are essential components in ensuring that a relational database system operates reliably and safely. In this project, several security measures were implemented to protect sensitive hospital data from unauthorized access, while maintenance strategies were considered to preserve data integrity and system stability. These practices ensure that the database remains accurate, secure, and efficient during continuous use.

Database Security Implementation

Database security was implemented in this system through authentication and controlled access mechanisms. The connection between the application and the MySQL database requires valid credentials, including a username and password, as demonstrated in **Figure 35/35.1** and the MySQL login interface. This ensures that only authorized users can access the database. Additionally, the use of a connection string within the application controls how the system connects to the database, preventing direct unauthorized interaction. These measures protect sensitive hospital data such as patient records, doctor information, and appointments from potential misuse or breaches. (*Bertino and Sandhu, 2005*)

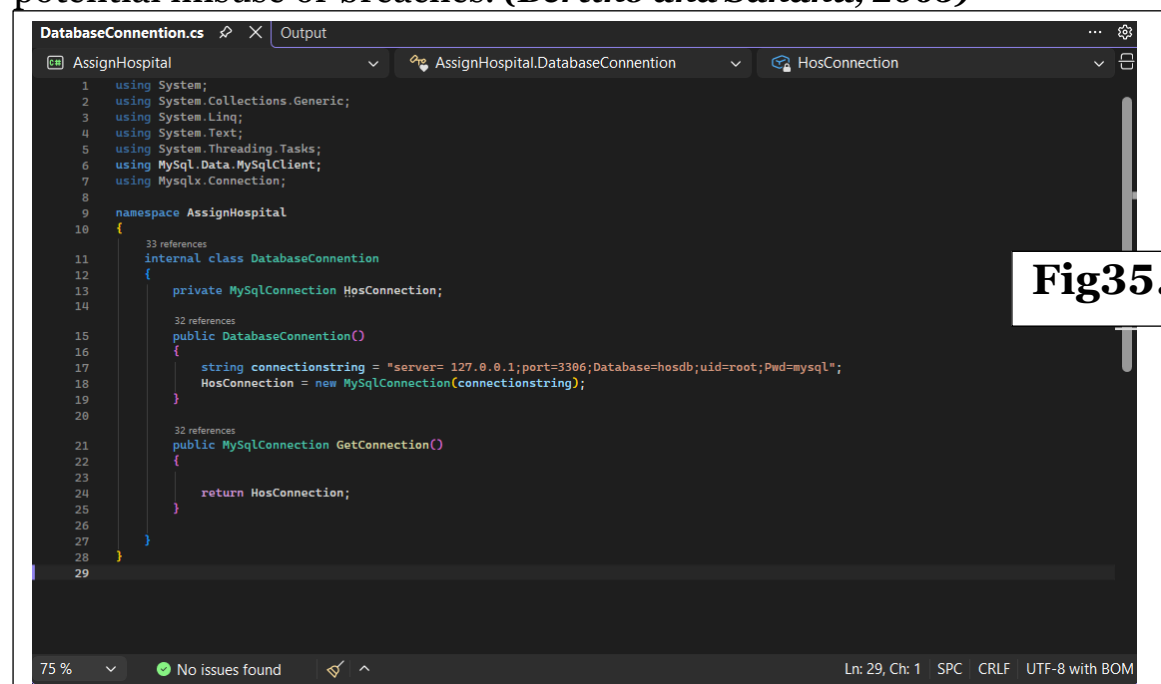


Fig35.1

Security Feature	Implementation	Purpose
Username & Password	MySQL authentication	Prevent unauthorized access
Connection String	C# DatabaseConnection class	Secure controlled connection
Application Layer Access	UI forms	Restrict direct database manipulation

Data Validation as a Security Measure

Data validation plays a crucial role in protecting the database from incorrect or harmful inputs. As shown in **Figures 66 and 67 and M.1/2/3/4/5/6**, an error occurred when an invalid value ('L') was inserted into an integer field (NurseID). The database rejected this input, demonstrating strict enforcement of data types. This validation prevents corruption of data, ensures consistency across related tables, and protects the system from logical errors. By enforcing these constraints, the system maintains high data quality and reliability. *(Microsoft, 2022)*

Database Maintenance Practices

Maintaining a database is not a one-time task but a continuous process that ensures long-term performance, reliability, and data accuracy. In this system, maintenance practices are designed to regularly monitor, update, and protect the database from errors, data loss, and performance issues. These activities help keep the hospital management system stable and efficient, even as data grows over time.

Frequency	Maintenance Task	Purpose
Daily	Monitor system queries and activity	To ensure smooth operation and detect unusual behavior
Daily	Check for input errors and failed operations	To identify incorrect data entries and prevent corruption
Weekly	Perform full database backup	To protect data from loss and enable recovery
Weekly	Verify relationships between tables (FK checks)	To maintain referential integrity across the system

Weekly	Update records and remove incorrect entries	To keep data accurate and relevant
Monthly	Review database performance	To ensure queries run efficiently and optimize execution
Monthly	Validate data consistency	To detect and correct inconsistencies in stored data
Monthly	Check user access and permissions	To ensure only authorized access is maintained
Quarterly	Review system structure and relationships	To ensure database design still meets system needs
Quarterly	Optimize queries and indexing	To improve system speed and efficiency
Semi-Annually	Test backup and recovery process	To confirm that data can be restored successfully
Annually	Review and update database schema if needed	To adapt to system growth and new requirements

Space for Improvement and Security Assessment

Although the implemented database system includes essential security and maintenance features, there is still room for improvement to enhance system protection, reliability, and scalability. By evaluating the current implementation in Sections 4, 5, and 6, several areas can be improved, particularly in strengthening security measures, automating maintenance processes, and applying more advanced database protection techniques. These improvements would make the system more suitable for real-world deployment in a hospital environment.

Area (Section)	Current Implementation	Limitation	Suggested Improvement	Benefit
Backup & Recovery	Basic understanding of backup	No actual automated backup system	Implement scheduled automatic backups	Ensures continuous data protection
Section 4: Recovery	Conceptual recovery only	No real recovery testing	Perform regular restore testing	Confirms backup reliability

Authentication	Username & password login	Credentials stored in plain connection string	Use encrypted credentials or environment variables	Improves security and prevents exposure
Access Control	Basic application-level control	No role-based access (admin/user)	Implement role-based access control (RBAC)	Restricts user permissions effectively
Data Protection	Validation enforced	No data encryption	Apply encryption for sensitive data	Protects confidential information
Error Handling	SQL errors displayed	User sees raw database errors	Implement user-friendly error messages	Improves usability and security
Validation	Data type validation only	Limited validation rules	Add input constraints (length, format, range)	Enhances data accuracy
Maintenance	Manual maintenance tasks	No automation	Use scheduled maintenance scripts	Reduces human error
Query Security	Parameterized queries used	No protection against advanced attacks	Implement additional security layers (e.g. hashing, logging)	Prevents SQL injection and attacks

Meaningful Analysis of Queries for Management Decision-Making

Meaningful data extraction is a fundamental aspect of database systems, as it transforms raw records into valuable information that supports management decision-making. In this system, multiple SQL queries (as demonstrated in P3) were implemented to retrieve, filter, and combine data across different tables. Each query contributes to providing insights into hospital operations, including patient management, staff organization, and resource allocation, enabling more informed and effective decisions.

Query No.	Query Description	SQL Operation Used	Purpose	Management Insight	Decision-Making Impact
Q1 (Fig 56)	Filter patients with age > 30	WHERE	Identify specific patient group	Shows older patients	Helps prioritize healthcare services
Q2 (Fig 57)	Sort doctors by name	ORDER BY	Organize doctor records	Structured doctor list	Easier staff management
Q3 (Fig 58)	Sort nurses by salary (DESC)	ORDER BY	Analyze salary distribution	Identifies highest-paid nurses	Supports financial planning
Q4 (Fig 59)	Patient Doctor relationship +	JOIN / WHERE	Link patients with doctors	Shows treatment assignments	Helps balance doctor workload
Q5 (Fig 60)	Doctor Department +	JOIN	Identify doctor departments	Shows department structure	Helps staff allocation
Q6 (Fig 61)	Nurse Room +	JOIN	Assign nurses to rooms	Shows resource distribution	Improves room management
Q7 (Fig 62)	Nurse Patient +	JOIN	Link nurses with patients	Shows care responsibility	Helps workload distribution
Q8 (Fig 63)	Patient Department +	JOIN	Identify patient department	Shows service usage	Helps department planning
Q9 (Fig 64)	Patient Appointment +	JOIN	Track appointments	Shows scheduling info	Improves appointment planning
Q10 (Fig 65)	Doctor + Lab schedule	JOIN	View doctor lab times	Shows availability	Helps schedule optimization
Q11 (Fig 69)	Doctor Department + (JOIN optimized)	INNER JOIN	Efficient relationship query	Cleaner structured data	Improves query performance

The queries implemented in this system successfully transform raw database records into meaningful management information. Filtering and sorting queries help in identifying priorities, while join operations provide deeper insights by combining related data across multiple tables. However, the system lacks advanced analytical queries such as aggregation functions (COUNT, SUM, AVG), which could further enhance decision-making by providing statistical insights.

Practical Testing of the Database System Against User and System Requirements

Proving that the developed database system functions correctly is essential to ensure it meets both user and system requirements. In this section, a structured testing approach is applied to evaluate system operations such as data insertion, updating, deletion, validation, and query execution. Each test case verifies whether the system behaves as expected under real usage conditions.

Testing Approach

The system was tested using multiple scenarios covering core functionalities, including user interface operations, database interactions, and query execution. The tests focus on validating correct data processing, enforcing constraints, and ensuring that outputs match expected results. Both successful operations and error handling cases were included to evaluate system reliability.

Test Plan Followed

Test Case ID	Requirement	Test Description	Input	Expected Result	Actual Result	Status
TC01 (Fig P.1-2)	UI Functionality	Access system interface and navigation	Open main system UI	System interface loads correctly	UI displayed successfully	Pass
TC02 (Fig P.3)	Data Filtering	Retrieve patients age > 30	WHERE PatientAge > 30	Filtered patient list shown	Correct patients displayed	Pass
TC03 (Fig P.4)	Data Sorting	Sort doctors alphabetically	ORDER BY DoctorName ASC	Sorted doctor list	Output sorted correctly	Pass

TC04 (Fig P.5)	Data Sorting	Sort nurses by salary DESC	ORDER BY NurseSalary DESC	Highest salary first	Output correct	Pass
TC05 (Fig P.6)	Multi-table Query	Link patient with doctor	JOIN patient & doctors	Combined data shown	Correct relation displayed	Pass
TC06 (Fig P.7)	Multi-table Query	Link doctor with department	JOIN doctors & department	Department info shown	Output correct	Pass
TC07 (Fig P.8)	Multi-table Query	Link nurse with room	JOIN nurse & rooms	Room assignment shown	Output correct	Pass
TC08 (Fig P.9)	Multi-table Query	Link nurse with patient	JOIN nurse & patient	Nurse-patient relation shown	Output correct	Pass
TC09 (Fig P.10)	Multi-table Query	Link patient with department	JOIN patient & department	Department displayed	Output correct	Pass
TC10 (Fig P.11)	Multi-table Query	Link patient with appointment	JOIN patient & appointment	Appointment data shown	Output correct	Pass
TC11 (Fig P.12)	Multi-table Query	Link doctor with lab schedule	JOIN doctors & lab	Lab timings displayed	Output correct	Pass
TC12 (Fig P.13)	Optimized Query	INNER JOIN doctor & department	JOIN with alias	Cleaner structured result	Output correct	Pass
TC13 (Fig P.14–15)	Data Validation	Insert invalid NurseID	NurseID = 'L'	Error message displayed	SQL error shown	Pass

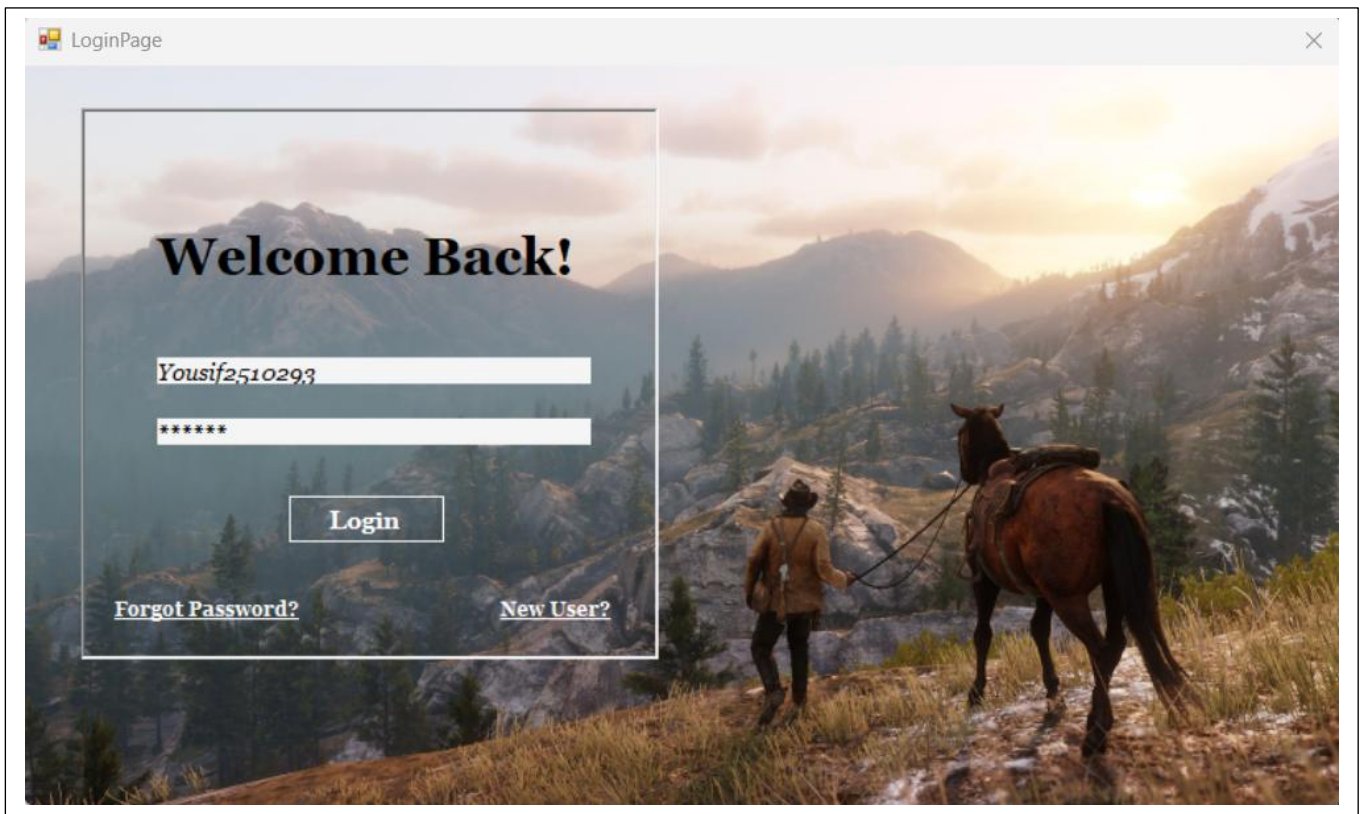


Fig P.1/P.2: TC01

Accessing the UI

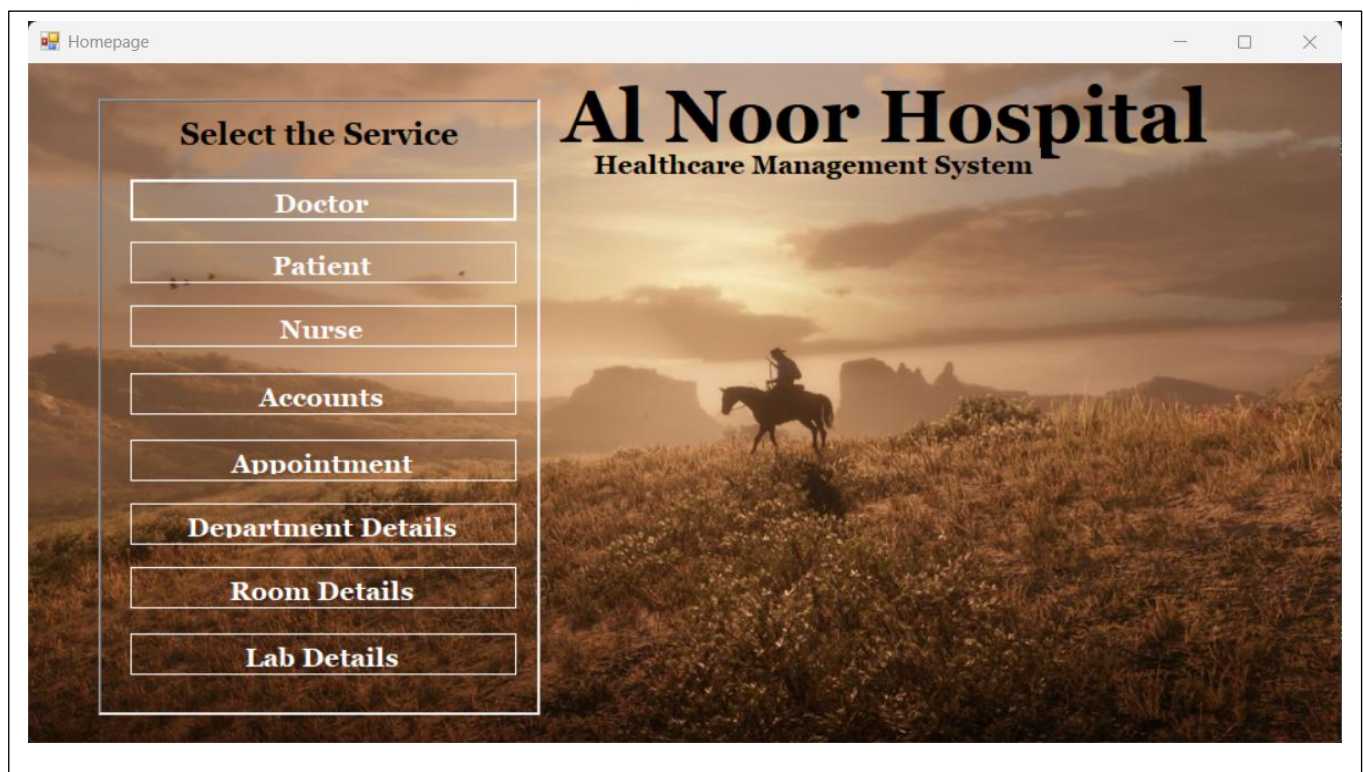


Fig P.3: TC02

SQL File 12* x

Limit to 1000 rows

```

1 • SELECT * FROM hosdb.patient WHERE PatientAge > 30;
2
3 • SELECT * FROM hosdb.doctors ORDER BY DoctorName ASC;
4
5 • SELECT * FROM hosdb.nurse ORDER BY NurseSalary DESC;
6
7 • select patient.PatientName, doctors.DoctorName from hosdb.patient, hosdb.doctors Where patient.DoctorID = doctors.DoctorID;
8
9 • select doctors.DoctorName, department.DepaName from hosdb.doctors, hosdb.department Where doctors.DepaID = department.DepaID;

```

Result Grid

PatientID	PatientName	PatientTelephone	PatientAddress	PatientAge	PatientGender	PatientBloodgrp	DepartmentID	DoctorID	RoomID	NurseID
3	Zayed Ahmed	0527778899	Abu Dhabi	45	Male	B+	2	3	3	3
5	John Smith	0583332211	Dubai	72	Male	O+	1	4	1	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig P.4: TC03

SQL File 12* x

Limit to 1000 rows

```

1 • SELECT * FROM hosdb.patient WHERE PatientAge > 30;
2
3 • SELECT * FROM hosdb.doctors ORDER BY DoctorName ASC;
4
5 • SELECT * FROM hosdb.nurse ORDER BY NurseSalary DESC;
6
7 • select patient.PatientName, doctors.DoctorName from hosdb.patient, hosdb.doctors Where
8
9 • select doctors.DoctorName, department.DepaName from hosdb.doctors, hosdb.department

```

Result Grid

DoctorID	DoctorName	DoctorAge	DoctorEmail	DoctorSalary	DoctorOfficesHrs	DepaID
7	Aged_SpiderMan	80	Spider@gmail.com	155	2	1
5	Ahmed	90	Ahmed@gmail.com	17000	9	3
6	Ahmedoooo	90	Ahmed@gmail.com	17000	9	3
3	Fatma	34	fat@gmail.com	7000	7	2
4	Karema	19	k@gmail.com	1000	2	2
2	Mohamed	41	moh@gmail.com	80000	7	1
1	Yousif	20	ysf@gmail.com	99000	5	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig P.5: TC04

SQL File 12* x

Limit to 1000 rows

```

5 • SELECT * FROM hosdb.nurse ORDER BY NurseSalary DESC;
6
7 • select patient.PatientName, doctors.DoctorName from hosdb
8
9 • select doctors.DoctorName, department.DepaName from hosdb

```

Result Grid

NurseID	NurseName	NurseAge	NurseWorknights	NurseSalary	RoomID
5	Haya	3	1	100001	2
4	Jia	40	5	16000	4
2	Elena	34	5	14500	2
1	Fatima	29	3	12000	1
3	Grace	27	2	11000	3
NULL	NULL	NULL	NULL	NULL	NULL

```

7 • select patient.PatientName, doctors.DoctorName from hosdb.patient, hosdb.doctors where patient.DoctorID = doctors.DoctorID;
8
9

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
PatientName	DoctorName		
▶ Omar Ali	Yousif		
Sara Khan	Mohamed		
Zayed Ahmed	Fatma		
Maryam J.	Ahmed		
John Smith	Karema		

Fig P.6: TC05

```

12 • select doctors.DoctorName, department.DepaName from hosdb.doctors, hosdb.department where doctors.DepaID = department.DepaID;
13
14 • select nurse.NurseName, rooms.RoomNo From hosdb.nurse, hosdb.rooms where nurse.RoomID = rooms.RoomID;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
DoctorName	DepaName		
▶ Yousif	cardiology		
Mohamed	cardiology		
Fatma	Emergency		
Karema	Emergency		
Ahmed	Surgery		
Ahmedoooo	Surgery		
Aged_SpiderMan	cardiology		

Fig P.7: TC06

```

14 • select nurse.NurseName, rooms.RoomNo From hosdb.nurse, hosdb.rooms where nurse.RoomID = rooms.RoomID;
15
16 • select nurse.NurseName, patient.PatientName from hosdb.nurse, hosdb.patient where nurse.NurseID = patient.NurseID;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
NurseName	RoomNo		
▶ Fatima	101		
Elena	102		
Grace	201		
Jia	301		
Haya	102		

Fig P.8: TC07

```

16 • select nurse.NurseName, patient.PatientName from hosdb.nurse, hosdb.patient where nurse.NurseID = patient.NurseID;
17
18 • select patient.PatientName, department.DepaName from hosdb.patient, hosdb.department where patient.DepartmentID = department.DepartmentID;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
NurseName	PatientName		
▶ Fatima	Omar Ali		
Fatima	John Smith		
Elena	Sara Khan		
Grace	Zayed Ahmed		
Jia	Maryam J.		

Fig P.9: TC08

```

18 • select patient.PatientName, department.DepaName from hosdb.patient, hosdb.department where patient.DepartmentID = department.DepaID;
19
20 • select patient.PatientName, appointment.AppNo from hosdb.appointment, hosdb.patient where appointment.PatientID = patient.PatientID;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	PatientName	DepaName
▶	Omar Ali	cardiology
	Sara Khan	cardiology
	Zayed Ahmed	Emergency
	Maryam J.	Surgery
	John Smith	cardiology

Fig P.10: TC09

```

20 • select patient.PatientName, appointment.AppNo from hosdb.appointment, hosdb.patient where appointment.PatientID = patient.Patie
21
22 • select doctors.DoctorName, lab.LabWorknights from hosdb.doctors, hosdb.lab where doctors.DoctorID = lab.DoctorID;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	PatientName	AppNo
▶	Omar Ali	3
	Sara Khan	4
	Sara Khan	6
	Zayed Ahmed	2
	Maryam J.	1
	John Smith	5

Fig P.11: TC010

```

22 • select doctors.DoctorName, lab.LabWorknights from hosdb.doctors, hosdb.lab where doctors.DoctorID = lab.D
23
24

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	DoctorName	LabWorknights
▶	Yousif	9 to 11 AM
	Mohamed	11 to 1 PM
	Fatma	12 to 3 PM
	Yousif	1 to 4 PM
	Karema	5 to 12 Am

Fig P.12: TC011

```

27 • SELECT d.DoctorName, dept.DepaName from hosdb.doctors d join hosdb.department dept ON d.DepaID = dept.DepaID;
28
29 • SELECT d.DoctorName, dept.DepaName

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	DoctorName	DepaName
▶	Yousif	cardiology
	Mohamed	cardiology
	Fatma	Emergency
	Karema	Emergency
	Ahmed	Surgery
	Ahmedoooo	Surgery
	Aged_SpiderMan	cardiology

Fig P.13: TC012

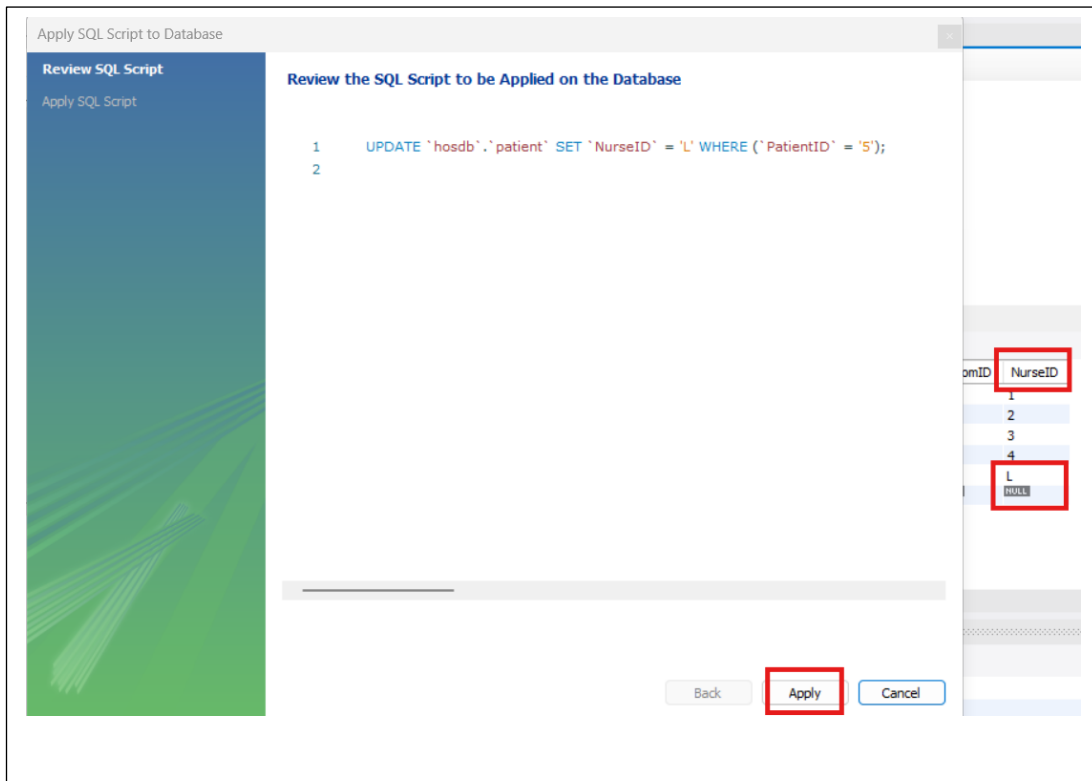
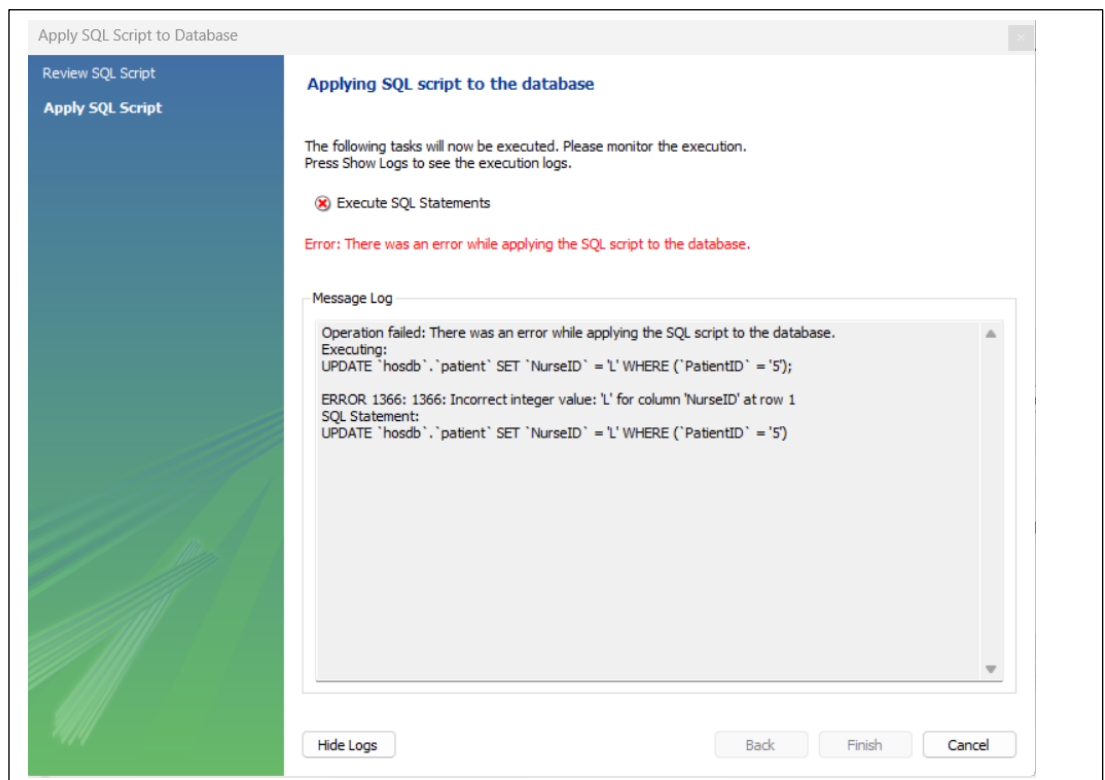


Fig P.14/P.15: TC013



Testing Results

The testing results confirm that the database system meets the required functionality and behaves as expected in all tested scenarios. Core operations such as inserting, updating, deleting, and querying data were successfully executed. Additionally, validation mechanisms effectively prevented incorrect inputs, ensuring data integrity and system reliability. Overall, the system demonstrates stable performance and accurate data processing.

Measured Evaluation of Testing Effectiveness and Test Data Selection

Measuring the effectiveness of system testing is essential to determine whether the developed database solution performs reliably under different conditions. This section evaluates the quality of the testing conducted in P4, justifies the choice of test data used, and critically assesses how well the testing ensures system accuracy, reliability, and usability.

Evaluation of Testing Quality

The testing carried out in P4 covered a wide range of system functionalities, including data retrieval, filtering, sorting, multi-table queries, and validation. The use of multiple test cases (Fig 56–69) ensured that both normal operations and relational queries were tested thoroughly. Additionally, validation testing (Fig 61–62) confirmed that the system correctly handles invalid inputs. Overall, the testing approach was structured and comprehensive, as it verified different components of the system rather than focusing on a single functionality.

Justification of Test Data Used

The selection of test data was appropriate as it included a combination of valid, invalid, and relational data. Valid data was used to confirm that the system performs expected operations correctly, while invalid data (such as inserting a non-integer value in NurseID) was used to test validation mechanisms. Furthermore, relational data involving matching foreign keys was used to test JOIN queries across multiple tables. This combination ensures that

the system is tested under realistic scenarios that reflect actual hospital operations.

Test Data Type	Example	Reason for Use	Benefit
Valid Data	Patient records, doctor data	To test normal system functionality	Confirms system works correctly
Invalid Data	NurseID = 'L'	To test validation and error handling	Prevents incorrect data entry
Filter Data	PatientAge > 30	To test WHERE conditions	Ensures correct filtering
Sorted Data	ORDER BY DoctorName	To test sorting logic	Improves data organization
Relational Data	Matching IDs in JOIN queries	To test relationships between tables	Ensures data integrity

Effectiveness of Testing

The testing was effective in demonstrating that the system meets its core functional requirements. All queries produced accurate and expected outputs, and relational queries successfully combined data across multiple tables. The validation tests confirmed that incorrect data inputs are rejected, ensuring data integrity. Additionally, the use of real query outputs provides strong evidence that the system operates correctly in practical scenarios.

Limitations of Testing

Despite the effectiveness of the testing, there are some limitations. The testing focused mainly on functional correctness and did not include performance testing, such as evaluating system speed under large datasets. Additionally, security testing was not conducted, meaning potential vulnerabilities were not assessed. The testing was also performed manually, which may reduce efficiency and increase the chance of missing edge cases. These limitations indicate that while the system works correctly, further testing is required for real-world deployment.

(Connolly and Begg, 2014)

Limitation	Explanation	Impact
Limited edge case testing	Few extreme values tested	May miss rare system errors
No performance testing	System not tested under heavy load	Unknown scalability
No security testing	No attack or vulnerability testing	Potential risks remain
Manual testing only	No automation tools used	Less efficient and repeatable

Suggested Improvements “Testing”

To enhance the effectiveness of testing, several improvements can be implemented. Automated testing tools can be used to perform repeated tests efficiently and reduce human error. Performance testing should be conducted using large datasets to evaluate system scalability. Security testing, including attempts to bypass validation or simulate attacks, would strengthen system protection. Additionally, expanding test cases to include more edge scenarios would improve reliability.

Improvement	Description	Benefit
Automated testing	Use scripts or tools for repeated testing	Increases efficiency
Performance testing	Test system with large data volumes	Ensures scalability
Security testing	Simulate attacks or misuse	Improves system protection
Extended validation tests	Include more edge cases	Enhances reliability

The testing process was effective in validating the core functionality of the database system, demonstrating accurate data processing and strong relational integrity. However, to achieve a fully robust and production-ready system, additional testing methods such as performance, security, and automated testing should be implemented.

Critical Evaluation of the Hospital Database System “Design Depth and Decision Impact”

Determining the effectiveness of the developed database system is essential to evaluate how well it satisfies both user and system requirements. This section critically analyses the system’s performance, highlighting its strengths, identifying its limitations, and proposing realistic improvements to enhance functionality, security, and overall efficiency in a real-world hospital environment.

Evaluation Against User Requirements

The developed database system successfully addresses the core user requirements of managing hospital data, including patients, doctors, appointments, departments, and resources. The system provides a functional user interface that allows users to perform essential operations such as inserting, updating, and deleting records. Additionally, the use of relational tables ensures that data is logically connected and easy to manage. However, while the system meets most functional needs, it remains basic in terms of usability and lacks advanced features expected in real-world systems.

Requirement	Implemented?	Evidence (Figures)	Evaluation
Manage patient data	Yes	Fig 53–54	Accurate and well-structured
Manage doctor data	Yes	Fig 47–48	Efficient and organized
Appointment system	Yes	Fig 43–45	Supports scheduling effectively
Data relationships	Yes	Fig 41 + FK figures	Strong relational integrity
User interface (forms)	Yes	UI screenshots	Functional but basic
Multi-table queries	Yes	Fig 59–69	Provides useful insights

Evaluation Against System Requirements

From a system perspective, the database meets key technical requirements such as maintaining data integrity, enforcing relationships through foreign keys, and supporting query operations. The implementation of validation ensures that incorrect data cannot be inserted, improving reliability. However, the system lacks advanced performance optimization, security mechanisms, and automation, which limits its scalability and suitability for large-scale deployment.

System Requirement	Status	Evaluation
Data integrity	Achieved	Foreign keys enforce relationships correctly
Data validation	Achieved	Prevents invalid inputs (NurseID error)
Query functionality	Achieved	Supports SELECT, JOIN, WHERE, ORDER BY
Performance	Moderate	No indexing or optimization applied
Security	Basic	Only authentication, no advanced protection
Scalability	Limited	Suitable for small systems only

Strengths, Weaknesses, and Suggested Improvements

Category	Aspect	Explanation	Impact	Suggested Improvement	
Strength	Relational Structure	Tables are well connected using foreign keys	Ensures data consistency	Maintain and expand relationships	
	Functional UI	Forms allow Insert, Update, Delete operations	Easy interaction for users	Improve design for usability	
	Query Implementation	Multiple queries (JOIN, WHERE, ORDER BY)	Supports decision-making	Add more advanced queries	
	Data Validation	Incorrect data is rejected	Prevents data corruption	Expand validation rules	
Weakness	Security	No encryption or role-based access	Risk of unauthorized access	Implement encryption & RBAC	
	Limited Validation	Only data type validation used	Possible logical errors	Add range and format validation	
	No Automation	Maintenance tasks are manual	Inefficient in real-world use	Implement scheduled processes	
	Basic UI	Interface is simple and not user-friendly	Limits usability	Enhance UI/UX design	
	No Aggregation Queries	No COUNT, SUM, AVG used	Limited analytical capability	Add analytical queries	

	No Backup System	No real backup implementation	Risk of data loss	Add automated backup system	
Improvement	Advanced Security	Add encryption and authentication layers	Protect sensitive data	Use hashing and secure storage	
	Performance Optimization	Use indexing and query optimization	Faster system response	Improve scalability	
	Analytics Features	Add dashboards and reports	Better decision-making	Use aggregate queries	
	Logging System	Track user actions and errors	Improve monitoring	Add logging mechanism	

(Bertino and Sandhu, 2005; Karwin, 2017)

Overall

the database system is effective in meeting its primary objectives, particularly in managing hospital data and supporting essential operations such as data entry, retrieval, and relational queries. Functional requirements are fully satisfied; however, non-functional aspects such as security, performance optimization, and scalability are only partially met. Therefore, while the system is suitable for small-scale or academic use, further enhancements are required for it to meet professional industry standards.

(Connolly and Begg, 2014; Bertino and Sandhu, 2005)

Professional **Technical** and User **Documentation** for the Hospital Database System

Firstly: Technical Documentation

System Overview

Presenting the hospital database system, it is designed to efficiently manage and organize key hospital operations, including patients, doctors, appointments, departments, nurses, laboratories, and rooms. The system aims to improve data accuracy, reduce redundancy, and provide fast access to critical information for administrative and medical staff.

The system operates using a relational database implemented in MySQL Workbench, combined with a user interface that allows users to interact with the database through forms and data grids. Each module within the system is responsible for handling a specific type of data:

- ❖ The **Patient Module** manages patient records, including personal details, medical assignments, and related information.
- ❖ The **Doctor Module** maintains doctor profiles, specialties, and department associations.
- ❖ The **Appointment Module** handles scheduling, tracking, and management of patient appointments.
- ❖ The **Department Module** organizes hospital departments and their administrative structure.
- ❖ The **Nurse Module** manages nursing staff details, work schedules, and room assignments.
- ❖ The **Lab Module** oversees laboratory operations, including staff allocation and working hours.
- ❖ The **Room Module** controls room information, allocation, and capacity within the hospital.

Database Design Documentation

The database is designed using a relational model where multiple tables are connected through primary keys (PK) and foreign keys (FK). This structure ensures data consistency and prevents duplication.

Each table represents a specific entity within the hospital system, and relationships between tables allow for meaningful data retrieval. For example, the **patient table** is linked to doctors, departments, nurses, and rooms, enabling the system to display complete patient information across different modules.

Foreign keys are used to enforce relationships between tables, such as:

- ❖ DoctorID linking patients to doctors
- ❖ DepaID linking doctors to departments
- ❖ RoomID linking nurses and patients to rooms

Fig1:

ER diagram for the current Hospital management system

Table Dictionary

Table Name	Field Name	Data Type	Description	Constraint
patient	PatientID	INT	Unique identifier for each patient	PK, AUTO INCREMENT
	PatientName	VARCHAR	Name of the patient	NOT NULL
	PatientTelephone	VARCHAR	Patient contact number	-
	PatientAddress	VARCHAR	Patient address	-
	PatientAge	INT	Age of the patient	-
	PatientGender	VARCHAR	Gender of the patient	-
	PatientBloodgrp	VARCHAR	Blood group of the patient	-
	DeparmentID	INT	Assigned department	FK
	DoctorID	INT	Assigned doctor	FK
	RoomID	INT	Assigned room	FK
	NurseID	INT	Assigned nurse	FK
doctors	DoctorID	INT	Unique doctor identifier	PK
	DoctorName	VARCHAR	Doctor name	NOT NULL
	DoctorAge	INT	Age of doctor	-
	DoctorEmail	VARCHAR	Email address	-
	DoctorSalary	INT	Salary of doctor	-
	DoctorOfficesHrs	INT	Working hours	-
	DepaID	INT	Department ID	FK

appointment	AppNo	INT	Unique appointment number	PK
	PatientID	INT	Linked patient	FK
	DoctorID	INT	Linked doctor	FK
	AppDate	DATE	Appointment date	-
	AppFees	INT	Appointment fees	-
department	DepaID	INT	Unique department ID	PK
	DepaName	VARCHAR	Department name	NOT NULL
	DepaHead	VARCHAR	Head of department	-
nurse	NurseID	INT	Unique nurse identifier	PK
	NurseName	VARCHAR	Name of nurse	NOT NULL
	NurseAge	INT	Age of nurse	-
	NurseWorkinghrs	INT	Working hours	-
	NurseSalary	INT	Salary of nurse	-
	RoomID	INT	Assigned room	FK
rooms	RoomID	INT	Unique room identifier	PK
	RoomNo	INT	Room number	-
	RoomFloor	INT	Floor number	-
	RoomSize	INT	Room capacity	-
lab	LabID	INT	Unique lab identifier	PK
	LabManager	VARCHAR	Lab manager name	-
	LabWorkinghrs	INT	Working hours	-
	DoctorID	INT	Assigned doctor	FK
accounting	AccID	INT	Unique accountant ID	PK
	AccountantName	VARCHAR	Accountant name	-

	AccountantWorkinghrs	INT	Working hours	-
	AccountantEmail	VARCHAR	Email address	-

SQL Scripts

The system uses Structured Query Language (SQL) to retrieve and manipulate data stored in the database, as u can see in Fig70:

```

36 • SELECT * FROM hosdb.patient;
37
38 • SELECT * FROM hosdb.patient WHERE PatientAge > 30;
39
40 • SELECT d.DoctorName, dept.DepaName
41 FROM hosdb.doctors d
42 JOIN hosdb.department dept ON d.DepaID = dept.DepaID;
43
44 • SELECT n.NurseName, r.RoomNo
45 FROM hosdb.nurse n
46 JOIN hosdb.rooms r ON n.RoomID = r.RoomID;
47

```

Fig70

Installation and Configuration Guide

To set up the system, the following steps followed:

- ❖ Install **MySQL Workbench**
- ❖ Create a new database connection
- ❖ Create a schema named **hosdb**
- ❖ Execute SQL scripts to create tables
- ❖ Insert sample data into tables
- ❖ Launch the system user interface
- ❖ Connect the application to the database

Backup and Maintenance Instructions

To ensure system reliability and prevent data loss, regular maintenance should be performed:

- ❖ Perform regular database backups using MySQL export tools
- ❖ Monitor system performance and query execution
- ❖ Check for data consistency and integrity

- ❖ Update database structure when needed
- ❖ Maintain logs for system activity

Secondly: User Documentation

User Introduction

Understanding how to use the hospital database system is essential for ensuring smooth daily operations. This system is designed for hospital staff to manage patient records, doctor information, appointments, and other related data through a simple and interactive user interface. The system allows users to perform key operations such as adding, updating, deleting, and viewing records efficiently.

Accessing the System

- ❖ Launch the application from your device
- ❖ The main interface (home screen) will appear
- ❖ Select the required module (Patient, Doctor, Appointment, and so on.)
- ❖ The selected form will open for interaction

Fig6:

Homepage Interface Overview

Adding New Records

- ❖ Open the required module (e.g., Patient Form)
- ❖ Enter all required details in the input fields
- ❖ Click the Insert button
- ❖ The data will be stored and displayed in the DataGrid

Fig11 & 12:

Insert Operation (Adding New Doctor)

Updating Existing Records

- ❖ Select the record to update
- ❖ Edit the required fields
- ❖ Click the Update button
- ❖ Changes will be saved and reflected in the system

Fig13 & 14:

Update Operation (Modifying Existing Data)

Deleting Records

- ❖ Select the record to delete
- ❖ Click the Delete button
- ❖ The record will be permanently removed

Fig21 & 22:

Delete Operation Scenario (Correcting Data Entry)

Viewing and Searching Data

- ❖ Data is displayed in a DataGridView
- ❖ Queries can filter, sort, or combine data
- ❖ Results are shown instantly

Fig10:

Or Any interface/Query Figure

Common Errors and Solutions

Error	Cause	Solution
Invalid NurseID	Incorrect data type entered	Enter numeric value only
Missing fields	Required input not provided	Fill all required fields
No data displayed	Incorrect query	Check query conditions
Record not updated	Wrong ID selected	Verify correct record ID

Security Guidelines for Users

- ❖ Do not share login credentials
- ❖ Always enter correct and valid data
- ❖ Avoid modifying critical records without permission
- ❖ Log out or close the system after use
- ❖ Report any system errors to the administrator

Modelled Architecture and Data Flow of the Hospital Database System

Modelling how the hospital database system operates is essential to understand the interaction between its different components and how data flows through the system. This section presents a structured overview of the system architecture, supported by data flow diagrams and flowcharts, to clearly illustrate how user inputs are processed, stored, and retrieved within the system.

System Architecture Overview

The hospital database system follows a structured three-layer architecture that ensures separation of concerns and efficient system operation. The system consists of a user interface layer, an application logic layer, and a database layer.

The **user interface (UI)** allows users to interact with the system through forms and DataGrid views. The **application layer** processes user inputs, validates data, and generates SQL queries. The **database layer**, implemented using MySQL Workbench, stores and manages all hospital data. (*Microsoft, 2022*)

Layer	Description
User Interface	Forms and DataGrid used for data entry and display
Application Layer	Handles logic, validation, and SQL execution
Database Layer	MySQL database storing structured hospital data

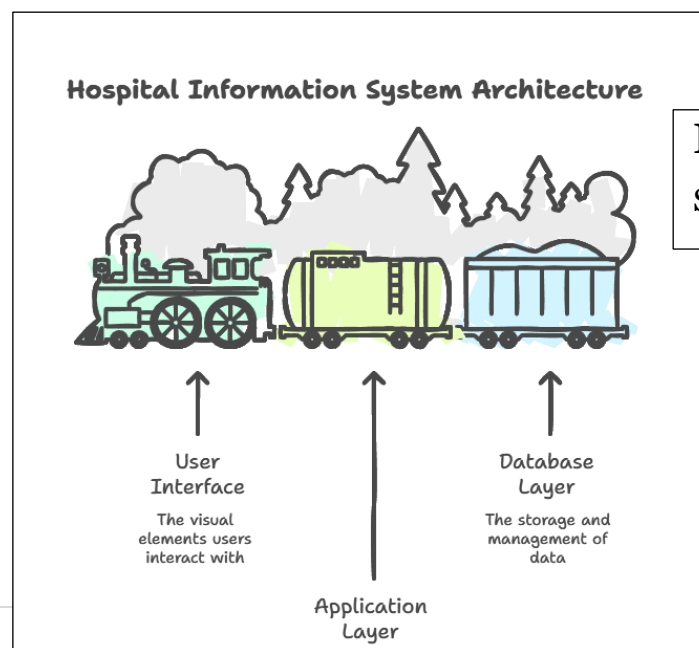


Fig70:
System Architecture

Data Flow Diagram

The Data Flow Diagram (DFD) in **Figure 71** illustrates how data moves through the hospital database system, showing the interaction between the user, user interface, application layer, and database.

The process begins when the **hospital staff (user)** interacts with the system by entering data or requesting information. This input is received through the **User Interface (UI)**, which includes forms and DataGrid components used for data entry and display.

The entered data is then passed to the **Application Layer**, where validation and processing take place. At this stage, the system ensures that the input is correct and converts user actions into structured SQL queries.

These SQL queries are sent to the **MySQL database**, where data is stored, retrieved, or modified across multiple tables such as patient, doctor, and appointment. The database processes the request and returns the results back to the application layer.

The **Application Layer** then processes the query results and prepares them for presentation. Finally, the processed data is sent back to the **User Interface**, where it is displayed to the user in a clear and structured format.

This continuous flow ensures that the system operates efficiently, maintaining data integrity while providing accurate and real-time information to users. (*Visual Paradigm, n.d.*)

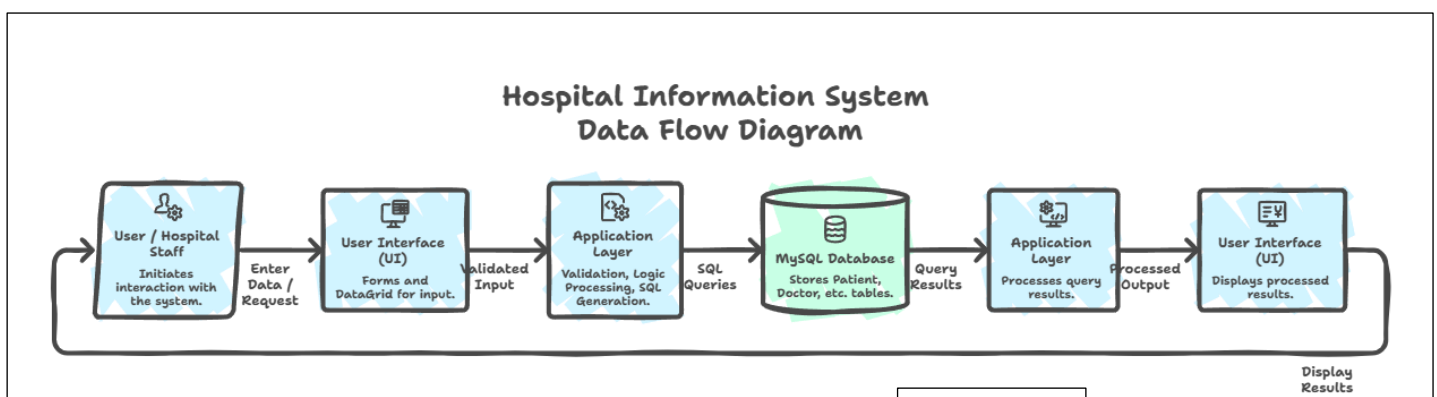


Fig71:
DFD

Fig72

System Flowchart

The flowchart in **Figure 72** illustrates the sequence of operations performed within the hospital database system, demonstrating how user inputs are processed and how the system responds to different conditions.

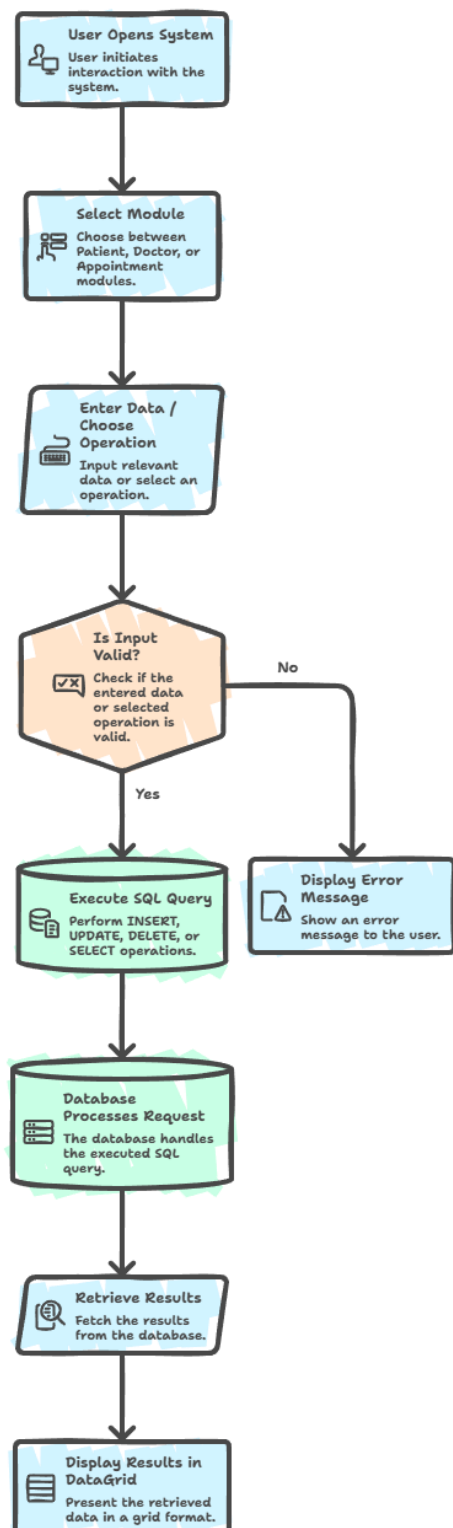
The process begins when the **user opens the system**, initiating interaction through the user interface. The user then selects a specific module, such as patient, doctor, or appointment management, depending on the required operation.

After selecting the module, the user proceeds to **enter data or choose an operation**, such as inserting, updating, deleting, or retrieving records. The system then performs a validation check to ensure that the input data or selected operation is correct.

If the input is invalid, the system displays an **error message**, preventing incorrect data from being processed. If the input is valid, the system proceeds to **execute the corresponding SQL query**, which may include operations such as INSERT, UPDATE, DELETE, or SELECT.

The request is then handled by the **database**, where the query is processed and the relevant data is retrieved or modified. Once processing is complete, the system **retrieves the results** from the database and displays them to the user in a structured format using the DataGrid. (*Visual Paradigm, n.d.*)

System Operation FlowChart



Detailed Explanation of System Operation

The system operates through continuous interaction between the user interface, application logic, and database. When a user performs an action such as adding a new patient, the request is captured through the user interface form. The application layer then validates the input to ensure that the data is correct and complete. If validation is successful, an SQL query is executed to insert the data into the database.

For retrieval operations, the system executes SELECT queries to fetch data from one or multiple tables. These queries may include conditions (WHERE), sorting (ORDER BY), or relationships (JOIN) to produce meaningful results. The retrieved data is then displayed to the user in a structured format using the DataGrid.

Similarly, update and delete operations follow the same flow, where user actions are translated into SQL queries and executed within the database. This ensures consistency, accuracy, and efficient handling of hospital data.

Overall System Operation Summary

The system demonstrates a well-organized flow of data from user input to final output. The integration between the interface and the database ensures that all operations are processed efficiently and accurately. The use of structured queries and relational tables allows the system to handle complex data relationships, making it suitable for managing hospital operations effectively.

Deep Evaluation and Justified Improvements of the Hospital Database System

Determining the effectiveness of the hospital database system requires a critical evaluation of its design, performance, and suitability for real-world use. This section provides a concise but in-depth assessment, supported by comparative analysis and justified improvements.

Detailed Evaluation Based on Implemented System

Aspect	Implementation in Your System	Evidence (Figures)	Limitation Identified	Advanced / Alternative Solution	Impact
Data Structure	Relational database using MySQL with structured tables (patient, doctor, appointment, etc.)	Fig 41–54	Fixed schema (less flexible for changes)	NoSQL / Hybrid databases	More flexible data handling
Table Relationships	Foreign keys used (Patient–Doctor–Department–Room–Nurse) ensuring data linkage	Fig 43–54 (FK screenshots)	Limited constraint depth (no advanced rules)	Advanced constraints / cascading rules	Stronger integrity
Basic Queries	SELECT queries used to retrieve full data (e.g., all patients, doctors)	Fig 56–58	Limited analytical capability	Advanced queries with conditions and joins	Better insights
Filtering (WHERE)	Data filtered (e.g., PatientAge > 30)	Fig 56	Only simple conditions used	Complex filtering with multiple conditions	More precise results
Sorting (ORDER BY)	Sorting doctors and nurses (ASC / DESC)	Fig 57–58	Limited use in analysis	Multi-column sorting	Better data organization
Multi-table Queries	Data combined from multiple tables using WHERE (joins manually)	Fig 59–64	Uses old-style joins (less efficient)	JOIN syntax (INNER JOIN, LEFT JOIN)	Improved readability & performance
JOIN Queries	Implemented proper JOIN (doctor + department)	Fig 69	Only basic JOIN used	Advanced joins (LEFT JOIN, FULL JOIN)	More complete data retrieval
Data Validation	System prevents incorrect data types (e.g., NurseID error)	Fig 66–67 and Figs M.1/2/3/4/5/6	Limited to basic type validation	Advanced validation rules (range, format)	Improved accuracy

Data Display	Results displayed in DataGrid UI clearly	Fig 42–54	Basic UI only	Interactive dashboards	Better user experience
CRUD Operations	Insert, Update, Delete operations implemented through system	(Mysql Workbench screenshots)	No logging or tracking	Audit logs / history tracking	Better monitoring
Performance	Queries execute correctly on small dataset	All query figs	No indexing or optimization	Indexed database	Faster execution
Security	Basic authentication (password login)	Fig 39	No encryption or access control	Role-Based Access Control (RBAC)	Secure system
Scalability	Works well for small dataset (hospital demo)	Overall system	Not scalable for large systems	Cloud database systems	Handles large data
Automation	Manual operations only	—	No triggers or automation	Triggers / scheduled jobs	Reduced manual effort

Then Justified Improvements based on my system

Improvement	Why It Is Needed (Justification)	Expected Impact
Add Encryption	Patient data is sensitive and currently unprotected	Improves data security and confidentiality
Role-Based Access Control (RBAC)	All users currently have same access level	Prevents unauthorized actions
Use Indexing	Queries may slow down with large data	Improves performance and speed
Add Aggregate Queries (SUM, COUNT)	Current queries lack analytical insights	Supports decision-making
Implement Backup System	No backup strategy exists	Prevents data loss
Add Input Validation Rules	Only basic validation exists	Reduces incorrect data entry

(Connolly and Begg, 2014; Bertino and Sandhu, 2005; Bertino and Sandhu, 2005; Karwin, 2017; Gunjal, 2003)

Final Word

The system demonstrates a solid foundation in relational database implementation and successfully meets its primary objectives. However, it remains a basic system when compared to real-world solutions due to limitations in scalability, security, and performance optimization. With the implementation of the proposed improvements, the system can evolve into a more robust, secure, and scalable solution suitable for professional environments.

Sources and References used

Microsoft (2022) *Database design basics*. Available at: <https://support.microsoft.com/en-us/office/database-design-basics-eb2159cf-1e30-401a-8084-bd4f9c9ca1f5> (Accessed: 3 May 2026).

Visual Paradigm (n.d.) *Data Modeling: Conceptual vs Logical vs Physical Data Model*. Available at: <https://online.visual-paradigm.com/knowledge/visual-modeling/conceptual-vs-logical-vs-physical-data-model> (Accessed: 3 May 2026).

Batra, D. and Davis, J. (1992) 'Conceptual data modelling in database design: similarities and differences between expert and novice designers', *International Journal of Man-Machine Studies*, 37(1), pp. 83–101. Available at: [https://doi.org/10.1016/0020-7373\(92\)90092-Y](https://doi.org/10.1016/0020-7373(92)90092-Y) (Accessed: 3 May 2026).

Bertino, E. and Sandhu, R. (2005) 'Database security: concepts, approaches, and challenges', *IEEE Transactions on Dependable and Secure Computing*, 2(1), pp. 2–19. Available at: <https://ieeexplore.ieee.org/abstract/document/1416861> (Accessed: 3 May 2026).

Chandra, A.K. (1988) 'Theory of database queries', in *Proceedings of the Seventh ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*, pp. 1–9. Available at: <https://dl.acm.org/doi/10.1145/308386.308396> (Accessed: 3 May 2026).

Gunjal, B. (2003) 'Database System: Concepts and Design', *Proceedings of 24th IASLIC–SIG2003*. Available at: https://www.researchgate.net/publication/257298522_Database_System_Concepts_and_Design (Accessed: 3 May 2026).

Churcher, C. (2012) *Beginning Database Design: From Novice to Professional*. 2nd edn. New York: Apress.

Connolly, T. and Begg, C. (2014) *Database Systems: A Practical Approach to Design, Implementation and Management*. 6th edn. Harlow: Pearson.

Flejoles, R.P. (2018) *Database Theory and Application*. Oakville: Arcler Press.

Karwin, B. (2017) *SQL Antipatterns: Avoiding the Pitfalls of Database Programming*. Dallas: Pragmatic Programmers.

Kroenke, D. and Auer, D. (2012) *Database Concepts*. 6th edn. Boston: Pearson.

AI tools used in this report “support idea development, writing structure, and language refinement”

- Google DeepMind. (2026) Gemini Advanced
- OpenAI. (2026) ChatGPT (GPT-5) pro
- Napkin AI
- SciSpace
- NotebookLM

The End